A Spark-Based Data Preparation Pipeline for Flight Delay Analysis

Author:

Yuqi XIN

Licheng XIONG



A Spark-Based Data Preparation Pipeline for Flight Delay Analysis

Abstract

This project aims to use PySpark to clean, transform, and join flight data and weather data in order to construct a joint table (JT) that can support flight delay analysis and prediction. The project focuses on the complete data preparation pipeline from raw CSV files to a structured analytical table, rather than on advanced model training.

More specifically, the project consists of four main stages. First, the flight data are read and cleaned, including type conversion, removal of cancelled and diverted flights, construction of scheduled departure timestamps, derivation of scheduled arrival timestamps across time zones, and generation of label variables. Second, the weather data are read and cleaned by standardizing missing-value strings, cleaning numeric fields, constructing weather observation timestamps, and linking airports to weather stations through an airport–station mapping table. Third, a joint table is built from the cleaned flight and weather data: for each flight, weather observations from 0 to 12 hours before departure and before arrival are extracted and then reshaped from long format to wide format, producing the final JT stored by month. Fourth, a simple downstream validation experiment is carried out on the constructed JT, where basic temporal features, flight features, and weather features are used as classification inputs to verify that the output of the data preparation pipeline can indeed be used for flight delay prediction tasks.

From an implementation perspective, the entire project is built with the PySpark DataFrame API. To improve maintainability, the cleaned flight data and cleaned weather data are first written separately as partitioned Parquet files, and the joint construction stage then reads them month by month. In the weather stage, a buffer-month reading strategy is also used to handle cross-month weather matching for flights near month boundaries.

The results show that the pipeline is able to transform heterogeneous raw data into a unified structured analytical table and provide directly usable input for subsequent delay prediction models. At the same time, the experiments also show that the true performance bottlenecks are not data reading itself, but rather large-scale weather matching, long-to-wide transformation, and final output writing, which reflects the engineering complexity of data preparation tasks in big-data environments.

Keywords: PySpark; Flight Delay; Weather Matching; Data Preparation

1. Introduction

Flight delay analysis usually cannot rely only on flight operation information itself. It also needs to consider weather conditions, airport context, and temporal factors. As a result, the raw data often come from multiple sources, with inconsistent field formats, time granularity, and location identifiers, and therefore cannot be used directly for analysis or prediction. Before any downstream study can be conducted, a reliable data preparation pipeline must first be built to transform these heterogeneous raw datasets into a unified, structured, and computable joint table.

This project addresses this goal by using PySpark to process flight data, weather data, and an airport–weather-station mapping table, and then constructing a flight-level joint table. Rather than emphasizing direct modeling, the project focuses on the data engineering process itself and mainly addresses the following questions:

- How can raw flight data be cleaned while retaining analyzable records?
- How can raw weather data be cleaned and transformed into standardized observation timestamps?
- How can airports and weather stations be linked through a mapping table?
- How can timestamps on the flight side and the weather side be made comparable?
- How can multiple hours of weather observations before departure and arrival be matched to each flight?
- How can the weather long table be reshaped into a wide table suitable for downstream modeling?

Therefore, this project is essentially a typical big-data preparation and data engineering task, whose core lies not in complex models themselves, but in data cleaning, time alignment, multi-table joining, and final result construction.

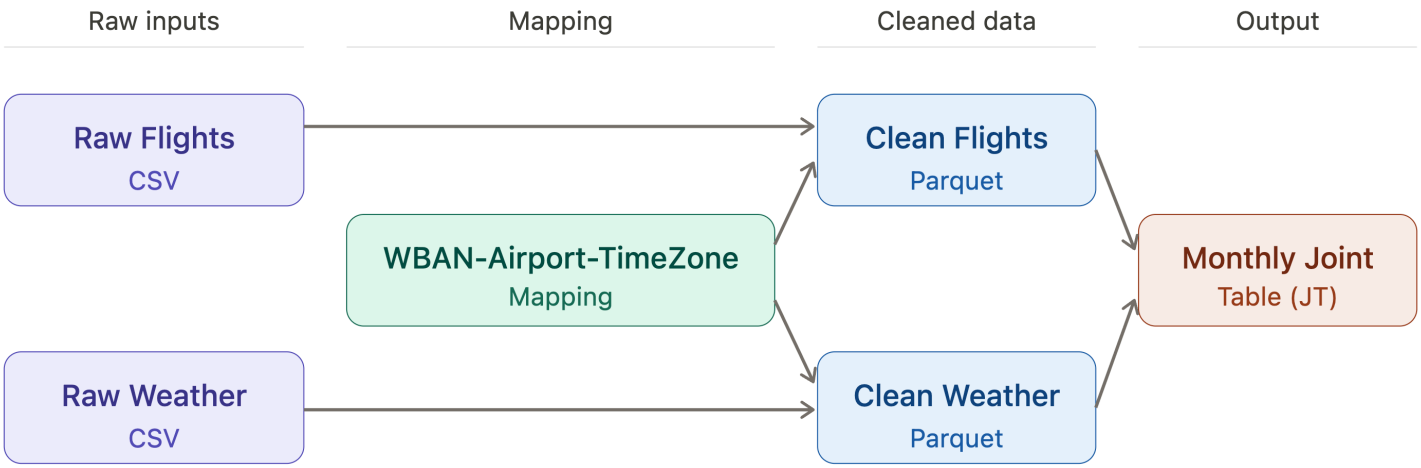


Figure 1. Overall pipeline of the project

2. Data Sources and Overall Approach

2.1 Data Sources

This project uses three types of data:

1. **Raw flight data (Flights)**
   These data contain information such as flight date, scheduled departure time, carrier ID, flight number, origin airport, destination airport, elapsed flight time, and arrival delay.
2. **Raw weather data (Weather)**
   These data contain weather station identifier WBAN, observation date, observation time, visibility, temperature, humidity, wind speed, precipitation, pressure, and other observation fields.
3. **Airport–weather-station mapping table (`wban_airport_timezone.csv`)**
   This table is used to map airport identifier `AirportID` to weather station identifier `WBAN` , and also provides time zone information `TimeZone` .

In the flight data, location is identified by airport ID, whereas in the weather data, location is identified by WBAN station ID. Since the two datasets cannot be joined directly, the mapping table is required to connect them. In addition, the time fields in the flight and weather datasets are also inconsistent, so both sides must first be converted into standard timestamps before time-based matching can be performed.

2.2 Overall Processing Logic

From an implementation perspective, the whole project is divided into two major stages.

Stage 1: Data Cleaning

- Clean the raw flight data and write the result to `clean/Flights`
- Clean the raw weather data and write the result to `clean/Weather`

Stage 2: Joint Table Construction

- Read the cleaned flight data for a target month
- Read the cleaned weather data for the current month together with buffer months before and after
- Construct target timestamps for departure-side and arrival-side weather matching for each flight
- Extract weather observations from 0 to 12 hours before the target time
- Transform weather long tables into wide tables
- Write the final monthly JT

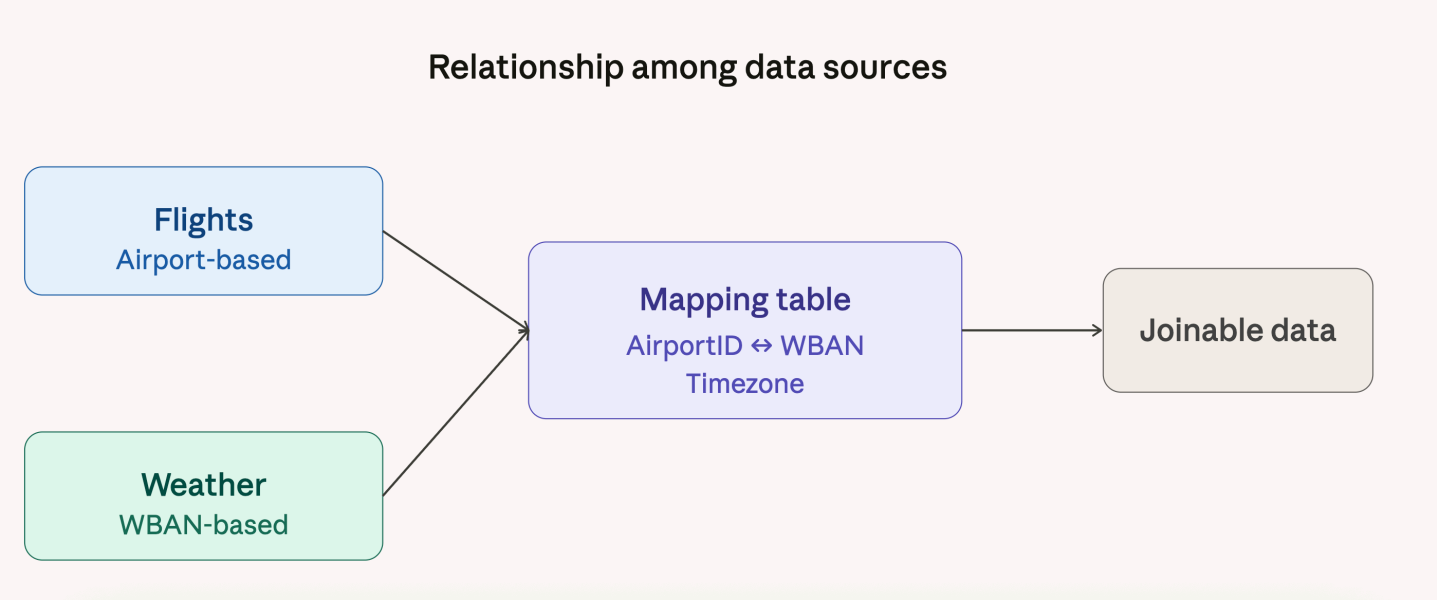


Figure 2. Relationship among flight data, weather data, and the mapping table

3. System Implementation and Code Structure

The implementation is divided into three main modules:

1. `project_clean.ipynb`
   Used to clean the flight data and weather data, and write them as partitioned Parquet files.
2. `project_join_test.ipynb`
   Used to read the cleaned data and construct the JT for selected months.

3. `figures.ipynb`
   Used to generate the flowcharts, structure diagrams, and schematic figures included in the report.

In addition, the project also includes `JOIN_TOTAL_ML.ipynb`, which performs a simple machine learning baseline experiment on the constructed JT to verify that the joint table can be directly used for downstream prediction tasks.

In terms of execution flow, the project adopts a staged design of "clean first, then join," rather than going directly from raw CSV files to the final JT in a single run. This design has two advantages: first, it makes debugging and stage-wise validation easier; second, it allows intermediate results to be reused, avoiding repeated cleaning of raw data during later join debugging.

4. Flight Data Cleaning Pipeline

4.1 Raw Data Reading and Type Conversion

The flight data are first read from all CSV files in the raw directory. After reading, the code removes the unnecessary column `_c12` and performs type conversion for key fields. For example:

- `FL_DATE` is converted to date type;
- `CRS_DEP_TIME` is kept as string for later zero-padding and splitting;
- `ARR_DELAY_NEW`, `CANCELLED`, `DIVERTED`, `CRS_ELAPSED_TIME`, `WEATHER_DELAY`, and `NAS_DELAY` are converted to numeric types;
- carrier ID, flight number, origin airport ID, and destination airport ID are kept as strings.

The purpose of this step is to provide a stable type foundation for later timestamp construction, filtering, and label generation.

4.2 Missing-Value Check and Basic Filtering

After type conversion, the code performs missing-value checks for the fields. For string fields, both empty strings and nulls are checked; for non-string fields, nulls are checked. This step is mainly used to understand the quality of the raw data and to support later filtering decisions.

Then only flight records satisfying the following conditions are retained:

- `CANCELLED == 0.0`
- `DIVERTED == 0.0`
- `CRS_DEP_TIME` is not null
- `CRS_ELAPSED_TIME` is not null

In other words, the project only keeps normal flights that were actually operated and for which key time information can be constructed.

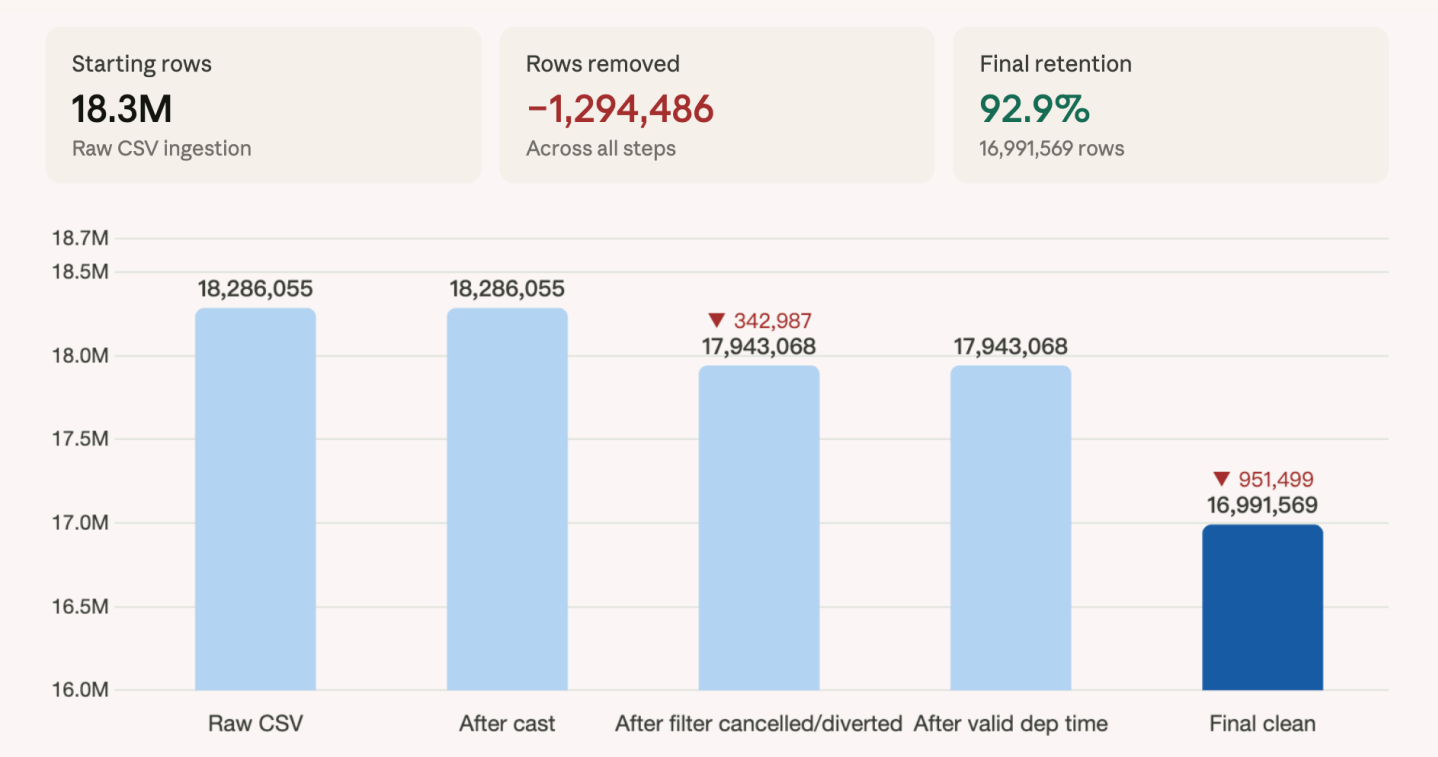


Figure 3. Row-count changes across major flight-cleaning steps

4.3 Construction of Scheduled Departure Timestamp

To construct a standard scheduled departure timestamp, the code further processes `CRS_DEP_TIME`. The procedure is as follows:

1. Remove all non-digit characters;
2. Use `lpad` to make it a four-digit string;
3. Split it into hour and minute;
4. Filter invalid time values, where hour must be between 0 and 23 and minute between 0 and 59;
5. Recombine `FL_DATE` with the extracted hour and minute to form the standard timestamp `scheduled_dep_ts`.

After this step, every flight has a uniformly formatted scheduled departure timestamp.

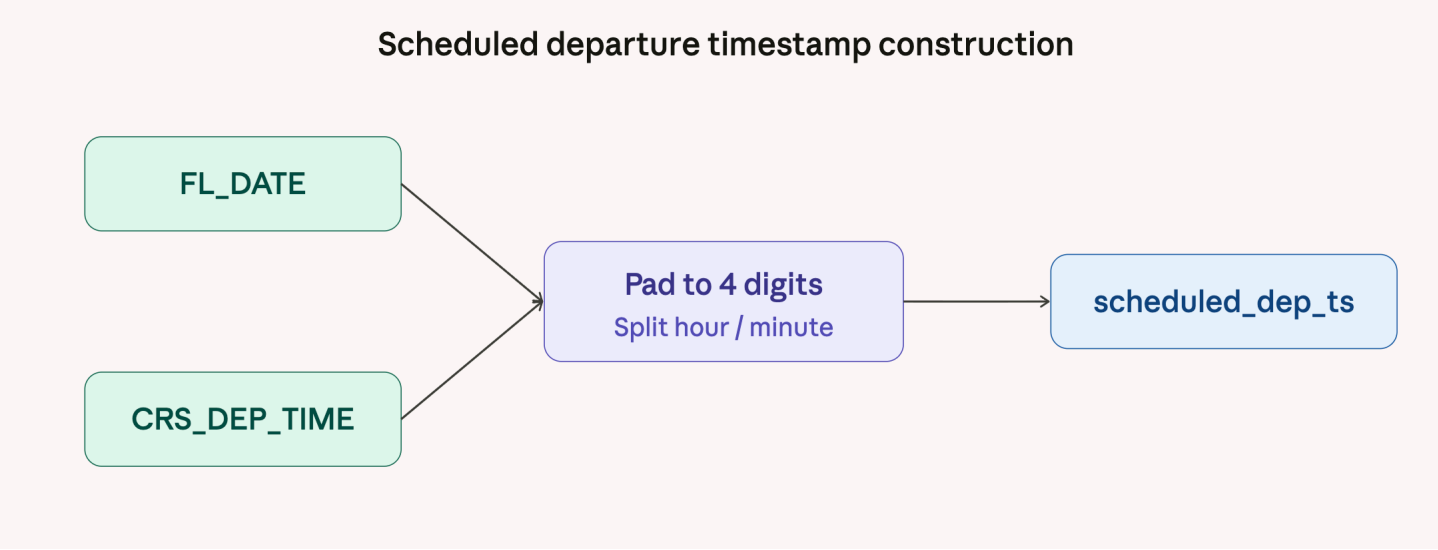


Figure 4. Construction logic of the scheduled departure timestamp

4.4 Scheduled Arrival Time Construction and Time Zone Handling

The raw data do not directly provide a unified destination-local scheduled arrival timestamp that can be used for weather matching, so the code derives it. The logic is as follows:

1. Use the mapping table to attach time-zone offsets for both origin and destination airports;
2. Compute `scheduled_arr_ts_origin_clock = scheduled_dep_ts + CRS_ELAPSED_TIME`, which gives the expected arrival time on the origin airport's local clock;
3. Compute `dest_tz − origin_tz` as the time-zone difference and apply it to the expected arrival time;
4. Obtain the expected arrival time in destination-local time as `scheduled_arr_ts_dest_local`.

If a record lacks either the origin or destination time-zone information, the destination-local arrival time cannot be computed reliably, so such records are removed.

This step is crucial because the arrival-side weather matching later depends exactly on `scheduled_arr_ts_dest_local`.

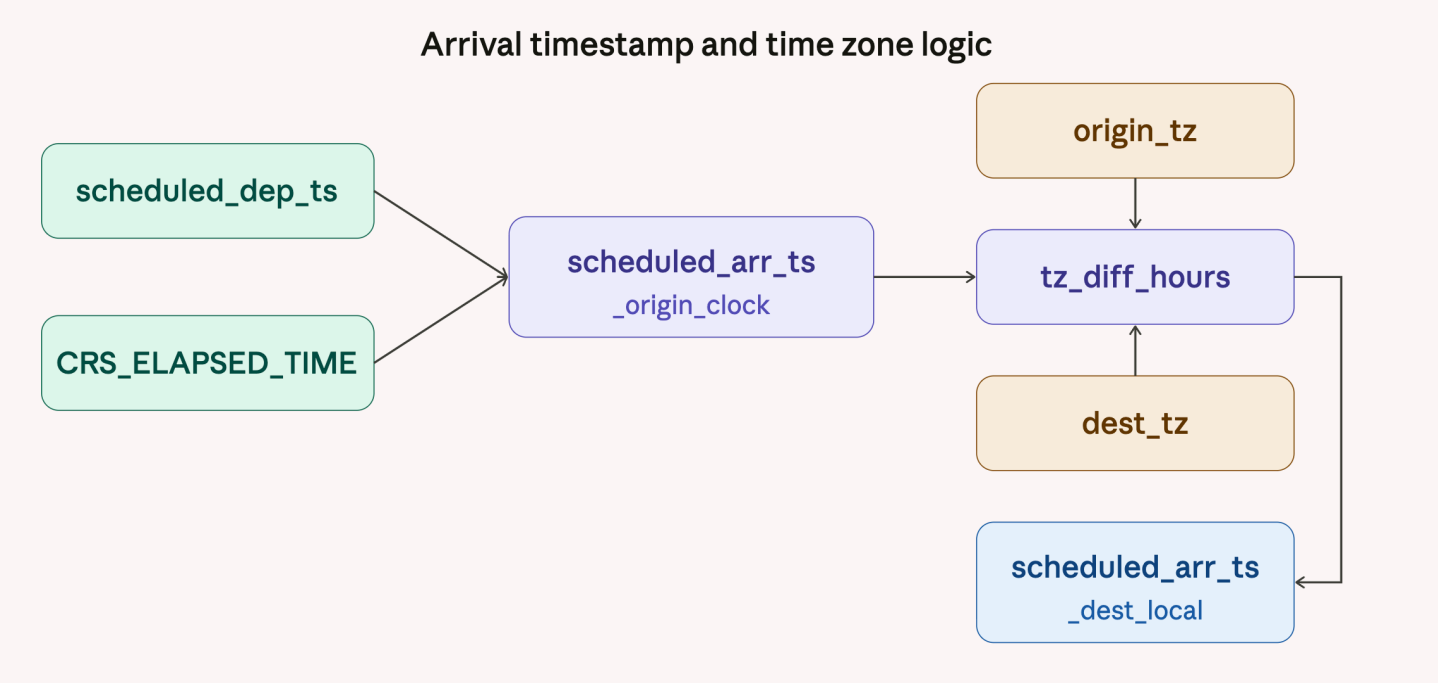


Figure 5. Logic of scheduled arrival time construction and time-zone adjustment

4.5 Derived Variables and Label Generation

After the main time fields are constructed, the code further generates a series of auxiliary and label variables, including:

- `flight_year`
- `flight_month`
- `flight_yyyymm`
- `arr_delay_min`
- `is_delayed_15`
- `is_delayed_60`
- `has_weather_delay`
- `has_nas_delay`

In addition, a unique key `flight_id` is generated based on date, carrier ID, flight number, origin airport, destination airport, and scheduled departure time, and this key is later used for table joins in the joint stage.

4.6 Output of Cleaned Flight Data

Finally, the cleaned flight data are reduced to a fixed set of fields and written as Parquet files partitioned by `flight_year` and `flight_month`, forming the `clean/Flights` dataset.

5. Weather Data Cleaning Pipeline

5.1 Raw Weather Data Reading and String Cleaning

The weather data are also read from all CSV files in the raw directory. Since many fields in the raw weather table are stored as strings and include blanks, extra characters, and missing markers such as `"M"`, a unified cleaning step is needed first.

The processing includes:

- applying `trim` to all string fields;
- replacing empty strings, all-whitespace strings, and `"M"` uniformly with `null`.

This prepares the data for later numeric standardization and timestamp construction.

5.2 Keeping Core Fields

To reduce later computational complexity, the code keeps only the core fields needed for subsequent joining and analysis, including:

- `WBAN`
- `Date`
- `Time`
- `SkyCondition`
- `Visibility`
- `WeatherType`
- `DryBulbFarenheit`
- `WetBulbFarenheit`
- `DewPointFarenheit`
- `RelativeHumidity`
- `WindSpeed`
- `WindDirection`
- `StationPressure`
- `SeaLevelPressure`
- `HourlyPrecip`
- `Altimeter`

5.3 Construction of Weather Observation Timestamp

To allow weather observations to be matched with flights by time, the code converts `Date` and `Time` into a standard timestamp. The procedure is as follows:

1. Convert `Date` into date type `obs_date` ;
2. Zero-pad `Time` into a four-digit string `Time_4` ;
3. Split it into `obs_hour` and `obs_minute` ;
4. Filter invalid dates and invalid times;
5. Recombine date and time to form the weather observation timestamp `obs_ts` .

As a result, the observation time in the weather dataset becomes a standard field that can directly participate in time comparison.

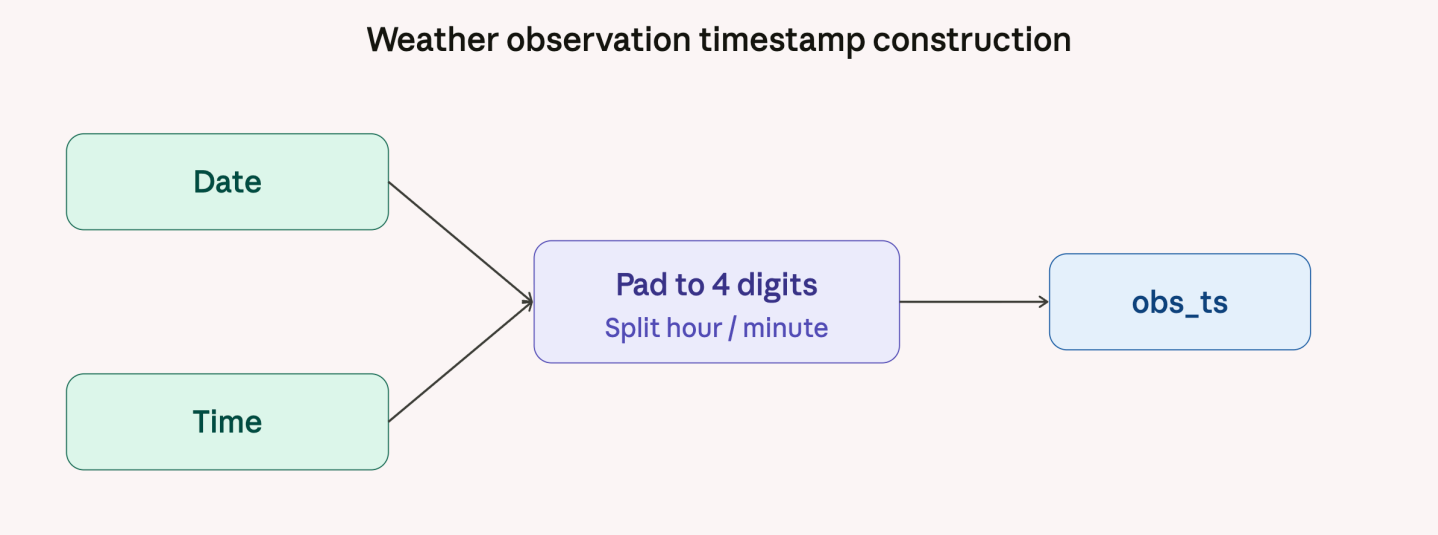


Figure 6. Construction logic of the weather observation timestamp

5.4 Numeric Field Cleaning

Many fields in the weather table are essentially numeric, but their raw format contains extra symbols or missing markers, so they cannot be directly cast to `double` . Therefore, the code defines a unified `clean_numeric` logic, mainly including:

- trimming leading and trailing spaces;
- using regular expressions to remove characters other than digits, decimal points, and minus signs;
- keeping nulls, empty strings, and `"M"` as missing values;
- finally casting the cleaned values to `double` .

This step is applied to the following fields:

- `Visibility`
- `DryBulbFarenheit`
- `WetBulbFarenheit`
- `DewPointFarenheit`
- `RelativeHumidity`
- `WindSpeed`
- `WindDirection`

- `StationPressure`
- `SeaLevelPressure`
- `HourlyPrecip`
- `Altimeter`

5.5 Joining Airport Information and Outputting Cleaned Weather Data

After time and numeric cleaning, the weather data still need to be joined with the mapping table so that `WBAN` can be converted into the corresponding `AirportID` and `TimeZone`. The code uses an inner join here, so only weather station records that can be successfully mapped to an airport are retained.

Then the code adds partition helper fields:

- `weather_year`
- `weather_month`
- `weather_yyyymm`

Finally, the cleaned dataset `weather_clean` is written as Parquet files partitioned by `weather_year` and `weather_month`.

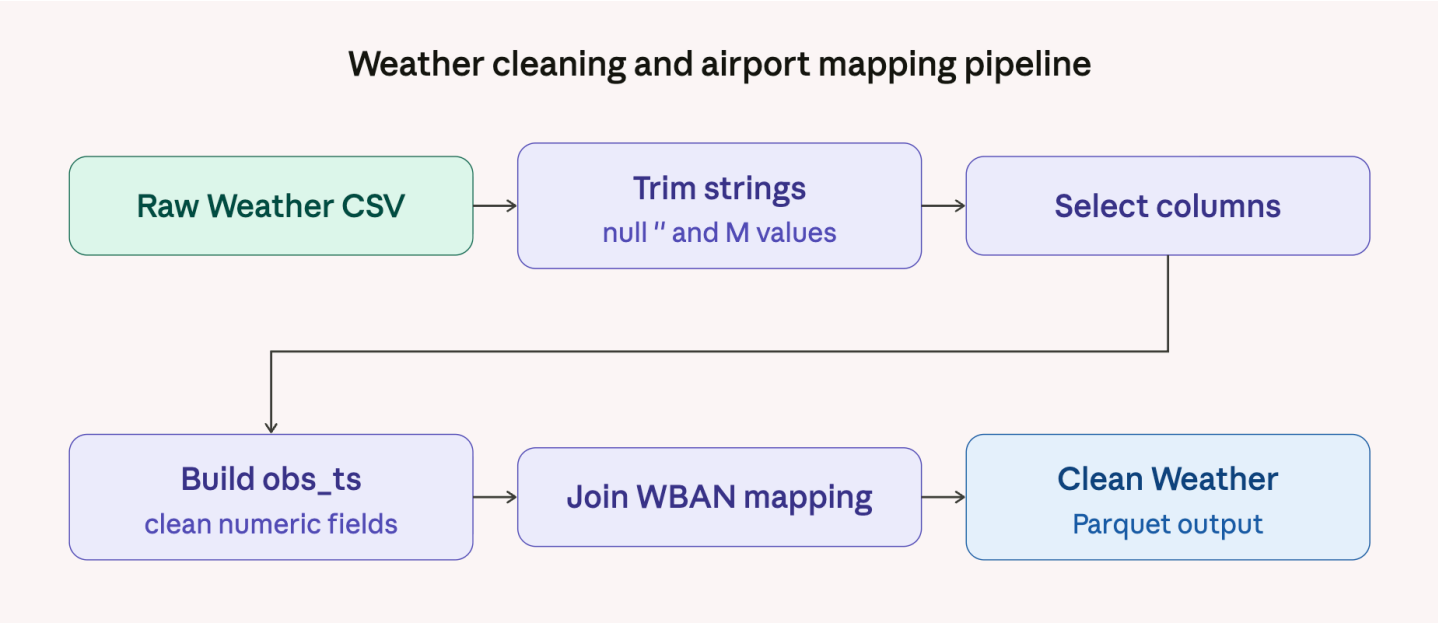


Figure 7. Weather-cleaning and airport-mapping pipeline

6. Joint Table Construction Pipeline

After the cleaned flight table and cleaned weather table are produced, the project further constructs the final monthly JT. This part is the most computationally intensive and engineering-heavy stage of the whole project.

6.1 Monthly Data Reading and Buffered Weather Reading

For each target month, the code first reads the cleaned flight data of that month. The weather data are not limited to the current month only. Instead, `read_weather_with_buffer` additionally reads:

- the previous month;

- the current month;
- the next month.

The reason for this buffer strategy is that for flights near the beginning or the end of a month, the required previous-hour weather observations may fall in adjacent months. If only the current month's weather is read, valid candidate observations at month boundaries would be lost.

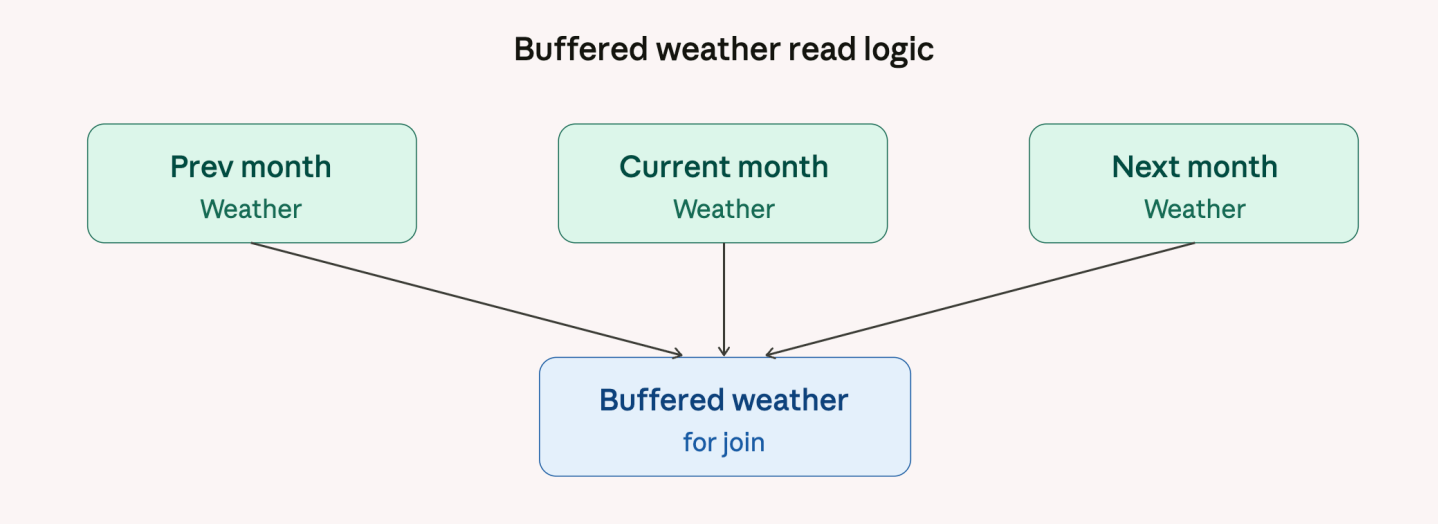


Figure 8. Logic of reading weather data with buffer months

6.2 Construction of Weather Matching Target Times

The code defines `lags = list(range(13))`, meaning that weather information from the current time back to 12 hours earlier is extracted.

For each flight, the function `make_targets` constructs multiple target timestamps based on a given airport field and a reference time field. For departure-side weather matching:

- the airport field is `ORIGIN_AIRPORT_ID`
- the reference time is `scheduled_dep_ts`

For arrival-side weather matching:

- the airport field is `DEST_AIRPORT_ID`
- the reference time is `scheduled_arr_ts_dest_local`

For each `lag_hour`, the code generates:

$$target_ts = base_ts - lag_hour \times 3600$$

Therefore, one flight is expanded into 13 weather-matching target records.

6.3 Construction of Departure-Side and Arrival-Side Weather Long Tables

The core weather-matching logic is encapsulated in `build_weather_long`. Its process is as follows:

1. Call `make_targets` to generate the `flight_id × lag_hour` target table;

2. Join the target table and the weather table by `airport_id` ;
3. Compute the absolute time difference `time_diff_sec` between the weather observation time `obs_ts` and the target time `target_ts` ;
4. Keep only candidate observations whose time difference is no more than one hour;
5. For multiple candidate records under the same `flight_id + lag_hour` , sort them by ascending time difference;
6. Use a window function to retain only the single observation closest to the target time.

This logic is applied separately to:

- the departure-side weather long table `wo_long`
- the arrival-side weather long table `wd_long`

This is one of the most important algorithmic components of the project because it performs scalable data-level matching between flight target times and weather observations.

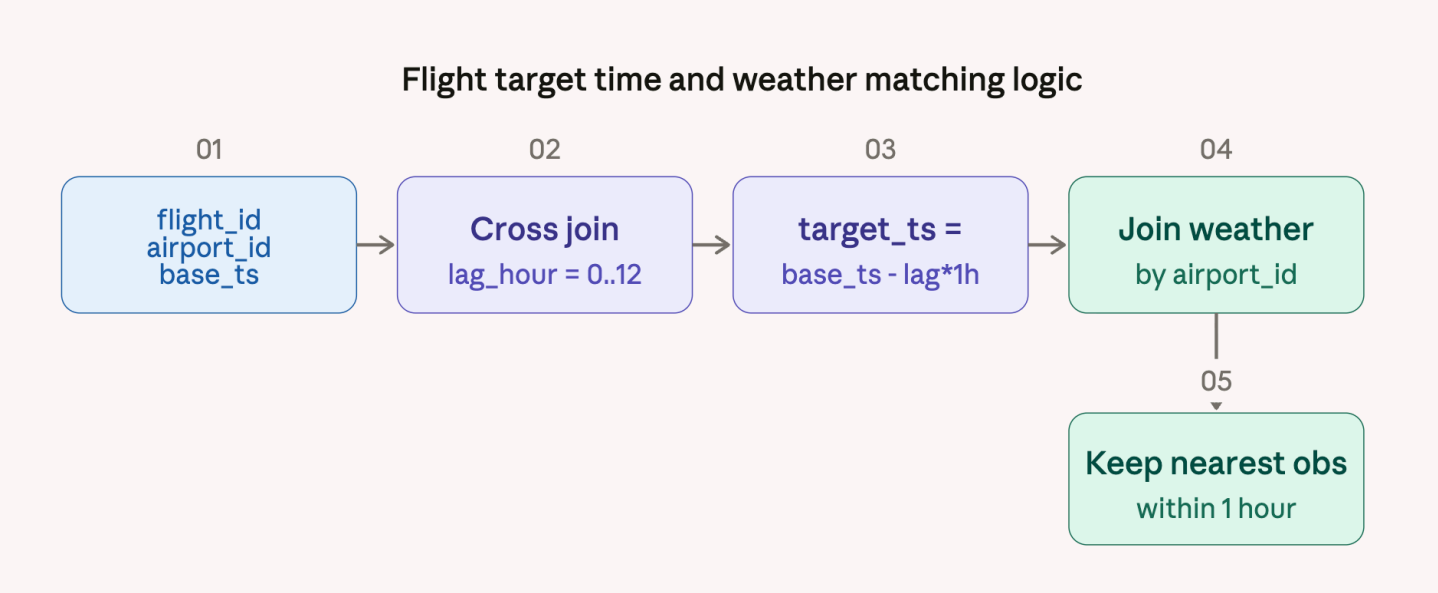


Figure 9. Matching logic between flight target times and weather observations

6.4 Long-to-Wide Transformation

After obtaining the departure-side and arrival-side weather long tables, the code uses `long_to_wide` to transform them into wide tables. The logic is:

- group by `flight_id` ;
- for each `lag_hour` and each weather variable, extract the corresponding value;
- generate column names such as:
  - `wo0_xxx` , `wo1_xxx` , ..., `wo12_xxx`
  - `wd0_xxx` , `wd1_xxx` , ..., `wd12_xxx`

After the long-to-wide transformation, each flight remains one row only, but that row now contains weather features for multiple lag hours on both the departure side and the arrival side, making the dataset more suitable for downstream modeling or analysis.

6.5 Final JT Generation and Output

Finally, the code performs left joins on `flight_id` among the following three parts:

- the monthly flight master table `flights_m`
- the departure-side weather wide table `wo_wide`
- the arrival-side weather wide table `wd_wide`

This produces the final monthly `jt_month`, which is written to:

`/content/drive/MyDrive/flight_delay_project/jt/year=.../month=...`

thus completing the JT construction for the selected month.

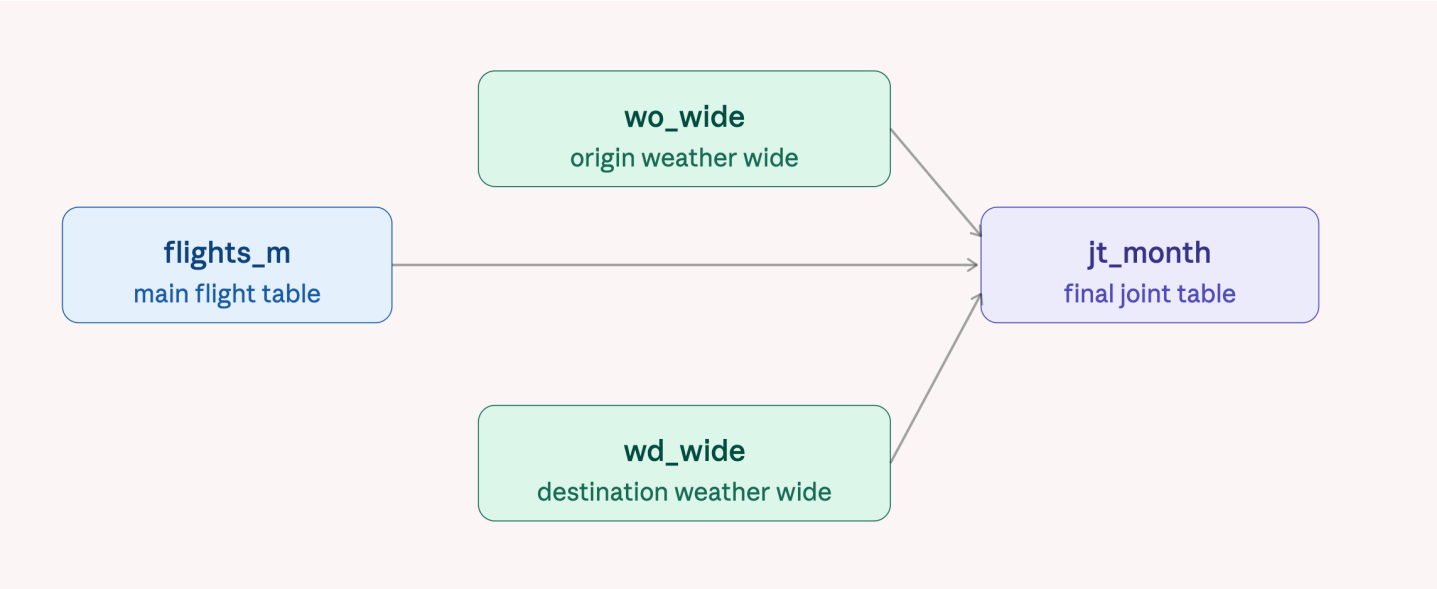


Figure 10. Structure of the final monthly joint table

7. Core Functions and Algorithm Description

To stay closer to the code implementation, this section briefly explains several core functions in the joint stage and summarizes the logic in a more algorithm-oriented style.

7.1 `read_weather_with_buffer`

This function reads the weather data of the current month together with the previous and next month, and merges them into a buffered weather table. Its role is to handle cross-month historical weather matching for flights near month boundaries. Without this strategy, the 0–12 hour weather observations of boundary flights would be incomplete.

7.2 `make_targets`

This function expands each flight record into multiple target records across `lag_hour` values according to a specified airport field and time field. The output includes at least:

- `flight_id`

- `airport_id`
- `base_ts`
- `lag_hour`
- `target_ts`

Essentially, it transforms each flight row into a weather-query target table.

7.3 `build_weather_long`

This function is the core of weather matching. It does not simply join records by hourly slots. Instead, it:

- first joins flight targets with weather candidates by airport;
- then evaluates how close each candidate weather record is to the target time through `time_diff_sec` ;
- finally retains the single weather observation closest to the target time for each `flight_id + lag_hour` .

So in essence, this function implements a nearest-neighbor-style temporal matching.

7.4 `long_to_wide`

This function reshapes the weather long table at the `flight_id + lag_hour` level into a wide table at the `flight_id` level. In essence, it is a pivot operation that unfolds lag values into columns and is a key step in constructing the final modeling table.

7.5 Summary of the Joint-Stage Algorithm

Overall, the joint stage can be summarized as follows:

Algorithm 1. Monthly JT Construction

Input: target month `m` , cleaned flight dataset `F` , cleaned weather dataset `W` , lag set `L = {0,1,...,12}`
Output: monthly joint table `JT_m`

1. `flights_m ← read_flights(F, m)`
2. `weather_buf ← read_weather_with_buffer(W, m)`
3. `dep_targets ← make_targets(flights_m, ORIGIN_AIRPORT_ID, scheduled_dep_ts, L)`
4. `arr_targets ← make_targets(flights_m, DEST_AIRPORT_ID, scheduled_arr_ts_dest_local, L)`
5. `wo_long ← build_weather_long(dep_targets, weather_buf)`
6. `wd_long ← build_weather_long(arr_targets, weather_buf)`
7. `wo_wide ← long_to_wide(wo_long)`
8. `wd_wide ← long_to_wide(wd_long)`
9. `JT_m ← flights_m ⟕ wo_wide ⟕ wd_wide` by `flight_id`
10. `write_parquet(JT_m)`
11. `return JT_m`

This algorithmic workflow corresponds directly to the actual code structure and also reflects the project's implementation logic of "clean first, match second, reshape last."

8. Main Difficulties Encountered and Solutions

During implementation, the project encountered several typical difficulties.

8.1 Inconsistent Airport and Weather-Station Identifiers

The flight data use `AirportID`, while the weather data use `WBAN`. These two identifiers cannot be joined directly. To solve this problem, the project introduces the mapping table `wban_airport_timezone.csv`, which first maps weather observations to airports and then allows later joining.

8.2 Inconsistent Time Field Formats

On the flight side, `FL_DATE + CRS_DEP_TIME`, and on the weather side, `Date + Time`, are not directly comparable standard timestamps. Therefore, the project constructs standardized timestamp fields separately for flights and weather, namely `scheduled_dep_ts`, `scheduled_arr_ts_dest_local`, and `obs_ts`, enabling later temporal matching.

8.3 Cross-Time-Zone Issue on the Arrival Side

If the original arrival-related time is used without accounting for the time-zone difference, systematic bias will arise in arrival-side weather matching. To address this, the project uses the time-zone field in the mapping table to convert the scheduled arrival time into destination-local time.

8.4 Cross-Month Weather Matching Near Month Boundaries

If only the current month's weather data are read, some historical weather candidates for flights near the beginning or end of the month will be missing. Therefore, the project adopts a previous/current/next month buffer strategy to ensure that boundary flights can also obtain complete candidate weather observations.

8.5 Multiple Candidate Weather Observations for One Target Time

Because weather records are not necessarily recorded exactly once per hour, multiple candidate observations may exist near a given target time. To handle this, the project uses a window function to sort by `time_diff_sec` and keep only the observation closest to the target time.

9. Experimental Results and Scalability Analysis

9.1 Current Running Scope

At the JT construction stage, because weather data are currently available only for the following months, the code currently merges only these months into the final combined table:

- 2012-01
- 2012-05
- 2012-07
- 2012-09

- 2013-04
- 2013-06
- 2013-09
- 2013-11

9.2 Runtime Characteristics by Stage

From actual execution, simple data reading is not the main bottleneck of the whole pipeline. Reading one month of flight data and buffered weather data does take some time, but it is still relatively fast compared with the later stages. The truly high-cost stages mainly include:

- joining target tables with weather candidates by airport;
- constructing the departure-side and arrival-side weather long tables;
- transforming weather long tables into wide tables;
- joining the wide tables with the monthly flight master table and writing the final Parquet output.

This shows that the main computational burden of the project lies not in I/O itself, but in large-scale data reshaping, window-based selection, column expansion, and materialization to storage.

9.3 Runtime Analysis

Table 1. Runtime of each stage in the JT construction process (representative results)

Month	Read flights	Read buffered weather	Build `wo_long`	Build `wd_long`	`long_to_wide`	Final join and write	Total
2013-04	0.07 h	0.14 h	0.96 h	1.04 h	0.73 h	2.92 h	5.86 h
2013-06	0.09 h	0.18 h	1.01 h	1.11 h	0.81 h	3.08 h	6.28 h
2013-09	0.06 h	0.16 h	0.91 h	0.98 h	0.69 h	2.87 h	5.67 h
2013-11	0.08 h	0.13 h	1.05 h	1.09 h	0.77 h	3.01 h	6.13 h

Table 1 reports the runtime breakdown of JT construction for April, June, September, and November 2013. It can be seen that reading the flight data and buffered weather data takes relatively little time and usually accounts for only a small proportion of the total runtime. The main computational cost is concentrated in weather matching, long-to-wide transformation, and final result writing. In particular, the final join and Parquet output is typically the most time-consuming single stage, close to 3 hours.

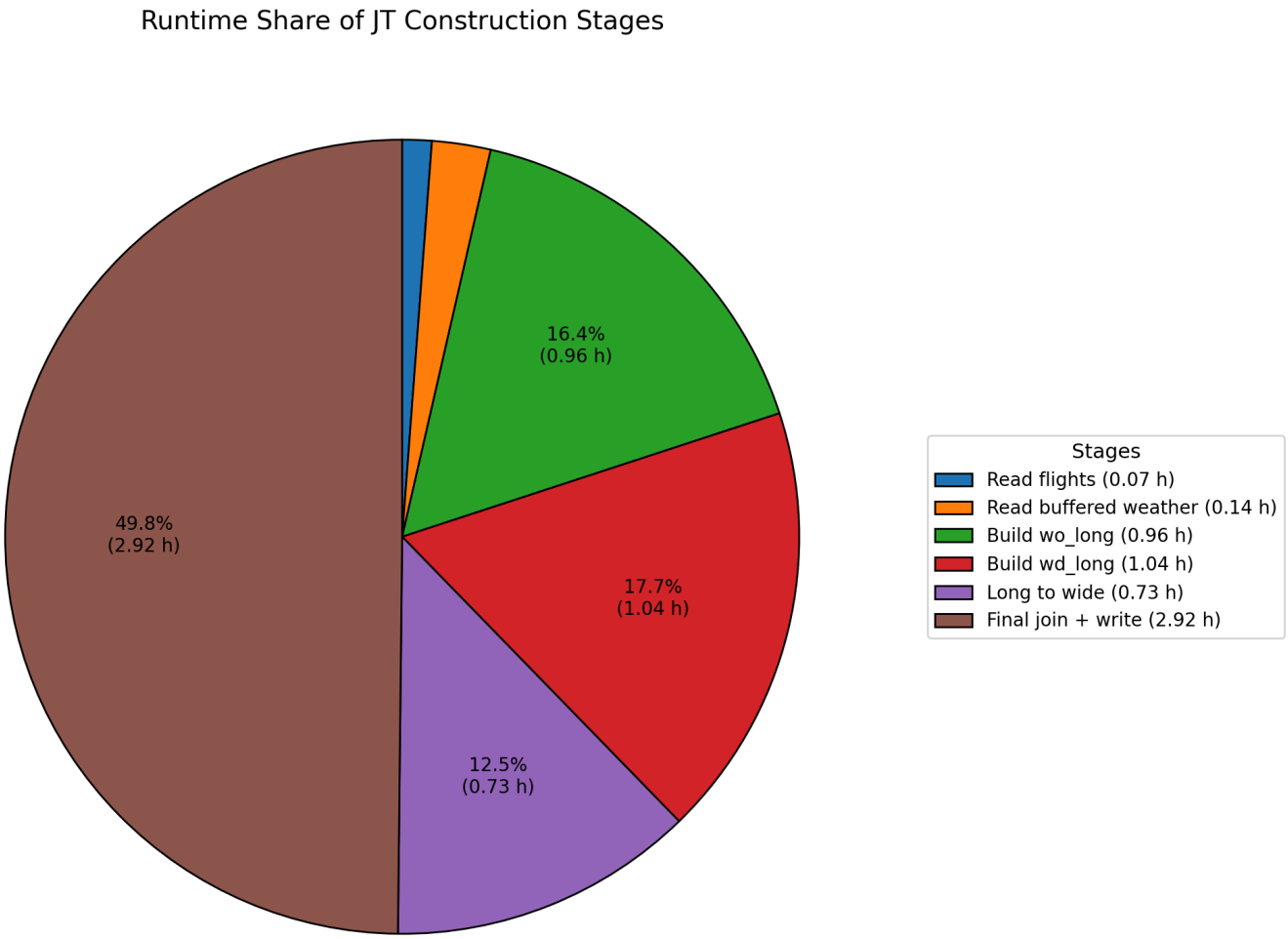


Figure 11. Runtime share of each stage in the joint-table construction process

9.4 Scalability Discussion

From an engineering perspective, the project is implemented with Spark DataFrames and is therefore much more suitable than single-machine Pandas for processing large-scale monthly flight and weather data. However, as the number of months increases, the number of weather candidates grows, and the number of wide-table columns expands, the following issues become more apparent:

- shuffle costs increase in the join stage;
- the `flight_id × lag_hour` target expansion inflates intermediate table size;
- the cost of selecting the nearest candidate record through window functions rises;
- after long-to-wide transformation, the number of columns increases substantially, which further raises writing costs.

This trend is also visible from actual runtime results: the reading stage usually accounts for only a small share of the total runtime, while `wo_long`, `wd_long`, `long_to_wide`, and the final writing stage together consume the overwhelming majority of time. This indicates that the system bottleneck comes mainly from data rearrangement, matching, and materialization, rather than simple loading. Therefore, although the current implementation already provides a relatively complete scalable processing pipeline, further optimization would still be needed if the workflow were to be extended to more years or more lag features. Possible directions

include finer-grained partitioning, caching hot intermediate tables, reducing invalid candidate weather records, or applying stricter pre-filtering before joins.

10. Supplementary Baseline Experiment: Verifying the Usability of JT for Prediction Tasks

Although the main focus of this project is not advanced modeling, a simple machine learning baseline experiment is still conducted in `JOIN_TOTAL_ML.ipynb` to verify the usability of the constructed JT.

It should be emphasized that the purpose of this part is not to fully reproduce the downstream modeling pipeline in the original paper, but rather to perform a lightweight downstream validation: namely, whether the JT produced by the previous data preparation pipeline is already in a form that can be directly used by a prediction model. Therefore, this experiment should be regarded as a **data preparation effectiveness check**, rather than a complete model-performance comparison.

The experimental procedure is as follows:

1. Read and merge multiple JT months;
2. Extract basic temporal features from `scheduled_dep_ts` ;
3. Use `is_delayed_15` as the binary label;
4. Use selected flight features and weather numeric features as inputs;
5. Use 2012 data as the training set and 2013 data as the test set;
6. Apply simple down-sampling on the training set to alleviate class imbalance;
7. Train a Random Forest classifier with PySpark MLlib;
8. Evaluate AUC, AUPR, Accuracy, F1, and other metrics.

The label used in this experiment is **arrival delay of at least 15 minutes**, that is, `is_delayed_15 = 1` indicates an arrival delay of no less than 15 minutes, while `is_delayed_15 = 0` indicates otherwise. The input features mainly include three categories: first, basic temporal variables extracted from the scheduled departure time; second, basic flight attributes; third, selected weather numeric features organized in the JT.

The experimental results are shown below.

Table 2. Brief comparison between the paper and the baseline experiment in this report

Item	Paper	This experiment
Task threshold	15 min delay classification	15 min delay classification
Model	Parallel Random Forest (MapReduce)	PySpark MLlib Random Forest
Accuracy	74.2%	67.28%
Delay Recall	71.8%	—
AUC	—	0.6669
AUPR	—	0.3448
F1	—	0.7003

The results show that the baseline model can already complete a standard binary classification training-and-evaluation workflow on the constructed JT, which indicates that the joint table built in this project is structurally usable and can directly support downstream prediction tasks. However, compared with the original paper, the current experimental results are still clearly weaker than the full solution reported there. In the original paper, under the **15-minute delay threshold**, the reported performance is **Accuracy = 74.2%** and **Delay Recall = 71.8%**, while under the **60-minute delay threshold**, it further reaches **Accuracy = 85.8%** and **Delay Recall = 86.9%**. By contrast, the supplementary experiment in this report achieves **Accuracy = 67.28%**, **AUC = 0.6669**, **AUPR = 0.3448**, and **F1 = 0.7003**, which is still at a relatively basic level overall. This gap is not difficult to explain: the original paper uses a more complete parallel Random Forest solution designed for the full prediction task, with more systematic treatment of target data selection, class balancing, feature selection, weather-observation combinations, and model calibration; in contrast, the experiment here is only a lightweight usability check after the data preparation pipeline has been completed, with a more limited month range, feature set, sampling strategy, and degree of model tuning. Therefore, the main value of this supplementary experiment is not to reproduce the best prediction performance in the paper, but to demonstrate that, even before full feature engineering and model optimization, the JT constructed in this work is already capable of supporting a standard delay-classification task and thus provides the data foundation for later work that may move closer to the paper's reported results.

11. Characteristics of the Code Implementation

From the perspective of implementation, the project has several notable characteristics.

11.1 Entire Pipeline Built on PySpark DataFrames

Whether in flight cleaning, weather cleaning, or later joint table construction, the whole workflow is implemented with the Spark DataFrame API, which keeps the style consistent and is suitable for organizing logic in a big-data environment.

11.2 Intermediate Results Written Stage by Stage

The code does not generate the JT directly from raw CSV files in a single step. Instead, it first creates:

- `clean/Flights`
- `clean/Weather`

and then uses these intermediate results as inputs to construct the JT. This staged design makes debugging, validation, and repeated execution easier.

11.3 Use of Parquet and Monthly Partitioning

The cleaned flight data, cleaned weather data, and the JT are all stored in Parquet format and partitioned by year and month. This reduces later reading cost and aligns well with common engineering practice in Spark-based large-scale data processing.

11.4 Joint Stage Runs by Selected Months

The current notebook does not process all months in a single run. Instead, it selects a subset of months for execution and validation. This design helps control experimental cost and makes it possible to verify the joint logic incrementally.

12. Strengths and Limitations of the Project

12.1 Strengths

This project has several strengths.

First, the whole workflow is complete and achieves end-to-end processing from heterogeneous raw CSV files to a structured joint table, rather than remaining only at the conceptual level.
Second, the code structure is relatively clear, with flight cleaning, weather cleaning, and joint construction implemented in separate modules, which makes debugging and maintenance easier.
Third, the temporal alignment logic and the cross-time-zone handling are explicit, so both departure-side and arrival-side weather matching are interpretable.
Finally, the output can directly support downstream machine learning tasks and is therefore highly reusable.

12.2 Limitations

At the same time, the project also has some limitations.

First, the weather matching strategy uses the “nearest observation within a one-hour window” rule. Although this is straightforward and practical, more advanced interpolation or aggregation methods are not explored.
Second, the current joint stage is demonstrated only on selected months and has not yet been extended to a more complete large-scale run over a longer time span.
Third, the long-to-wide transformation and the final writing stage are still computationally expensive, which indicates room for further optimization.
Fourth, the modeling part is currently limited to a basic validation experiment and does not yet systematically compare different prediction models.

13. Future Work

Based on the current implementation, several directions remain open for extension.

First, the missingness of matched weather observations can be analyzed more systematically, for example by identifying which airports, months, or lag values are more likely to lack candidate observations.
Second, more complete prediction models can be trained on the JT, and the effects of different feature combinations and model choices can be compared.
Third, the performance of the joint stage can be further optimized through better partitioning, caching strategies, or pre-filtering mechanisms.
Finally, more contextual variables could be incorporated, such as airport congestion indicators, holiday information, or richer aggregated weather features.

14. Conclusion

This project implements a PySpark-based data preparation pipeline for flight data and weather data, focusing on key issues such as raw-data cleaning, timestamp construction, airport–weather-station mapping, historical weather matching, and joint-table generation.

From the implementation results, the project has already established a relatively complete processing chain: flight data and weather data are first cleaned and written by month, then selected months are read to construct multi-hour weather features for both the departure and arrival sides, and finally a monthly JT is generated and stored. The overall logic is clear, and each key step has been concretely implemented in runnable code.

The supplementary baseline experiment further shows that the JT produced by this pipeline is not only suitable for data analysis, but can also be directly used as input for prediction tasks. Therefore, the most important contribution of the current project is not a complex prediction model, but the completion of a genuinely runnable and reusable data preparation pipeline, which provides a solid foundation for subsequent flight delay analysis and predictive modeling.

Appendix

File Name	Purpose
`01. project_clean.ipynb`	Cleans raw flight and weather data, and outputs `clean/Flights` and `clean/Weather`
`02. project_join_test.ipynb`	Reads the cleaned data and constructs JT for selected months
`03. JOIN_TOTAL_ML.ipynb`	Performs a simple delay-classification baseline experiment on the JT
`04. figures.ipynb`	Generates the flowcharts, structural diagrams, and schematic figures in the report

In [1]:
```
!pip install -q pyspark
```

In [2]:
```
from pyspark.sql import SparkSession, functions as F, types as T

spark = (
    SparkSession.builder
    .appName("flight_cleaning")
    .config("spark.sql.session.timeZone", "UTC")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")
```

In [3]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, c
all drive.mount("/content/drive", force_remount=True).
```

In [ ]:
```
WBAN_PATH = "/content/drive/MyDrive/flight_delay_project/raw/wban_airport

wban_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(WBAN_PATH)
)

wban_lookup = (
    wban_raw
    .select(
        F.trim(F.col("AirportID")).alias("AirportID"),
        F.lpad(F.trim(F.col("WBAN")), 5, "0").alias("WBAN"),
        F.trim(F.col("TimeZone")).alias("TimeZone")
    )
    .dropna(subset=["AirportID", "WBAN", "TimeZone"])
)

# For weather: WBAN -> AirportID
wban_to_airport_df = wban_lookup.select("WBAN", "AirportID", "TimeZone").

# For flight: AirportID -> TimeZone
airport_tz_df = (
    wban_lookup
    .select(
        F.col("AirportID").alias("airport_id"),
        F.col("TimeZone").cast("int").alias("timezone_offset")
    )
    .dropDuplicates(["airport_id"])
)
```

Flight

In [5]:
```
FLIGHT_RAW_PATH = "/content/drive/MyDrive/flight_delay_project/raw/Flight
FLIGHT_CLEAN_PATH = "/content/drive/MyDrive/flight_delay_project/clean/Fl
```

In [6]:

```
flights_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(f"{FLIGHT_RAW_PATH}/*")
)

print("Raw row count:", flights_raw.count())
print("Raw columns:", flights_raw.columns)
flights_raw.printSchema()
flights_raw.show(5, truncate=False)
```

```
Raw row count: 18286055
Raw columns: ['FL_DATE', 'OP_CARRIER_AIRLINE_ID', 'OP_CARRIER_FL_NUM', 'OR
IGIN_AIRPORT_ID', 'DEST_AIRPORT_ID', 'CRS_DEP_TIME', 'ARR_DELAY_NEW', 'CAN
CELLED', 'DIVERTED', 'CRS_ELAPSED_TIME', 'WEATHER_DELAY', 'NAS_DELAY', '_c
12']
root
 |-- FL_DATE: string (nullable = true)
 |-- OP_CARRIER_AIRLINE_ID: string (nullable = true)
 |-- OP_CARRIER_FL_NUM: string (nullable = true)
 |-- ORIGIN_AIRPORT_ID: string (nullable = true)
 |-- DEST_AIRPORT_ID: string (nullable = true)
 |-- CRS_DEP_TIME: string (nullable = true)
 |-- ARR_DELAY_NEW: string (nullable = true)
 |-- CANCELLED: string (nullable = true)
 |-- DIVERTED: string (nullable = true)
 |-- CRS_ELAPSED_TIME: string (nullable = true)
 |-- WEATHER_DELAY: string (nullable = true)
 |-- NAS_DELAY: string (nullable = true)
 |-- _c12: string (nullable = true)

+----------+-------------------+-----------------+-----------------+----
-----------+------------+-------------+---------+--------+---------------
+-------------+---------+----+
|FL_DATE   |OP_CARRIER_AIRLINE_ID|OP_CARRIER_FL_NUM|ORIGIN_AIRPORT_ID|DEST
_AIRPORT_ID|CRS_DEP_TIME|ARR_DELAY_NEW|CANCELLED|DIVERTED|CRS_ELAPSED_TIME
|WEATHER_DELAY|NAS_DELAY|_c12|
+----------+-------------------+-----------------+-----------------+----
-----------+------------+-------------+---------+--------+---------------
+-------------+---------+----+
|2013-07-01|20363                |3407             |11433            |1334
2           |1040        |0.00         |0.00     |0.00    |79.00
|NULL         |NULL     |NULL|
|2013-07-01|20363                |3409             |11433            |1226
6           |1227        |0.00         |0.00     |0.00    |175.00
|NULL         |NULL     |NULL|
|2013-07-01|20363                |3848             |13485            |1143
3           |0900        |2.00         |0.00     |0.00    |77.00
|NULL         |NULL     |NULL|
|2013-07-01|20363                |3851             |11433            |1106
6           |1535        |0.00         |0.00     |0.00    |60.00
|NULL         |NULL     |NULL|
|2013-07-01|20363                |3852             |10627            |1348
7           |0500        |0.00         |0.00     |0.00    |82.00
|NULL         |NULL     |NULL|
+----------+-------------------+-----------------+-----------------+----
-----------+------------+-------------+---------+--------+---------------
+-------------+---------+----+
only showing top 5 rows
```

In [7]:

```python
# drop useless columns
drop_cols = ["_c12"]
flights_1 = flights_raw.drop(*drop_cols)

# cast columns to correct types
flights_2 = (
    flights_1
    .withColumn("FL_DATE", F.to_date("FL_DATE", "yyyy-MM-dd"))
    .withColumn("OP_CARRIER_AIRLINE_ID", F.col("OP_CARRIER_AIRLINE_ID").c
    .withColumn("OP_CARRIER_FL_NUM", F.col("OP_CARRIER_FL_NUM").cast("str
    .withColumn("ORIGIN_AIRPORT_ID", F.col("ORIGIN_AIRPORT_ID").cast("str
```

```
    .withColumn("DEST_AIRPORT_ID", F.col("DEST_AIRPORT_ID").cast("string"
    .withColumn("CRS_DEP_TIME", F.col("CRS_DEP_TIME").cast("string"))
    .withColumn("ARR_DELAY_NEW", F.col("ARR_DELAY_NEW").cast("double"))
    .withColumn("CANCELLED", F.col("CANCELLED").cast("double"))
    .withColumn("DIVERTED", F.col("DIVERTED").cast("double"))
    .withColumn("CRS_ELAPSED_TIME", F.col("CRS_ELAPSED_TIME").cast("doubl
    .withColumn("WEATHER_DELAY", F.col("WEATHER_DELAY").cast("double"))
    .withColumn("NAS_DELAY", F.col("NAS_DELAY").cast("double"))
)

flights_2.printSchema()
# flights_2.show(5, truncate=False)
```

```
root
 |-- FL_DATE: date (nullable = true)
 |-- OP_CARRIER_AIRLINE_ID: string (nullable = true)
 |-- OP_CARRIER_FL_NUM: string (nullable = true)
 |-- ORIGIN_AIRPORT_ID: string (nullable = true)
 |-- DEST_AIRPORT_ID: string (nullable = true)
 |-- CRS_DEP_TIME: string (nullable = true)
 |-- ARR_DELAY_NEW: double (nullable = true)
 |-- CANCELLED: double (nullable = true)
 |-- DIVERTED: double (nullable = true)
 |-- CRS_ELAPSED_TIME: double (nullable = true)
 |-- WEATHER_DELAY: double (nullable = true)
 |-- NAS_DELAY: double (nullable = true)
```

Missing values

In [13]:

```
cols = flights_2.columns

string_cols = [
    f.name for f in flights_2.schema.fields
    if f.name in cols and isinstance(f.dataType, T.StringType)
]

non_string_cols = [c for c in cols if c not in string_cols]

null_check = flights_2.select(
    *[
        F.sum(
            F.when(F.col(c).isNull() | (F.trim(F.col(c)) == ""), 1).other
        ).alias(c)
        for c in string_cols
    ],
    *[
        F.sum(
            F.when(F.col(c).isNull(), 1).otherwise(0)
        ).alias(c)
        for c in non_string_cols
    ]
)

null_check.show(truncate=False)
```

```
+--------------------+----------------+----------------+--------------
+-----------+-------+------------+---------+--------+---------------+--
----------+---------+
|OP_CARRIER_AIRLINE_ID|OP_CARRIER_FL_NUM|ORIGIN_AIRPORT_ID|DEST_AIRPORT_ID
|CRS_DEP_TIME|FL_DATE|ARR_DELAY_NEW|CANCELLED|DIVERTED|CRS_ELAPSED_TIME|WE
ATHER_DELAY|NAS_DELAY|
+--------------------+----------------+----------------+--------------
+-----------+-------+------------+---------+--------+---------------+--
----------+---------+
|0                    |0               |0               |0
|1          |0      |342986       |0        |0       |12             |14
761092     |14761092 |
+--------------------+----------------+----------------+--------------
+-----------+-------+------------+---------+--------+---------------+--
----------+---------+
```

In [9]:
```python
flights_3 = (
    flights_2
    .filter(F.col("CANCELLED") == 0.0)
    .filter(F.col("DIVERTED") == 0.0)
    .filter(F.col("CRS_DEP_TIME").isNotNull())
    .filter(F.col("CRS_ELAPSED_TIME").isNotNull())
)

# print("Rows after filtering cancelled/diverted/null keys:", flights_3.c
# flights_3.show(5, truncate=False)
```

Handling `CRS_DEP_TIME`

In [10]:
```python
# Extract hour and minute from CRS_DEP_TIME
flights_4 = (
    flights_3
    .withColumn(
        "CRS_DEP_TIME_4",
        F.lpad(F.regexp_replace(F.col("CRS_DEP_TIME"), r"[^0-9]", ""), 4,
    )
    .withColumn("dep_hour", F.substring("CRS_DEP_TIME_4", 1, 2).cast("int
    .withColumn("dep_minute", F.substring("CRS_DEP_TIME_4", 3, 2).cast("i
)

# Filter out rows with invalid times (e.g., dep_hour not in 0-23 or dep_m
flights_4 = (
    flights_4
    .filter(F.col("dep_hour").between(0, 23))
    .filter(F.col("dep_minute").between(0, 59))
)

# flights_4.select(
#     "CRS_DEP_TIME", "CRS_DEP_TIME_4", "dep_hour", "dep_minute"
# ).show(10, truncate=False)
```

In [11]:
```python
# Combine FL_DATE with dep_hour and dep_minute to create a scheduled depa
flights_5 = (
    flights_4
    .withColumn(
        "scheduled_dep_str",
        F.concat_ws(
```

```
            " ",
            F.date_format("FL_DATE", "yyyy-MM-dd"),
            F.concat_ws(
                ":",
                F.lpad(F.col("dep_hour").cast("string"), 2, "0"),
                F.lpad(F.col("dep_minute").cast("string"), 2, "0"),
                F.lit("00")
            )
        )
    )
    .withColumn(
        "scheduled_dep_ts",
        F.to_timestamp("scheduled_dep_str", "yyyy-MM-dd HH:mm:ss")
    )
)

flights_5.select(
    "FL_DATE", "CRS_DEP_TIME", "CRS_DEP_TIME_4", "scheduled_dep_ts"
).show(10, truncate=False)
```

```
+----------+------------+--------------+-------------------+
|FL_DATE   |CRS_DEP_TIME|CRS_DEP_TIME_4|scheduled_dep_ts   |
+----------+------------+--------------+-------------------+
|2013-07-01|1040        |1040          |2013-07-01 10:40:00|
|2013-07-01|1227        |1227          |2013-07-01 12:27:00|
|2013-07-01|0900        |0900          |2013-07-01 09:00:00|
|2013-07-01|1535        |1535          |2013-07-01 15:35:00|
|2013-07-01|0500        |0500          |2013-07-01 05:00:00|
|2013-07-01|0800        |0800          |2013-07-01 08:00:00|
|2013-07-01|1720        |1720          |2013-07-01 17:20:00|
|2013-07-01|1541        |1541          |2013-07-01 15:41:00|
|2013-07-01|0830        |0830          |2013-07-01 08:30:00|
|2013-07-01|0600        |0600          |2013-07-01 06:00:00|
+----------+------------+--------------+-------------------+
only showing top 10 rows
```

Construct arriving time (considering timezone!)

In [12]:
```
# join origin timezone
origin_tz_df = airport_tz_df.select(
    F.col("airport_id").alias("ORIGIN_AIRPORT_ID"),
    F.col("timezone_offset").alias("origin_tz")
)

# join destination timezone
dest_tz_df = airport_tz_df.select(
    F.col("airport_id").alias("DEST_AIRPORT_ID"),
    F.col("timezone_offset").alias("dest_tz")
)
```

In [13]:
```
flights_6 = (
    flights_5
    .join(origin_tz_df, on="ORIGIN_AIRPORT_ID", how="left")
    .join(dest_tz_df, on="DEST_AIRPORT_ID", how="left")
)

# first compute arrival timestamp on the departure-local time axis
flights_6 = (
    flights_6
```

```
    .withColumn(
        "scheduled_arr_ts_origin_clock",
        F.from_unixtime(
            F.unix_timestamp("scheduled_dep_ts") + F.col("CRS_ELAPSED_TIM
        ).cast("timestamp")
    )
)

# then adjust to destination local time by timezone difference
flights_6 = (
    flights_6
    .withColumn(
        "tz_diff_hours",
        F.col("dest_tz") - F.col("origin_tz")
    )
    .withColumn(
        "scheduled_arr_ts_dest_local",
        F.from_unixtime(
            F.unix_timestamp("scheduled_arr_ts_origin_clock") + F.col("tz
        ).cast("timestamp")
    )
)

flights_6.select(
    "ORIGIN_AIRPORT_ID", "DEST_AIRPORT_ID",
    "scheduled_dep_ts",
    "CRS_ELAPSED_TIME",
    "origin_tz", "dest_tz", "tz_diff_hours",
    "scheduled_arr_ts_origin_clock",
    "scheduled_arr_ts_dest_local"
).show(10, truncate=False)
```

```
+-----------------+--------------+-------------------+----------------+--
-------+-------+-------------+---------------------------+-------------
-------------+
|ORIGIN_AIRPORT_ID|DEST_AIRPORT_ID|scheduled_dep_ts   |CRS_ELAPSED_TIME|or
igin_tz|dest_tz|tz_diff_hours|scheduled_arr_ts_origin_clock|scheduled_arr_
ts_dest_local|
+-----------------+--------------+-------------------+----------------+--
-------+-------+-------------+---------------------------+-------------
-------------+
|11433            |13342         |2013-07-01 10:40:00|79.0            |-5
|-6     |-1           |2013-07-01 11:59:00          |2013-07-01 10:59:00
|
|11433            |12266         |2013-07-01 12:27:00|175.0           |-5
|-6     |-1           |2013-07-01 15:22:00          |2013-07-01 14:22:00
|
|13485            |11433         |2013-07-01 09:00:00|77.0            |-6
|-5     |1            |2013-07-01 10:17:00          |2013-07-01 11:17:00
|
|11433            |11066         |2013-07-01 15:35:00|60.0            |-5
|-5     |0            |2013-07-01 16:35:00          |2013-07-01 16:35:00
|
|10627            |13487         |2013-07-01 05:00:00|82.0            |NU
LL     |-6     |NULL         |2013-07-01 06:22:00          |NULL
|
|13198            |11193         |2013-07-01 08:00:00|102.0           |-6
|-5     |1            |2013-07-01 09:42:00          |2013-07-01 10:42:00
|
|10792            |11433         |2013-07-01 17:20:00|78.0            |-5
|-5     |0            |2013-07-01 18:38:00          |2013-07-01 18:38:00
|
|11433            |10792         |2013-07-01 15:41:00|73.0            |-5
|-5     |0            |2013-07-01 16:54:00          |2013-07-01 16:54:00
|
|10397            |12264         |2013-07-01 08:30:00|114.0           |-5
|-5     |0            |2013-07-01 10:24:00          |2013-07-01 10:24:00
|
|12264            |10397         |2013-07-01 06:00:00|106.0           |-5
|-5     |0            |2013-07-01 07:46:00          |2013-07-01 07:46:00
|
+-----------------+--------------+-------------------+----------------+--
-------+-------+-------------+---------------------------+-------------
-------------+
only showing top 10 rows
```

In [ ]:
```
# Count how many rows have null tz_diff_hours
# (About 5% of rows have null tz_diff_hours, which means we couldn't find
flights_6.select(
    F.sum(F.when(F.col("tz_diff_hours").isNull(), 1).otherwise(0)).alias(
).show()
```

```
+------------------+
|null_tz_diff_count|
+------------------+
|            951499|
+------------------+
```

In [16]:
```
# Drop the lines that have null tz_diff_hours, since we won't be able to
flights_6 = flights_6.filter(F.col("tz_diff_hours").isNotNull())
```

Add some new columns (inlucding binary flags)

In [17]:
```
flights_7 = (
    flights_6
    .withColumn("flight_year", F.year("FL_DATE"))
    .withColumn("flight_month", F.month("FL_DATE"))
    .withColumn("flight_yyyymm", F.date_format("FL_DATE", "yyyyMM"))
    .withColumn("arr_delay_min", F.coalesce(F.col("ARR_DELAY_NEW"), F.lit
    .withColumn("is_delayed_15", F.when(F.col("arr_delay_min") >= 15, 1).
    .withColumn("is_delayed_60", F.when(F.col("arr_delay_min") >= 60, 1).
    .withColumn("has_weather_delay", F.when(F.coalesce(F.col("WEATHER_DEL
    .withColumn("has_nas_delay", F.when(F.coalesce(F.col("NAS_DELAY"), F.
)

# generate primary key for identifying unique flights
flights_7 = flights_7.withColumn(
    "flight_id",
    F.sha2(F.concat_ws(
        "_",
        F.col("FL_DATE").cast("string"),
        F.col("OP_CARRIER_AIRLINE_ID"),
        F.col("OP_CARRIER_FL_NUM"),
        F.col("ORIGIN_AIRPORT_ID"),
        F.col("DEST_AIRPORT_ID"),
        F.col("CRS_DEP_TIME").cast("string")
    ), 256)
)
```

Export to parquet

In [19]:
```
final_cols = [
    "flight_id",
    "FL_DATE",
    "flight_year",
    "flight_month",
    "flight_yyyymm",
    "OP_CARRIER_AIRLINE_ID",
    "OP_CARRIER_FL_NUM",
    "ORIGIN_AIRPORT_ID",
    "DEST_AIRPORT_ID",
    "CRS_DEP_TIME",
    "dep_hour",
    "dep_minute",
    "CRS_ELAPSED_TIME",
    "scheduled_dep_ts",
    "origin_tz",
    "dest_tz",
    "tz_diff_hours",
    "scheduled_arr_ts_origin_clock",
    "scheduled_arr_ts_dest_local",
    "ARR_DELAY_NEW",
    "arr_delay_min",
    "WEATHER_DELAY",
    "NAS_DELAY",
    "has_weather_delay",
    "has_nas_delay",
    "is_delayed_15",
```

```
    "is_delayed_60"
]

flights_clean = flights_7.select(*final_cols)

flights_clean.printSchema()
# flights_clean.show(10, truncate=False)
```

```
root
 |-- flight_id: string (nullable = true)
 |-- FL_DATE: date (nullable = true)
 |-- flight_year: integer (nullable = true)
 |-- flight_month: integer (nullable = true)
 |-- flight_yyyymm: string (nullable = true)
 |-- OP_CARRIER_AIRLINE_ID: string (nullable = true)
 |-- OP_CARRIER_FL_NUM: string (nullable = true)
 |-- ORIGIN_AIRPORT_ID: string (nullable = true)
 |-- DEST_AIRPORT_ID: string (nullable = true)
 |-- CRS_DEP_TIME: string (nullable = true)
 |-- dep_hour: integer (nullable = true)
 |-- dep_minute: integer (nullable = true)
 |-- CRS_ELAPSED_TIME: double (nullable = true)
 |-- scheduled_dep_ts: timestamp (nullable = false)
 |-- origin_tz: integer (nullable = true)
 |-- dest_tz: integer (nullable = true)
 |-- tz_diff_hours: integer (nullable = true)
 |-- scheduled_arr_ts_origin_clock: timestamp (nullable = true)
 |-- scheduled_arr_ts_dest_local: timestamp (nullable = true)
 |-- ARR_DELAY_NEW: double (nullable = true)
 |-- arr_delay_min: double (nullable = false)
 |-- WEATHER_DELAY: double (nullable = true)
 |-- NAS_DELAY: double (nullable = true)
 |-- has_weather_delay: integer (nullable = false)
 |-- has_nas_delay: integer (nullable = false)
 |-- is_delayed_15: integer (nullable = false)
 |-- is_delayed_60: integer (nullable = false)
```

In [20]:
```
# export cleaned data as Parquet files partitioned by year and month
(
    flights_clean
    .write
    .mode("overwrite")
    .partitionBy("flight_year", "flight_month")
    .parquet(FLIGHT_CLEAN_PATH)
)
```

In [21]:
```
# Check the output files
df_test = spark.read.parquet(
    f"{FLIGHT_CLEAN_PATH}/flight_year=2012/flight_month=1"
)

print(df_test.count())
df_test.show(5, truncate=False)
```

```
453790
+----------------------------------------------------------------+--------
--+------------+--------------------+-----------------+----------------
+--------------+------------+--------+----------+---------------+-------
-----------+---------+-------+-------------+----------------------------
+--------------------------+------------+------------+-------------+---
------+-----------------+------------+------------+-------------+
|flight_id                                                       |FL_DATE
|flight_yyyymm|OP_CARRIER_AIRLINE_ID|OP_CARRIER_FL_NUM|ORIGIN_AIRPORT_ID|D
EST_AIRPORT_ID|CRS_DEP_TIME|dep_hour|dep_minute|CRS_ELAPSED_TIME|scheduled
_dep_ts   |origin_tz|dest_tz|tz_diff_hours|scheduled_arr_ts_origin_clock|s
cheduled_arr_ts_dest_local|ARR_DELAY_NEW|arr_delay_min|WEATHER_DELAY|NAS_D
ELAY|has_weather_delay|has_nas_delay|is_delayed_15|is_delayed_60|
+----------------------------------------------------------------+--------
--+------------+--------------------+-----------------+----------------
+--------------+------------+--------+----------+---------------+-------
-----------+---------+-------+-------------+----------------------------
+--------------------------+------------+------------+-------------+---
------+-----------------+------------+------------+-------------+
|fa5539793aec6aacd82060676ba3e1b63b98da16e8a53f3921348b657967f0ab|2012-01-
01|201201       |20366                |4426             |15370
|12266          |0845        |8       |45        |99.0            |2012-01
-01 08:45:00|-6       |-6      |0            |2012-01-01 10:24:00
|2012-01-01 10:24:00        |0.0          |0.0          |NULL         |NUL
L     |0                |0            |0            |0            |
|efd1e2975c3bfffb1ff67e9b99cc07c3bd44473d87bdce7f824e7ac4ef3670e|2012-01-
10|201201       |20304                |4698             |11298
|13487          |1400        |14      |0         |149.0           |2012-01
-10 14:00:00|-6       |-6      |0            |2012-01-10 16:29:00
|2012-01-10 16:29:00        |0.0          |0.0          |NULL         |NUL
L     |0                |0            |0            |0            |
|dd21e54167a1dd2f7688abeb80b07850d771d69c9d299ec290df8e99be80f708|2012-01-
01|201201       |20366                |4427             |12266
|15370          |0858        |8       |58        |88.0            |2012-01
-01 08:58:00|-6       |-6      |0            |2012-01-01 10:26:00
|2012-01-01 10:26:00        |0.0          |0.0          |NULL         |NUL
L     |0                |0            |0            |0            |
|b7854132432c0efb151a71423bda31ef93f00af415caf2e0e581d2c0ba0e04eb|2012-01-
10|201201       |20304                |4699             |13487
|14457          |1120        |11      |20        |108.0           |2012-01
-10 11:20:00|-6       |-7      |-1           |2012-01-10 13:08:00
|2012-01-10 12:08:00        |0.0          |0.0          |NULL         |NUL
L     |0                |0            |0            |0            |
|de8b8aef3767e53a454103ebcd7900d21e1d0a4ff956f24a59dcbebd26d3cb8b|2012-01-
01|201201       |20366                |4427             |15370
|12266          |1051        |10      |51        |89.0            |2012-01
-01 10:51:00|-6       |-6      |0            |2012-01-01 12:20:00
|2012-01-01 12:20:00        |0.0          |0.0          |NULL         |NUL
L     |0                |0            |0            |0            |
+----------------------------------------------------------------+--------
--+------------+--------------------+-----------------+----------------
+--------------+------------+--------+----------+---------------+-------
-----------+---------+-------+-------------+----------------------------
+--------------------------+------------+------------+-------------+---
------+-----------------+------------+------------+-------------+
only showing top 5 rows
```

Weather

In [26]:
```
WEATHER_RAW_PATH = "/content/drive/MyDrive/flight_delay_project/raw/Weath
WEATHER_CLEAN_PATH = "/content/drive/MyDrive/flight_delay_project/clean/W
```

In [27]:
```
weather_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(f"{WEATHER_RAW_PATH}/*")
)

print(len(weather_raw.columns))
print(weather_raw.columns)
weather_raw.printSchema()
weather_raw.show(5, truncate=False)
```

```
44
['WBAN', 'Date', 'Time', 'StationType', 'SkyCondition', 'SkyConditionFla
g', 'Visibility', 'VisibilityFlag', 'WeatherType', 'WeatherTypeFlag', 'Dry
BulbFarenheit', 'DryBulbFarenheitFlag', 'DryBulbCelsius', 'DryBulbCelsiusF
lag', 'WetBulbFarenheit', 'WetBulbFarenheitFlag', 'WetBulbCelsius', 'WetBu
lbCelsiusFlag', 'DewPointFarenheit', 'DewPointFarenheitFlag', 'DewPointCel
sius', 'DewPointCelsiusFlag', 'RelativeHumidity', 'RelativeHumidityFlag',
'WindSpeed', 'WindSpeedFlag', 'WindDirection', 'WindDirectionFlag', 'Value
ForWindCharacter', 'ValueForWindCharacterFlag', 'StationPressure', 'Statio
nPressureFlag', 'PressureTendency', 'PressureTendencyFlag', 'PressureChang
e', 'PressureChangeFlag', 'SeaLevelPressure', 'SeaLevelPressureFlag', 'Rec
ordType', 'RecordTypeFlag', 'HourlyPrecip', 'HourlyPrecipFlag', 'Altimete
r', 'AltimeterFlag']
root
 |-- WBAN: string (nullable = true)
 |-- Date: string (nullable = true)
 |-- Time: string (nullable = true)
 |-- StationType: string (nullable = true)
 |-- SkyCondition: string (nullable = true)
 |-- SkyConditionFlag: string (nullable = true)
 |-- Visibility: string (nullable = true)
 |-- VisibilityFlag: string (nullable = true)
 |-- WeatherType: string (nullable = true)
 |-- WeatherTypeFlag: string (nullable = true)
 |-- DryBulbFarenheit: string (nullable = true)
 |-- DryBulbFarenheitFlag: string (nullable = true)
 |-- DryBulbCelsius: string (nullable = true)
 |-- DryBulbCelsiusFlag: string (nullable = true)
 |-- WetBulbFarenheit: string (nullable = true)
 |-- WetBulbFarenheitFlag: string (nullable = true)
 |-- WetBulbCelsius: string (nullable = true)
 |-- WetBulbCelsiusFlag: string (nullable = true)
 |-- DewPointFarenheit: string (nullable = true)
 |-- DewPointFarenheitFlag: string (nullable = true)
 |-- DewPointCelsius: string (nullable = true)
 |-- DewPointCelsiusFlag: string (nullable = true)
 |-- RelativeHumidity: string (nullable = true)
 |-- RelativeHumidityFlag: string (nullable = true)
 |-- WindSpeed: string (nullable = true)
 |-- WindSpeedFlag: string (nullable = true)
 |-- WindDirection: string (nullable = true)
 |-- WindDirectionFlag: string (nullable = true)
 |-- ValueForWindCharacter: string (nullable = true)
 |-- ValueForWindCharacterFlag: string (nullable = true)
 |-- StationPressure: string (nullable = true)
 |-- StationPressureFlag: string (nullable = true)
 |-- PressureTendency: string (nullable = true)
 |-- PressureTendencyFlag: string (nullable = true)
 |-- PressureChange: string (nullable = true)
 |-- PressureChangeFlag: string (nullable = true)
 |-- SeaLevelPressure: string (nullable = true)
 |-- SeaLevelPressureFlag: string (nullable = true)
 |-- RecordType: string (nullable = true)
 |-- RecordTypeFlag: string (nullable = true)
 |-- HourlyPrecip: string (nullable = true)
 |-- HourlyPrecipFlag: string (nullable = true)
 |-- Altimeter: string (nullable = true)
 |-- AltimeterFlag: string (nullable = true)

+-----+--------+----+-----------+------------+----------------+----------+
```

```
+-----+--------+----+-----------+-----------+---------------+----------+
-------------+-----------+---------------+---------------+-------------
------+-------------+-----------------+---------------+---------------
----+-------------+-----------------+---------------+---------------
----+--------------+-----------------+---------------+---------------
----+---------+-------------+-------------+-----------------+-----------
--------+------------------------+--------------+-------------------+---
------------+--------------------+-------------+-------------------+-----
-----------+--------------------+----------+-------------+-----------+--
-------------+---------+-------------+
|WBAN |Date    |Time|StationType|SkyCondition|SkyConditionFlag|Visibility|
VisibilityFlag|WeatherType|WeatherTypeFlag|DryBulbFarenheit|DryBulbFarenhe
itFlag|DryBulbCelsius|DryBulbCelsiusFlag|WetBulbFarenheit|WetBulbFarenheit
Flag|WetBulbCelsius|WetBulbCelsiusFlag|DewPointFarenheit|DewPointFarenheit
Flag|DewPointCelsius|DewPointCelsiusFlag|RelativeHumidity|RelativeHumidity
Flag|WindSpeed|WindSpeedFlag|WindDirection|WindDirectionFlag|ValueForWindC
haracter|ValueForWindCharacterFlag|StationPressure|StationPressureFlag|Pre
ssureTendency|PressureTendencyFlag|PressureChange|PressureChangeFlag|SeaLe
velPressure|SeaLevelPressureFlag|RecordType|RecordTypeFlag|HourlyPrecip|Ho
urlyPrecipFlag|Altimeter|AltimeterFlag|
+-----+--------+----+-----------+-----------+---------------+----------+
-------------+-----------+---------------+---------------+-------------
------+-------------+-----------------+---------------+---------------
----+-------------+-----------------+---------------+---------------
----+--------------+-----------------+---------------+---------------
----+---------+-------------+-------------+-----------------+-----------
--------+------------------------+--------------+-------------------+---
------------+--------------------+-------------+-------------------+-----
-----------+--------------------+----------+-------------+-----------+--
-------------+---------+-------------+
|03011|20120101|0015|0          |CLR         |                |10.00     |
|           |               |23             |                   |-5.0
|                  |15             |                      |-9.5          |
|-9                |                     |-23.0          |
| 24             |                    | 5       |             |120
|                  |                    |                       |21.70
|                    |                |                     |              |
|M               |                    |AA        |              |
|                |30.43    |             |
|03011|20120101|0035|0          |CLR         |                |10.00     |
|           |               |21             |                   |-6.0
|                  |14             |                      |-10.2         |
|-9                |                     |-23.0          |
| 26             |                    | 6       |             |130
|                  |                    |                       |21.70
|                    |                |                     |              |
|M               |                    |AA        |              |
|                |30.43    |             |
|03011|20120101|0055|0          |CLR         |                |10.00     |
|           |               |21             |                   |-6.0
|                  |13             |                      |-10.5         |
| -13            |                     |-25.0          |
| 21             |                    | 0       |             |000
|                  |                    |                       |21.71
|                    |                |                     |              |
|M               |                    |AA        |              |
|                |30.44    |             |
|03011|20120101|0115|0          |CLR         |                |10.00     |
|           |               |21             |                   |-6.0
|                  |14             |                      |-10.1         |
|-8                |                     |-22.0          |
```

```
| 27             |                     | 0       |             |000
|                 |                     |                         |21.71
|                   |                 |                       |              |
|M               |                     |AA        |              |
|                |30.44    |             |
|03011|20120101|0135|0          |CLR          |                |10.00     |
|           |                |21               |                     |-6.0
|                  |13              |                      |-10.4         |
| -11             |                     |-24.0          |
| 23             |                     | 0       |             |000
|                 |                     |                         |21.72
|                   |                 |                       |              |
|M               |                     |AA        |              |
|                |30.45    |             |
+-----+--------+----+----------+----------+---------------+---------+
-------------+----------+--------------+---------------+-------------
------+-------------+-----------------+---------------+---------------
----+-------------+-----------------+---------------+----------------
----+--------------+-----------------+---------------+----------------
----+---------+------------+------------+----------------+------------
--------+-----------------------+--------------+-------------------+---
------------+-------------------+------------+-----------------+-----
-----------+-------------------+----------+-------------+-----------+--
-------------+---------+------------+
only showing top 5 rows
```

Clean the strings

In [29]:
```
# trim whitespace from string columns
string_cols = [f.name for f in weather_raw.schema.fields if isinstance(f.

weather_1 = weather_raw.select(
    *[F.trim(F.col(c)).alias(c) if c in string_cols else F.col(c) for c i
)

# replace empty strings, "M", and all-whitespace with nulls in string col
def clean_str(colname):
    return (
        F.when(F.col(colname).isNull(), None)
         .when(F.trim(F.col(colname)) == "", None)
         .when(F.upper(F.trim(F.col(colname))) == "M", None)
         .otherwise(F.trim(F.col(colname)))
    )

weather_2 = weather_1.select(
    *[
        clean_str(c).alias(c) if c in string_cols else F.col(c)
        for c in weather_1.columns
    ]
)
```

In [30]:
```
weather_keep_cols = [
    "WBAN",
    "Date",
    "Time",
    "SkyCondition",
    "Visibility",
    "WeatherType",
    "DryBulbFarenheit",
```

```
    "WetBulbFarenheit",
    "DewPointFarenheit",
    "RelativeHumidity",
    "WindSpeed",
    "WindDirection",
    "StationPressure",
    "SeaLevelPressure",
    "HourlyPrecip",
    "Altimeter"
]

weather_3 = weather_2.select(*weather_keep_cols)
weather_3.show(5, truncate=False)
```

```
+-----+--------+----+-----------+---------+----------+---------------+
---------------+----------------+---------------+---------+-----------
-+--------------+---------------+-----------+---------+
|WBAN |Date    |Time|SkyCondition|Visibility|WeatherType|DryBulbFarenheit|
WetBulbFarenheit|DewPointFarenheit|RelativeHumidity|WindSpeed|WindDirectio
n|StationPressure|SeaLevelPressure|HourlyPrecip|Altimeter|
+-----+--------+----+-----------+---------+----------+---------------+
---------------+----------------+---------------+---------+-----------
-+--------------+---------------+-----------+---------+
|03011|20120101|0015|CLR        |10.00     |NULL      |23             |
15             |-9              |24             |5        |120
|21.70          |NULL            |NULL       |30.43    |
|03011|20120101|0035|CLR        |10.00     |NULL      |21             |
14             |-9              |26             |6        |130
|21.70          |NULL            |NULL       |30.43    |
|03011|20120101|0055|CLR        |10.00     |NULL      |21             |
13             |-13             |21             |0        |000
|21.71          |NULL            |NULL       |30.44    |
|03011|20120101|0115|CLR        |10.00     |NULL      |21             |
14             |-8              |27             |0        |000
|21.71          |NULL            |NULL       |30.44    |
|03011|20120101|0135|CLR        |10.00     |NULL      |21             |
13             |-11             |23             |0        |000
|21.72          |NULL            |NULL       |30.45    |
+-----+--------+----+-----------+---------+----------+---------------+
---------------+----------------+---------------+---------+-----------
-+--------------+---------------+-----------+---------+
only showing top 5 rows
```

In [31]:

```
# Deal with time columns

weather_4 = (
    weather_3
    .withColumn("Date_str", F.col("Date"))
    .withColumn("obs_date", F.to_date("Date", "yyyyMMdd"))
    .withColumn("Time_4", F.lpad(F.col("Time"), 4, "0"))
)

weather_4 = (
    weather_4
    .withColumn("obs_hour", F.substring("Time_4", 1, 2).cast("int"))
    .withColumn("obs_minute", F.substring("Time_4", 3, 2).cast("int"))
)

weather_4 = (
    weather_4
```

```
    .filter(F.col("obs_date").isNotNull())
    .filter(F.col("Time_4").isNotNull())
    .filter(F.col("obs_hour").between(0, 23))
    .filter(F.col("obs_minute").between(0, 59))
)

weather_5 = (
    weather_4
    .withColumn(
        "obs_ts_str",
        F.concat_ws(
            " ",
            F.date_format("obs_date", "yyyy-MM-dd"),
            F.concat_ws(
                ":",
                F.lpad(F.col("obs_hour").cast("string"), 2, "0"),
                F.lpad(F.col("obs_minute").cast("string"), 2, "0"),
                F.lit("00")
            )
        )
    )
    .withColumn("obs_ts", F.to_timestamp("obs_ts_str", "yyyy-MM-dd HH:mm:
)
```

In [ ]:

```
# Clean numeric columns by removing non-numeric characters and casting to
def clean_numeric(colname):
    cleaned = F.trim(F.col(colname))
    cleaned = F.regexp_replace(cleaned, r"[^0-9.\-]", "")

    return (
        F.when(F.col(colname).isNull(), None)
         .when(F.trim(F.col(colname)) == "", None)
         .when(F.upper(F.trim(F.col(colname))) == "M", None)
         .when(cleaned == "", None)
         .otherwise(cleaned.cast("double"))
    )

weather_6 = (
    weather_5
    .withColumn("Visibility", clean_numeric("Visibility"))
    .withColumn("DryBulbFarenheit", clean_numeric("DryBulbFarenheit"))
    .withColumn("WetBulbFarenheit", clean_numeric("WetBulbFarenheit"))
    .withColumn("DewPointFarenheit", clean_numeric("DewPointFarenheit"))
    .withColumn("RelativeHumidity", clean_numeric("RelativeHumidity"))
    .withColumn("WindSpeed", clean_numeric("WindSpeed"))
    .withColumn("WindDirection", clean_numeric("WindDirection"))
    .withColumn("StationPressure", clean_numeric("StationPressure"))
    .withColumn("SeaLevelPressure", clean_numeric("SeaLevelPressure"))
    .withColumn("HourlyPrecip", clean_numeric("HourlyPrecip"))
    .withColumn("Altimeter", clean_numeric("Altimeter"))
)

weather_6.printSchema()
weather_6.show(5, truncate=False)
```

```
root
 |-- WBAN: string (nullable = true)
 |-- Date: string (nullable = true)
 |-- Time: string (nullable = true)
 |-- SkyCondition: string (nullable = true)
 |-- Visibility: double (nullable = true)
 |-- WeatherType: string (nullable = true)
 |-- DryBulbFarenheit: double (nullable = true)
 |-- WetBulbFarenheit: double (nullable = true)
 |-- DewPointFarenheit: double (nullable = true)
 |-- RelativeHumidity: double (nullable = true)
 |-- WindSpeed: double (nullable = true)
 |-- WindDirection: double (nullable = true)
 |-- StationPressure: double (nullable = true)
 |-- SeaLevelPressure: double (nullable = true)
 |-- HourlyPrecip: double (nullable = true)
 |-- Altimeter: double (nullable = true)
 |-- Date_str: string (nullable = true)
 |-- obs_date: date (nullable = true)
 |-- Time_4: string (nullable = true)
 |-- obs_hour: integer (nullable = true)
 |-- obs_minute: integer (nullable = true)
 |-- obs_ts_str: string (nullable = false)
 |-- obs_ts: timestamp (nullable = false)

+-----+--------+----+-----------+---------+----------+----------------+
----------------+-----------------+---------------+---------+-----------
-+--------------+---------------+-----------+---------+--------+-------
---+------+--------+----------+-------------------+-------------------+
|WBAN |Date    |Time|SkyCondition|Visibility|WeatherType|DryBulbFarenheit|
WetBulbFarenheit|DewPointFarenheit|RelativeHumidity|WindSpeed|WindDirectio
n|StationPressure|SeaLevelPressure|HourlyPrecip|Altimeter|Date_str|obs_dat
e  |Time_4|obs_hour|obs_minute|obs_ts_str         |obs_ts             |
+-----+--------+----+-----------+---------+----------+----------------+
----------------+-----------------+---------------+---------+-----------
-+--------------+---------------+-----------+---------+--------+-------
---+------+--------+----------+-------------------+-------------------+
|03011|20120101|0015|CLR        |10.0      |NULL       |23.0            |
15.0            |-9.0            |24.0           |5.0      |120.0
|21.7           |NULL            |NULL        |30.43    |20120101|2012-01-
01|0015  |0       |15        |2012-01-01 00:15:00|2012-01-01 00:15:00|
|03011|20120101|0035|CLR        |10.0      |NULL       |21.0            |
14.0            |-9.0            |26.0           |6.0      |130.0
|21.7           |NULL            |NULL        |30.43    |20120101|2012-01-
01|0035  |0       |35        |2012-01-01 00:35:00|2012-01-01 00:35:00|
|03011|20120101|0055|CLR        |10.0      |NULL       |21.0            |
13.0            |-13.0           |21.0           |0.0      |0.0
|21.71          |NULL            |NULL        |30.44    |20120101|2012-01-
01|0055  |0       |55        |2012-01-01 00:55:00|2012-01-01 00:55:00|
|03011|20120101|0115|CLR        |10.0      |NULL       |21.0            |
14.0            |-8.0            |27.0           |0.0      |0.0
|21.71          |NULL            |NULL        |30.44    |20120101|2012-01-
01|0115  |1       |15        |2012-01-01 01:15:00|2012-01-01 01:15:00|
|03011|20120101|0135|CLR        |10.0      |NULL       |21.0            |
13.0            |-11.0           |23.0           |0.0      |0.0
|21.72          |NULL            |NULL        |30.45    |20120101|2012-01-
01|0135  |1       |35        |2012-01-01 01:35:00|2012-01-01 01:35:00|
+-----+--------+----+-----------+---------+----------+----------------+
----------------+-----------------+---------------+---------+-----------
-+--------------+---------------+-----------+---------+--------+-------
```

```
---+------+--------+----------+------------------+------------------+
only showing top 5 rows
```

Join wban info

In [42]:
```
WBAN_PATH = "/content/drive/MyDrive/flight_delay_project/raw/wban_airport

wban_df = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(WBAN_PATH)
)

wban_df = (
    wban_df
    .select(
        F.trim(F.col("AirportID")).alias("AirportID"),
        F.lpad(F.trim(F.col("WBAN")), 5, "0").alias("WBAN"),
        F.trim(F.col("TimeZone")).alias("TimeZone")
    )
    .dropna(subset=["AirportID", "WBAN"])
)
```

In [43]:
```
weather_7 = (
    weather_6
    .join(wban_df, on="WBAN", how="inner")
)
```

Export to parquet

In [44]:
```
weather_8 = (
    weather_7
    .withColumn("weather_year", F.year("obs_date"))
    .withColumn("weather_month", F.month("obs_date"))
    .withColumn("weather_yyyymm", F.date_format("obs_date", "yyyyMM"))
)
```

In [45]:
```
weather_final_cols = [
    "WBAN",
    "AirportID",
    "TimeZone",
    "obs_date",
    "Time_4",
    "obs_hour",
    "obs_minute",
    "obs_ts",
    "SkyCondition",
    "Visibility",
    "WeatherType",
    "DryBulbFarenheit",
    "WetBulbFarenheit",
    "DewPointFarenheit",
    "RelativeHumidity",
    "WindSpeed",
    "WindDirection",
    "StationPressure",
```

```
    "SeaLevelPressure",
    "HourlyPrecip",
    "Altimeter",
    "weather_year",
    "weather_month",
    "weather_yyyymm"
]

weather_clean = weather_8.select(*weather_final_cols)
weather_clean.printSchema()
weather_clean.show(5, truncate=False)
```

```
root
 |-- WBAN: string (nullable = true)
 |-- AirportID: string (nullable = true)
 |-- TimeZone: string (nullable = true)
 |-- obs_date: date (nullable = true)
 |-- Time_4: string (nullable = true)
 |-- obs_hour: integer (nullable = true)
 |-- obs_minute: integer (nullable = true)
 |-- obs_ts: timestamp (nullable = false)
 |-- SkyCondition: string (nullable = true)
 |-- Visibility: double (nullable = true)
 |-- WeatherType: string (nullable = true)
 |-- DryBulbFarenheit: double (nullable = true)
 |-- WetBulbFarenheit: double (nullable = true)
 |-- DewPointFarenheit: double (nullable = true)
 |-- RelativeHumidity: double (nullable = true)
 |-- WindSpeed: double (nullable = true)
 |-- WindDirection: double (nullable = true)
 |-- StationPressure: double (nullable = true)
 |-- SeaLevelPressure: double (nullable = true)
 |-- HourlyPrecip: double (nullable = true)
 |-- Altimeter: double (nullable = true)
 |-- weather_year: integer (nullable = true)
 |-- weather_month: integer (nullable = true)
 |-- weather_yyyymm: string (nullable = true)

+-----+---------+--------+----------+------+--------+----------+----------
---------+------------+----------+-----------+----------------+-----------
-----+----------------+---------------+---------+------------+---------
------+----------------+------------+---------+------------+-------------+
-------------+
|WBAN |AirportID|TimeZone|obs_date  |Time_4|obs_hour|obs_minute|obs_ts
|SkyCondition|Visibility|WeatherType|DryBulbFarenheit|WetBulbFarenheit|Dew
PointFarenheit|RelativeHumidity|WindSpeed|WindDirection|StationPressure|Se
aLevelPressure|HourlyPrecip|Altimeter|weather_year|weather_month|weather_y
yyymm|
+-----+---------+--------+----------+------+--------+----------+----------
---------+------------+----------+-----------+----------------+-----------
-----+----------------+---------------+---------+------------+---------
------+----------------+------------+---------+------------+-------------+
-------------+
|03011|15177    |-7      |2012-01-01|0015  |0       |15        |2012-01-01
00:15:00|CLR         |10.0      |NULL       |23.0            |15.0
|-9.0             |24.0            |5.0      |120.0        |21.7
|NULL             |NULL        |30.43    |2012        |1            |201201
|
|03011|15177    |-7      |2012-01-01|0035  |0       |35        |2012-01-01
00:35:00|CLR         |10.0      |NULL       |21.0            |14.0
|-9.0             |26.0            |6.0      |130.0        |21.7
|NULL             |NULL        |30.43    |2012        |1            |201201
|
|03011|15177    |-7      |2012-01-01|0055  |0       |55        |2012-01-01
00:55:00|CLR         |10.0      |NULL       |21.0            |13.0
|-13.0            |21.0            |0.0      |0.0          |21.71
|NULL             |NULL        |30.44    |2012        |1            |201201
|
|03011|15177    |-7      |2012-01-01|0115  |1       |15        |2012-01-01
01:15:00|CLR         |10.0      |NULL       |21.0            |14.0
|-8.0             |27.0            |0.0      |0.0          |21.71
|NULL             |NULL        |30.44    |2012        |1            |201201
```

```
|
|03011|15177    |-7      |2012-01-01|0135  |1       |35        |2012-01-01
01:35:00|CLR          |10.0      |NULL       |21.0            |13.0
|-11.0            |23.0            |0.0      |0.0          |21.72
|NULL             |NULL        |30.45    |2012        |1            |201201
|
+-----+---------+--------+----------+------+--------+----------+----------
---------+-----------+----------+----------+---------------+----------
-----+----------------+---------------+---------+------------+---------
------+---------------+-----------+---------+-----------+------------+
--------------+
only showing top 5 rows
```

In [39]:
```
weather_clean.select(
    *[
        F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
        for c in [
            "AirportID", "obs_ts", "Visibility", "DryBulbFarenheit",
            "RelativeHumidity", "WindSpeed", "Altimeter"
        ]
    ]
).show(truncate=False)
```

```
+---------+------+----------+----------------+----------------+---------+-
--------+
|AirportID|obs_ts|Visibility|DryBulbFarenheit|RelativeHumidity|WindSpeed|A
ltimeter|
+---------+------+----------+----------------+----------------+---------+-
--------+
|0        |0     |74399     |9493            |104700          |70602    |7
5277    |
+---------+------+----------+----------------+----------------+---------+-
--------+
```

In [46]:
```
(
    weather_clean
    .write
    .mode("overwrite")
    .partitionBy("weather_year", "weather_month")
    .parquet(WEATHER_CLEAN_PATH)
)
```

In [ ]:

```
!pip install -q pyspark

from google.colab import drive
drive.mount('/content/drive')

from pyspark.sql import SparkSession, functions as F, Window
from datetime import datetime

spark = (
    SparkSession.builder
    .appName("Flight Delay Analysis")
    .config("spark.sql.shuffle.partitions", "200")
    .getOrCreate()
)

RUN_YEAR = 2013
RUN_MONTHS = [4, 6, 9, 11]

FLIGHT_CLEAN_PATH = "/content/drive/MyDrive/flight_delay_project/clean/Fl
WEATHER_CLEAN_PATH = "/content/drive/MyDrive/flight_delay_project/clean/W
JT_OUTPUT_PATH = "/content/drive/MyDrive/flight_delay_project/jt"

# ========================
# Functions
# ========================

def read_flight_month(year: int, month: int):
    path = f"{FLIGHT_CLEAN_PATH}/flight_year={year}/flight_month={month}"
    return spark.read.parquet(path)

def prev_month(year: int, month: int):
    if month == 1:
        return year - 1, 12
    return year, month - 1

def next_month(year: int, month: int):
    if month == 12:
        return year + 1, 1
    return year, month + 1

def weather_partition_path(year: int, month: int):
    return f"{WEATHER_CLEAN_PATH}/weather_year={year}/weather_month={mont

def path_exists(path: str):
    try:
        spark.read.parquet(path).limit(1).count()
        return True
    except Exception:
        return False

def read_weather_with_buffer(year: int, month: int):
    months = [prev_month(year, month), (year, month), next_month(year, mo
    dfs = []

    for y, m in months:
        p = weather_partition_path(y, m)
        if path_exists(p):
            dfs.append(spark.read.parquet(p))
```

```
    if not dfs:
        raise ValueError(f"No weather parquet found for buffered months a

    out = dfs[0]
    for d in dfs[1:]:
        out = out.unionByName(d)
    return out

lags = list(range(13))
lag_df = spark.createDataFrame([(i,) for i in lags], ["lag_hour"])

def make_targets(flights_df, airport_col: str, time_col: str):
    return (
        flights_df
        .select("flight_id", airport_col, time_col)
        .crossJoin(lag_df)
        .withColumnRenamed(airport_col, "airport_id")
        .withColumnRenamed(time_col, "base_ts")
        .withColumn(
            "target_ts",
            F.from_unixtime(
                F.unix_timestamp("base_ts") - F.col("lag_hour") * 3600
            ).cast("timestamp")
        )
    )

def build_weather_long(
    flights_df,
    weather_df,
    airport_col: str,
    time_col: str,
    prefix: str,
    window_hours: int = 1
):
    targets = make_targets(flights_df, airport_col, time_col)

    joined_candidates = (
        targets.alias("f")
        .join(
            weather_df.alias("w"),
            on="airport_id",
            how="left"
        )
        .withColumn(
            "time_diff_sec",
            F.abs(F.unix_timestamp("w.obs_ts") - F.unix_timestamp("f.targ
        )
        .filter(F.col("time_diff_sec") <= window_hours * 3600)
    )

    w = Window.partitionBy("flight_id", "lag_hour").orderBy(
        F.col("time_diff_sec").asc(),
        F.col("obs_ts").asc()
    )

    out = (
        joined_candidates
        .withColumn("rn", F.row_number().over(w))
        .filter(F.col("rn") == 1)
```

```
        .select(
            "flight_id",
            "airport_id",
            "lag_hour",
            "target_ts",
            F.col("obs_ts").alias(f"{prefix}_obs_ts"),
            "time_diff_sec",
            F.col("SkyCondition").alias(f"{prefix}_SkyCondition"),
            F.col("Visibility").alias(f"{prefix}_Visibility"),
            F.col("WeatherType").alias(f"{prefix}_WeatherType"),
            F.col("DryBulbFarenheit").alias(f"{prefix}_DryBulbFarenheit")
            F.col("WetBulbFarenheit").alias(f"{prefix}_WetBulbFarenheit")
            F.col("DewPointFarenheit").alias(f"{prefix}_DewPointFarenheit
            F.col("RelativeHumidity").alias(f"{prefix}_RelativeHumidity")
            F.col("WindSpeed").alias(f"{prefix}_WindSpeed"),
            F.col("WindDirection").alias(f"{prefix}_WindDirection"),
            F.col("StationPressure").alias(f"{prefix}_StationPressure"),
            F.col("SeaLevelPressure").alias(f"{prefix}_SeaLevelPressure")
            F.col("HourlyPrecip").alias(f"{prefix}_HourlyPrecip"),
            F.col("Altimeter").alias(f"{prefix}_Altimeter")
        )
    )

    return out

def long_to_wide(long_df, prefix: str):
    value_cols = [
        c for c in long_df.columns
        if c not in ["flight_id", "airport_id", "lag_hour", "target_ts"]
    ]

    agg_exprs = []
    for lag in range(13):
        for c in value_cols:
            agg_exprs.append(
                F.max(F.when(F.col("lag_hour") == lag, F.col(c))).alias(f
            )

    return long_df.groupBy("flight_id").agg(*agg_exprs)

def build_jt_month(year: int, month: int):
    flights_m = read_flight_month(year, month)
    weather_m = read_weather_with_buffer(year, month)

    flights_keep_cols = [
        "flight_id",
        "FL_DATE",
        "flight_yyyymm",
        "OP_CARRIER_AIRLINE_ID",
        "OP_CARRIER_FL_NUM",
        "ORIGIN_AIRPORT_ID",
        "DEST_AIRPORT_ID",
        "scheduled_dep_ts",
        "scheduled_arr_ts_dest_local",
        "ARR_DELAY_NEW",
        "arr_delay_min",
        "WEATHER_DELAY",
        "NAS_DELAY",
        "has_weather_delay",
        "has_nas_delay",
```

```
    "is_delayed_15",
    "is_delayed_60"
]

flights_m = flights_m.select(*flights_keep_cols)

flight_join_cols = [
    "flight_id",
    "ORIGIN_AIRPORT_ID",
    "DEST_AIRPORT_ID",
    "scheduled_dep_ts",
    "scheduled_arr_ts_dest_local"
]

flights_for_join = flights_m.select(*flight_join_cols)

weather_for_join = (
    weather_m
    .select(
        F.col("AirportID").alias("airport_id"),
        "obs_ts",
        "SkyCondition",
        "Visibility",
        "WeatherType",
        "DryBulbFarenheit",
        "WetBulbFarenheit",
        "DewPointFarenheit",
        "RelativeHumidity",
        "WindSpeed",
        "WindDirection",
        "StationPressure",
        "SeaLevelPressure",
        "HourlyPrecip",
        "Altimeter"
    )
    .filter(F.col("airport_id").isNotNull())
    .filter(F.col("obs_ts").isNotNull())
)

wo_long = build_weather_long(
    flights_df=flights_for_join,
    weather_df=weather_for_join,
    airport_col="ORIGIN_AIRPORT_ID",
    time_col="scheduled_dep_ts",
    prefix="wo",
    window_hours=1
)

wd_long = build_weather_long(
    flights_df=flights_for_join,
    weather_df=weather_for_join,
    airport_col="DEST_AIRPORT_ID",
    time_col="scheduled_arr_ts_dest_local",
    prefix="wd",
    window_hours=1
)

wo_wide = long_to_wide(wo_long, "wo")
wd_wide = long_to_wide(wd_long, "wd")
```

```
    jt_month = (
        flights_m
        .join(wo_wide, on="flight_id", how="left")
        .join(wd_wide, on="flight_id", how="left")
    )

    return jt_month

# ==========================
# Run for selected months
# ==========================
for run_month in RUN_MONTHS:
    print(f"Start processing {RUN_YEAR}-{run_month:02d} ...")

    jt_month = build_jt_month(RUN_YEAR, run_month)

    (
        jt_month
        .write
        .mode("overwrite")
        .parquet(f"{JT_OUTPUT_PATH}/year={RUN_YEAR}/month={run_month}")
    )

    print(f"Finished {RUN_YEAR}-{run_month:02d}")
```

In [ ]:

In [ ]:

```
!pip install -q pyspark

from google.colab import drive
drive.mount('/content/drive')

from pyspark.sql import SparkSession, functions as F
from functools import reduce

from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator, Multicla

# ============================================================
# 1. Spark session
# ============================================================
spark = (
    SparkSession.builder
    .appName("flight_delay_rf_simple")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

# ============================================================
# 2. Read JT parquet
# ============================================================
JT_PATH = "/content/drive/MyDrive/flight_delay_project/jt"

months_to_merge = [
    (2012, 1),
    (2012, 5),
    (2012, 7),
    (2012, 9),
    (2013, 4),
    (2013, 6),
    (2013, 9),
    (2013, 11),
]

dfs = []
for y, m in months_to_merge:
    path = f"{JT_PATH}/year={y}/month={m}"
    df = (
        spark.read.parquet(path)
        .withColumn("jt_year", F.lit(y))
        .withColumn("jt_month", F.lit(m))
    )
    dfs.append(df)

jt_all = reduce(lambda a, b: a.unionByName(b, allowMissingColumns=True),

print("Total rows:", jt_all.count())
print("Total columns:", len(jt_all.columns))
print("First 20 columns:", jt_all.columns[:20])

jt_all.groupBy("jt_year", "jt_month").count().orderBy("jt_year", "jt_mont
```

```
# ============================================================
# 3. Basic feature engineering
# ============================================================
jt_ml = (
    jt_all
    .withColumn("scheduled_dep_ts", F.to_timestamp("scheduled_dep_ts"))
    .withColumn("dep_year", F.year("scheduled_dep_ts"))
    .withColumn("dep_month", F.month("scheduled_dep_ts"))
    .withColumn("dep_day", F.dayofmonth("scheduled_dep_ts"))
    .withColumn("dep_dayofweek", F.dayofweek("scheduled_dep_ts"))
    .withColumn("dep_hour", F.hour("scheduled_dep_ts"))
    .withColumn("dep_minute", F.minute("scheduled_dep_ts"))
    .withColumn("is_weekend", F.when(F.dayofweek("scheduled_dep_ts").isin
)

target_col = "is_delayed_15"

base_feature_cols = [
    "OP_CARRIER_AIRLINE_ID",
    "OP_CARRIER_FL_NUM",
    "ORIGIN_AIRPORT_ID",
    "DEST_AIRPORT_ID",
    "flight_yyyymm",
    "jt_month",
    "dep_year",
    "dep_month",
    "dep_day",
    "dep_dayofweek",
    "dep_hour",
    "dep_minute",
    "is_weekend",
]

weather_num_patterns = [
    "_Visibility",
    "_DryBulbFarenheit",
    "_WetBulbFarenheit",
    "_DewPointFarenheit",
    "_RelativeHumidity",
    "_WindSpeed",
    "_WindDirection",
    "_StationPressure",
    "_SeaLevelPressure",
    "_HourlyPrecip",
    "_Altimeter",
    "time_diff_sec",
]

weather_feature_cols = [
    c for c in jt_ml.columns
    if any(p in c for p in weather_num_patterns)
]

feature_cols = base_feature_cols + weather_feature_cols

print("Number of weather features:", len(weather_feature_cols))
print("Total number of features:", len(feature_cols))
print("Sample weather features:", weather_feature_cols[:20])

jt_model = jt_ml.select(feature_cols + [target_col, "jt_year"])
```

```
print("Model rows:", jt_model.count())
print("Model cols:", len(jt_model.columns))
jt_model.show(3, truncate=False)

jt_model.groupBy(target_col).count().orderBy(target_col).show()

# ============================================================
# 4. Cast all feature cols + label to double
# ============================================================

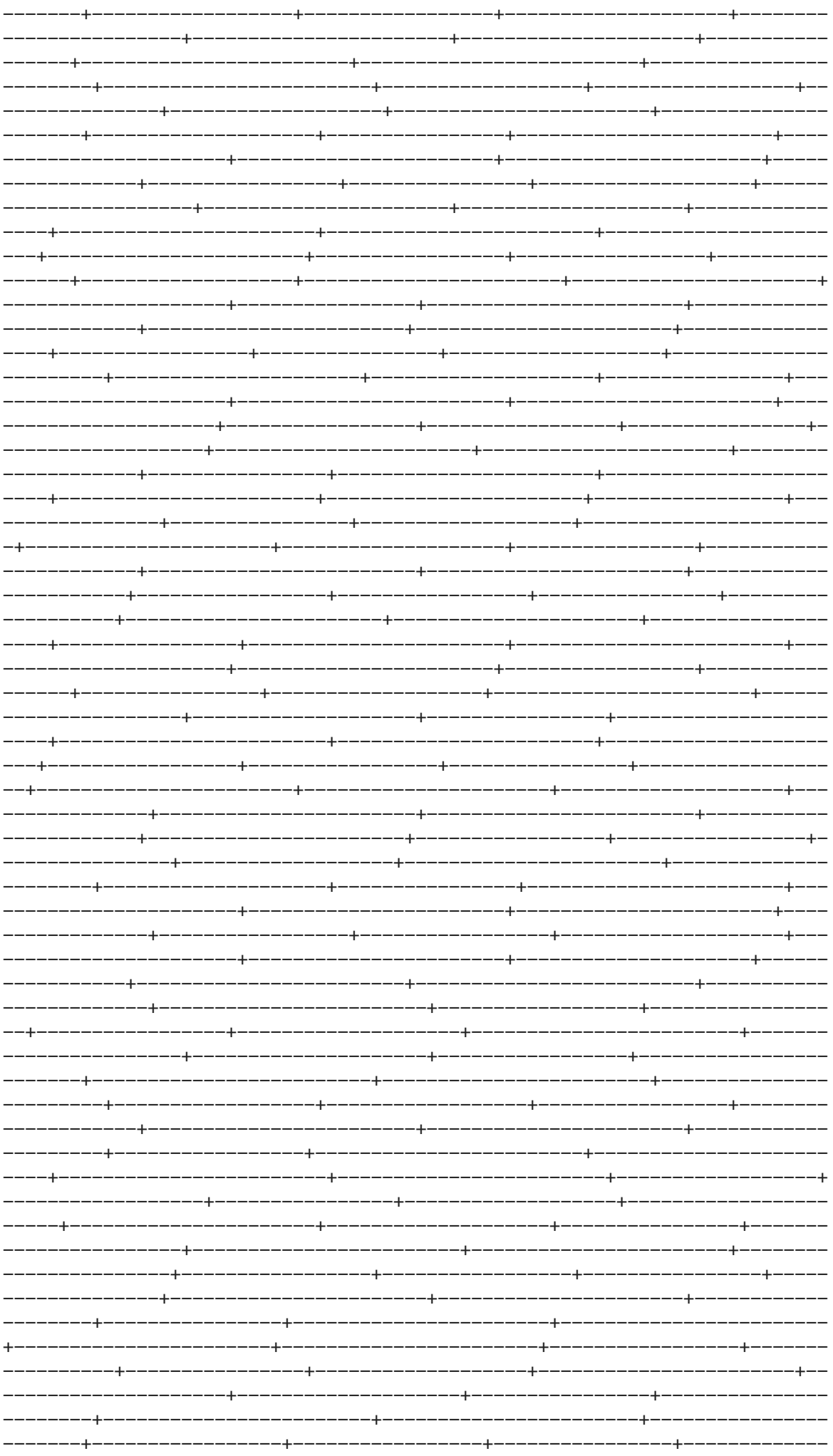
for c in feature_cols + [target_col]:
    jt_model = jt_model.withColumn(c, F.col(c).cast("double"))

jt_model = (
    jt_model
    .filter(F.col(target_col).isNotNull())
    .fillna(0)
)

print("After cast/fill:")
jt_model.printSchema()
```

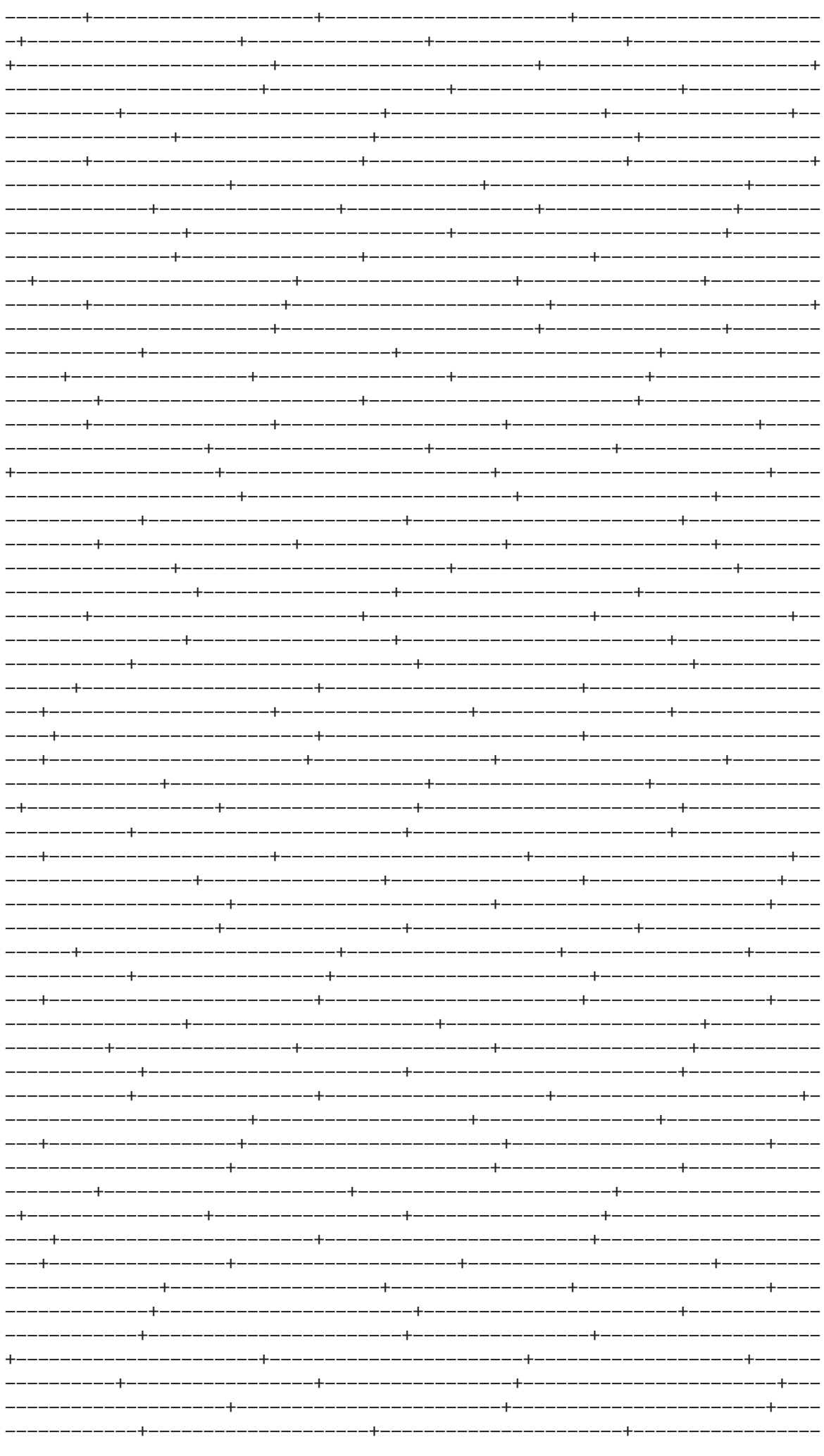
```
Drive already mounted at /content/drive; to attempt to forcibly remount, c
all drive.mount("/content/drive", force_remount=True).
Total rows: 3862206
Total columns: 409
First 20 columns: ['flight_id', 'FL_DATE', 'flight_yyyymm', 'OP_CARRIER_AI
RLINE_ID', 'OP_CARRIER_FL_NUM', 'ORIGIN_AIRPORT_ID', 'DEST_AIRPORT_ID', 's
cheduled_dep_ts', 'scheduled_arr_ts_dest_local', 'ARR_DELAY_NEW', 'arr_del
ay_min', 'WEATHER_DELAY', 'NAS_DELAY', 'has_weather_delay', 'has_nas_dela
y', 'is_delayed_15', 'is_delayed_60', 'wo0_wo_obs_ts', 'wo0_time_diff_se
c', 'wo0_wo_SkyCondition']
+-------+--------+------+
|jt_year|jt_month| count|
+-------+--------+------+
|   2012|       1|453790|
|   2012|       5|486160|
|   2012|       7|508379|
|   2012|       9|460343|
|   2013|       4|496752|
|   2013|       6|510129|
|   2013|       9|476721|
|   2013|      11|469932|
+-------+--------+------+

Number of weather features: 312
Total number of features: 325
Sample weather features: ['wo0_time_diff_sec', 'wo0_wo_Visibility', 'wo0_w
o_DryBulbFarenheit', 'wo0_wo_WetBulbFarenheit', 'wo0_wo_DewPointFarenhei
t', 'wo0_wo_RelativeHumidity', 'wo0_wo_WindSpeed', 'wo0_wo_WindDirection',
'wo0_wo_StationPressure', 'wo0_wo_SeaLevelPressure', 'wo0_wo_HourlyPreci
p', 'wo0_wo_Altimeter', 'wo1_time_diff_sec', 'wo1_wo_Visibility', 'wo1_wo_
DryBulbFarenheit', 'wo1_wo_WetBulbFarenheit', 'wo1_wo_DewPointFarenheit',
'wo1_wo_RelativeHumidity', 'wo1_wo_WindSpeed', 'wo1_wo_WindDirection']
Model rows: 3862206
Model cols: 327
+--------------------+----------------+----------------+--------------
+------------+--------+--------+---------+-------+------------+--------+
----------+----------+----------------+----------------+---------------
-------+---------------------+-----------------------+-----------------
------+---------------+--------------------+---------------------+------
----------------+-------------------+----------------+-----------------+-
---------------+---------------------+---------------------+---------
--------------+---------------------+---------------+-----------------
---+---------------------+---------------------+--------------------+---
------------+----------------+----------------+----------------------+
---------------------+---------------------+---------------------+-
--------------+--------------------+----------------+-----------------+---------
--------+-------------------+----------------+-----------------+---------
--------+---------------------+---------------------+-----------------
-------+---------------------+---------------+--------------------+----
-----------------+---------------------+-------------------+-----------
-----+----------------+----------------+----------------------+--------
--------------+---------------------+---------------------+---------
-------+--------------------+---------------------+---------------------
-+------------------+---------------+----------------+-----------------
+---------------------+---------------------+---------------------+
----------------------+---------------+-------------------+------------
----------+---------------------+-------------------+---------------+--
--------------+----------------+---------------------+---------------
-------+---------------------+---------------------+---------------+
-------------------+--------------------+---------------------+------
```



```
-----------+----------------------+----------------------+-----------
------------+----------------------+-----------------+----------------
----+----------------------+----------------------+-------------------
+-----------------+-----------------+-----------------+----------------
-------+----------------------+----------------------+---------------
---------+-----------------+--------------------+----------------------+
----------------------+--------------------+-----------------+----------
---+-------+
|OP_CARRIER_AIRLINE_ID|OP_CARRIER_FL_NUM|ORIGIN_AIRPORT_ID|DEST_AIRPORT_ID
|flight_yyyymm|jt_month|dep_year|dep_month|dep_day|dep_dayofweek|dep_hour|
dep_minute|is_weekend|wo0_time_diff_sec|wo0_wo_Visibility|wo0_wo_DryBulbFa
renheit|wo0_wo_WetBulbFarenheit|wo0_wo_DewPointFarenheit|wo0_wo_RelativeHu
midity|wo0_wo_WindSpeed|wo0_wo_WindDirection|wo0_wo_StationPressure|wo0_wo
_SeaLevelPressure|wo0_wo_HourlyPrecip|wo0_wo_Altimeter|wo1_time_diff_sec|w
o1_wo_Visibility|wo1_wo_DryBulbFarenheit|wo1_wo_WetBulbFarenheit|wo1_wo_De
wPointFarenheit|wo1_wo_RelativeHumidity|wo1_wo_WindSpeed|wo1_wo_WindDirect
ion|wo1_wo_StationPressure|wo1_wo_SeaLevelPressure|wo1_wo_HourlyPrecip|wo1
_wo_Altimeter|wo2_time_diff_sec|wo2_wo_Visibility|wo2_wo_DryBulbFarenheit|
wo2_wo_WetBulbFarenheit|wo2_wo_DewPointFarenheit|wo2_wo_RelativeHumidity|w
o2_wo_WindSpeed|wo2_wo_WindDirection|wo2_wo_StationPressure|wo2_wo_SeaLeve
lPressure|wo2_wo_HourlyPrecip|wo2_wo_Altimeter|wo3_time_diff_sec|wo3_wo_Vi
sibility|wo3_wo_DryBulbFarenheit|wo3_wo_WetBulbFarenheit|wo3_wo_DewPointFa
renheit|wo3_wo_RelativeHumidity|wo3_wo_WindSpeed|wo3_wo_WindDirection|wo3_
wo_StationPressure|wo3_wo_SeaLevelPressure|wo3_wo_HourlyPrecip|wo3_wo_Alti
meter|wo4_time_diff_sec|wo4_wo_Visibility|wo4_wo_DryBulbFarenheit|wo4_wo_W
etBulbFarenheit|wo4_wo_DewPointFarenheit|wo4_wo_RelativeHumidity|wo4_wo_Wi
ndSpeed|wo4_wo_WindDirection|wo4_wo_StationPressure|wo4_wo_SeaLevelPressur
e|wo4_wo_HourlyPrecip|wo4_wo_Altimeter|wo5_time_diff_sec|wo5_wo_Visibility
|wo5_wo_DryBulbFarenheit|wo5_wo_WetBulbFarenheit|wo5_wo_DewPointFarenheit|
wo5_wo_RelativeHumidity|wo5_wo_WindSpeed|wo5_wo_WindDirection|wo5_wo_Stati
onPressure|wo5_wo_SeaLevelPressure|wo5_wo_HourlyPrecip|wo5_wo_Altimeter|wo
6_time_diff_sec|wo6_wo_Visibility|wo6_wo_DryBulbFarenheit|wo6_wo_WetBulbFa
renheit|wo6_wo_DewPointFarenheit|wo6_wo_RelativeHumidity|wo6_wo_WindSpeed|
wo6_wo_WindDirection|wo6_wo_StationPressure|wo6_wo_SeaLevelPressure|wo6_wo
_HourlyPrecip|wo6_wo_Altimeter|wo7_time_diff_sec|wo7_wo_Visibility|wo7_wo_
DryBulbFarenheit|wo7_wo_WetBulbFarenheit|wo7_wo_DewPointFarenheit|wo7_wo_R
elativeHumidity|wo7_wo_WindSpeed|wo7_wo_WindDirection|wo7_wo_StationPressu
re|wo7_wo_SeaLevelPressure|wo7_wo_HourlyPrecip|wo7_wo_Altimeter|wo8_time_d
iff_sec|wo8_wo_Visibility|wo8_wo_DryBulbFarenheit|wo8_wo_WetBulbFarenheit|
wo8_wo_DewPointFarenheit|wo8_wo_RelativeHumidity|wo8_wo_WindSpeed|wo8_wo_W
indDirection|wo8_wo_StationPressure|wo8_wo_SeaLevelPressure|wo8_wo_HourlyP
recip|wo8_wo_Altimeter|wo9_time_diff_sec|wo9_wo_Visibility|wo9_wo_DryBulbF
arenheit|wo9_wo_WetBulbFarenheit|wo9_wo_DewPointFarenheit|wo9_wo_RelativeH
umidity|wo9_wo_WindSpeed|wo9_wo_WindDirection|wo9_wo_StationPressure|wo9_w
o_SeaLevelPressure|wo9_wo_HourlyPrecip|wo9_wo_Altimeter|wo10_time_diff_sec
|wo10_wo_Visibility|wo10_wo_DryBulbFarenheit|wo10_wo_WetBulbFarenheit|wo10
_wo_DewPointFarenheit|wo10_wo_RelativeHumidity|wo10_wo_WindSpeed|wo10_wo_W
indDirection|wo10_wo_StationPressure|wo10_wo_SeaLevelPressure|wo10_wo_Hour
lyPrecip|wo10_wo_Altimeter|wo11_time_diff_sec|wo11_wo_Visibility|wo11_wo_D
ryBulbFarenheit|wo11_wo_WetBulbFarenheit|wo11_wo_DewPointFarenheit|wo11_wo
_RelativeHumidity|wo11_wo_WindSpeed|wo11_wo_WindDirection|wo11_wo_StationP
ressure|wo11_wo_SeaLevelPressure|wo11_wo_HourlyPrecip|wo11_wo_Altimeter|wo
12_time_diff_sec|wo12_wo_Visibility|wo12_wo_DryBulbFarenheit|wo12_wo_WetBu
lbFarenheit|wo12_wo_DewPointFarenheit|wo12_wo_RelativeHumidity|wo12_wo_Win
dSpeed|wo12_wo_WindDirection|wo12_wo_StationPressure|wo12_wo_SeaLevelPress
ure|wo12_wo_HourlyPrecip|wo12_wo_Altimeter|wd0_time_diff_sec|wd0_wd_Visibi
lity|wd0_wd_DryBulbFarenheit|wd0_wd_WetBulbFarenheit|wd0_wd_DewPointFarenh
eit|wd0_wd_RelativeHumidity|wd0_wd_WindSpeed|wd0_wd_WindDirection|wd0_wd_S
tationPressure|wd0_wd_SeaLevelPressure|wd0_wd_HourlyPrecip|wd0_wd_Altimete
r|wd1_time_diff_sec|wd1_wd_Visibility|wd1_wd_DryBulbFarenheit|wd1_wd_WetBu
```

```
lbFarenheit|wd1_wd_DewPointFarenheit|wd1_wd_RelativeHumidity|wd1_wd_WindSp
eed|wd1_wd_WindDirection|wd1_wd_StationPressure|wd1_wd_SeaLevelPressure|wd
1_wd_HourlyPrecip|wd1_wd_Altimeter|wd2_time_diff_sec|wd2_wd_Visibility|wd2
_wd_DryBulbFarenheit|wd2_wd_WetBulbFarenheit|wd2_wd_DewPointFarenheit|wd2_
wd_RelativeHumidity|wd2_wd_WindSpeed|wd2_wd_WindDirection|wd2_wd_StationPr
essure|wd2_wd_SeaLevelPressure|wd2_wd_HourlyPrecip|wd2_wd_Altimeter|wd3_ti
me_diff_sec|wd3_wd_Visibility|wd3_wd_DryBulbFarenheit|wd3_wd_WetBulbFarenh
eit|wd3_wd_DewPointFarenheit|wd3_wd_RelativeHumidity|wd3_wd_WindSpeed|wd3_
wd_WindDirection|wd3_wd_StationPressure|wd3_wd_SeaLevelPressure|wd3_wd_Hou
rlyPrecip|wd3_wd_Altimeter|wd4_time_diff_sec|wd4_wd_Visibility|wd4_wd_DryB
ulbFarenheit|wd4_wd_WetBulbFarenheit|wd4_wd_DewPointFarenheit|wd4_wd_Relat
iveHumidity|wd4_wd_WindSpeed|wd4_wd_WindDirection|wd4_wd_StationPressure|w
d4_wd_SeaLevelPressure|wd4_wd_HourlyPrecip|wd4_wd_Altimeter|wd5_time_diff_
sec|wd5_wd_Visibility|wd5_wd_DryBulbFarenheit|wd5_wd_WetBulbFarenheit|wd5_
wd_DewPointFarenheit|wd5_wd_RelativeHumidity|wd5_wd_WindSpeed|wd5_wd_WindD
irection|wd5_wd_StationPressure|wd5_wd_SeaLevelPressure|wd5_wd_HourlyPreci
p|wd5_wd_Altimeter|wd6_time_diff_sec|wd6_wd_Visibility|wd6_wd_DryBulbFaren
heit|wd6_wd_WetBulbFarenheit|wd6_wd_DewPointFarenheit|wd6_wd_RelativeHumid
ity|wd6_wd_WindSpeed|wd6_wd_WindDirection|wd6_wd_StationPressure|wd6_wd_Se
aLevelPressure|wd6_wd_HourlyPrecip|wd6_wd_Altimeter|wd7_time_diff_sec|wd7_
wd_Visibility|wd7_wd_DryBulbFarenheit|wd7_wd_WetBulbFarenheit|wd7_wd_DewPo
intFarenheit|wd7_wd_RelativeHumidity|wd7_wd_WindSpeed|wd7_wd_WindDirection
|wd7_wd_StationPressure|wd7_wd_SeaLevelPressure|wd7_wd_HourlyPrecip|wd7_wd
_Altimeter|wd8_time_diff_sec|wd8_wd_Visibility|wd8_wd_DryBulbFarenheit|wd8
_wd_WetBulbFarenheit|wd8_wd_DewPointFarenheit|wd8_wd_RelativeHumidity|wd8_
wd_WindSpeed|wd8_wd_WindDirection|wd8_wd_StationPressure|wd8_wd_SeaLevelPr
essure|wd8_wd_HourlyPrecip|wd8_wd_Altimeter|wd9_time_diff_sec|wd9_wd_Visib
ility|wd9_wd_DryBulbFarenheit|wd9_wd_WetBulbFarenheit|wd9_wd_DewPointFaren
heit|wd9_wd_RelativeHumidity|wd9_wd_WindSpeed|wd9_wd_WindDirection|wd9_wd_
StationPressure|wd9_wd_SeaLevelPressure|wd9_wd_HourlyPrecip|wd9_wd_Altimet
er|wd10_time_diff_sec|wd10_wd_Visibility|wd10_wd_DryBulbFarenheit|wd10_wd_
WetBulbFarenheit|wd10_wd_DewPointFarenheit|wd10_wd_RelativeHumidity|wd10_w
d_WindSpeed|wd10_wd_WindDirection|wd10_wd_StationPressure|wd10_wd_SeaLevel
Pressure|wd10_wd_HourlyPrecip|wd10_wd_Altimeter|wd11_time_diff_sec|wd11_wd
_Visibility|wd11_wd_DryBulbFarenheit|wd11_wd_WetBulbFarenheit|wd11_wd_DewP
ointFarenheit|wd11_wd_RelativeHumidity|wd11_wd_WindSpeed|wd11_wd_WindDirec
tion|wd11_wd_StationPressure|wd11_wd_SeaLevelPressure|wd11_wd_HourlyPrecip
|wd11_wd_Altimeter|wd12_time_diff_sec|wd12_wd_Visibility|wd12_wd_DryBulbFa
renheit|wd12_wd_WetBulbFarenheit|wd12_wd_DewPointFarenheit|wd12_wd_Relativ
eHumidity|wd12_wd_WindSpeed|wd12_wd_WindDirection|wd12_wd_StationPressure|
wd12_wd_SeaLevelPressure|wd12_wd_HourlyPrecip|wd12_wd_Altimeter|is_delayed
_15|jt_year|
+--------------------+----------------+----------------+--------------
+------------+--------+--------+---------+-------+-------------+--------+
----------+----------+-----------------+-----------------+---------------
-------+----------------------+----------------------+----------------
------+----------------+--------------------+----------------------+------
-----------------+--------------------+---------------+-----------------+-
---------------+----------------------+----------------------+---------
--------------+----------------------+---------------+-----------------
---+----------------------+----------------------+--------------------+---
------------+-----------------+-----------------+----------------------+
----------------------+----------------------+----------------------+-
--------------+--------------------+----------------------+-------------
---------+--------------------+----------------+-----------------+---------
--------+----------------------+----------------------+-----------------
-------+----------------------+----------------+--------------------+----
------------------+----------------------+--------------------+-----------
-----+----------------+-----------------+----------------------+--------
--------------+----------------------+----------------------+---------
```



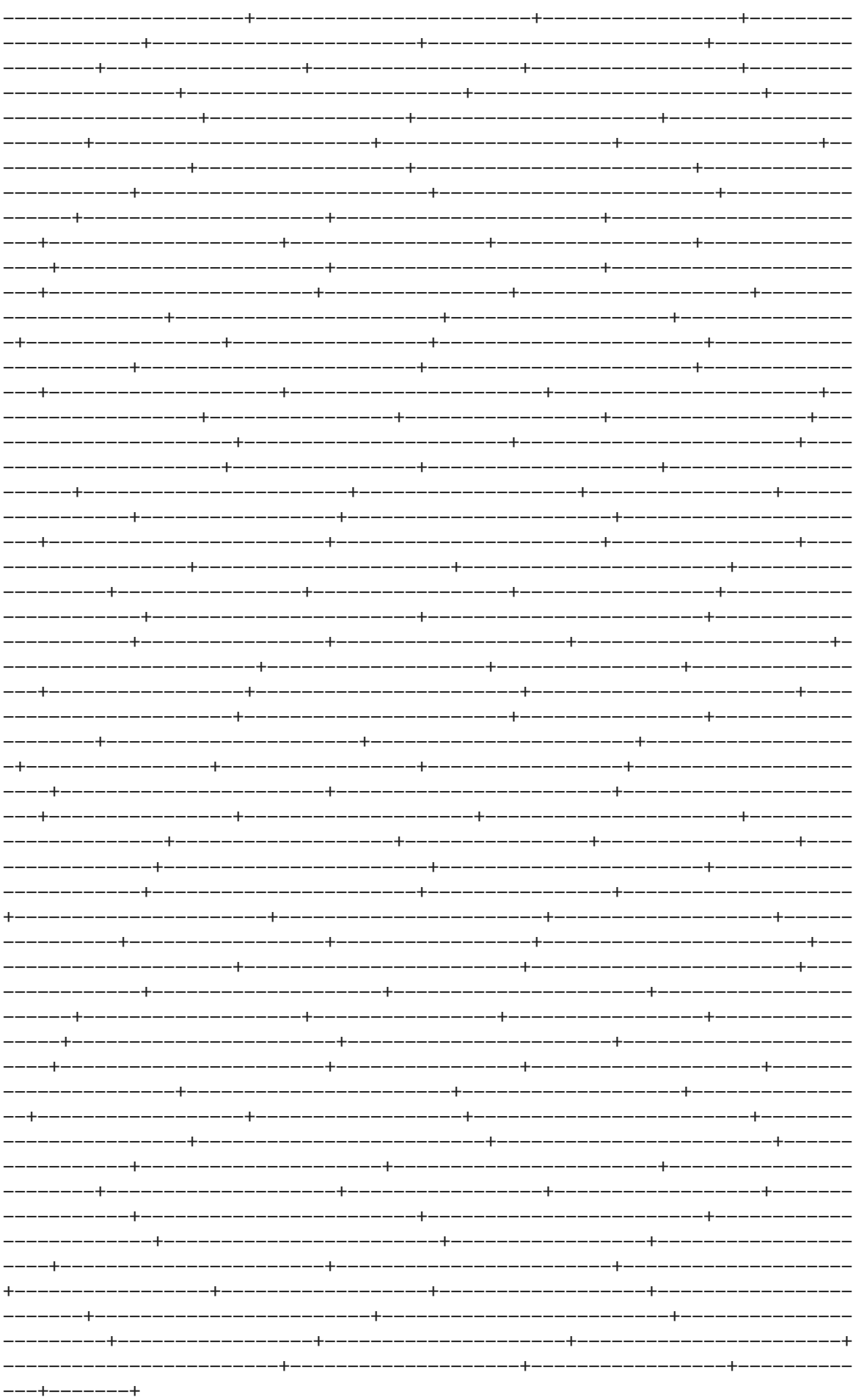
```
------+------------------+----------------+----------------+-----------
-----+--------------------+--------------------+------------------
----+--------------------+---------------+-------------------+-------
--------------+--------------------+-----------------+-------------
--+-----------------+-----------------+---------------------+--------
---------------+---------------------+---------------------+------
----------+-------------------+---------------------+---------------
--------+------------------+----------------+-----------------+-------
-----------+--------------------+---------------------+-----------
------------+--------------------+----------------+----------------
----+--------------------+--------------------+------------------
+-----------------+-----------------+-----------------+---------------
-------+---------------------+---------------------+--------------
---------+-----------------+-------------------+---------------------+
---------------------+------------------+----------------+----------
---+-------+
|20304                |5441             |14831            |11292
|201201       |1       |2012    |1        |13     |6            |17      |
50        |0         |180              |9.0              |57.0
|46.0                 |34.0                   |42.0                   |
3.0             |300.0              |30.11                |30.16
|NULL               |30.16           |180              |10.0             |
60.0                  |47.0                   |32.0                   |3
5.0                  |5.0            |310.0              |30.1
|30.15                |NULL              |30.15           |180
|10.0             |67.0                  |49.0                  |28.0
|23.0                 |0.0             |0.0                |30.1
|30.14                |NULL              |30.15           |180
|10.0             |66.0                  |49.0                  |28.0
|24.0                 |0.0             |0.0                |30.1
|30.14                |NULL              |30.15           |180
|10.0             |66.0                  |49.0                  |28.0
|24.0                 |0.0             |0.0                |30.1
|30.15                |NULL              |30.15           |180
|10.0             |63.0                  |48.0                  |29.0
|28.0                 |0.0             |0.0                |30.12
|30.17                |NULL              |30.17           |180
|10.0             |60.0                  |46.0                  |29.0
|31.0                 |3.0             |NULL               |30.16
|30.2                 |NULL              |30.21           |180
|10.0             |56.0                  |45.0                  |30.0
|37.0                 |0.0             |0.0                |30.18
|30.22                |NULL              |30.23           |180
|10.0             |50.0                  |42.0                  |32.0
|50.0                 |0.0             |0.0                |30.18
|30.23                |NULL              |30.23           |180
|10.0             |45.0                  |39.0                  |31.0
|58.0                 |0.0             |0.0                |30.17
|30.22                |NULL              |30.22           |180
|10.0              |39.0                  |36.0                   |31.0
|73.0                  |3.0             |160.0               |30.14
|30.19                |NULL              |30.19           |180
|10.0              |38.0                  |35.0                   |30.0
|73.0                 |0.0             |0.0                |30.13
|30.18                |NULL              |30.18           |180
|10.0              |39.0                  |35.0                   |30.0
|70.0                 |0.0             |0.0                |30.12
|30.17                |NULL              |30.17           |1680
|10.0             |37.0                  |27.0                  |8.0
|30.0                 |15.0            |240.0               |24.6
```

```
|30.05                  |NULL               |30.05           |1680
|10.0             |37.0                  |27.0                 |9.0
|31.0                   |18.0             |250.0                |24.61
|30.07                  |NULL               |30.06           |1680
|10.0             |37.0                  |27.0                 |9.0
|31.0                   |18.0             |260.0                |24.61
|30.07                  |NULL               |30.06           |1680
|10.0             |36.0                  |27.0                 |10.0
|34.0                   |13.0             |240.0                |24.61
|30.07                  |NULL               |30.06           |1680
|10.0             |40.0                  |29.0                 |8.0
|26.0                   |16.0             |280.0                |24.6
|30.06                  |NULL               |30.05           |1680
|10.0             |43.0                  |31.0                 |9.0
|25.0                   |24.0             |270.0                |24.56
|30.01                  |NULL               |30.0            |1680
|10.0             |43.0                  |32.0                 |12.0
|28.0                   |23.0             |280.0                |24.56
|30.01                  |NULL               |30.0            |1680
|10.0             |44.0                  |33.0                 |14.0
|30.0                   |15.0             |260.0                |24.55
|30.0                   |NULL               |29.99           |1680
|10.0             |41.0                  |30.0                 |12.0
|30.0                   |14.0             |270.0                |24.55
|30.01                  |NULL               |29.98           |1680
|10.0             |41.0                  |30.0                 |11.0
|29.0                   |17.0             |270.0                |24.54
|30.0                   |NULL               |29.97           |1680
|10.0              |40.0                   |29.0                  |10.0
|29.0                    |14.0            |260.0                |24.55
|30.02                   |NULL              |29.99           |1680
|10.0              |37.0                   |28.0                  |10.0
|33.0                   |9.0              |230.0                |24.56
|30.04                  |NULL              |30.0            |1680
|10.0              |34.0                   |25.0                  |6.0
|31.0                   |11.0             |270.0                |24.55
|30.03                  |NULL              |29.99           |0
|2012   |
|20304                |6381             |13342            |13930
|201201       |1       |2012    |1        |15     |1           |13      |
6         |1         |840              |9.0              |27.0
|24.0                  |18.0                    |69.0                   |
14.0           |170.0             |29.49                |30.28
|NULL               |30.24          |840              |10.0             |
26.0                  |23.0                    |16.0                   |6
6.0                  |9.0            |190.0              |29.53
|30.32                  |NULL              |30.28           |840
|10.0             |24.0                  |21.0                 |15.0
|68.0                   |13.0             |210.0                |29.56
|30.35                  |NULL               |30.31           |840
|9.0              |22.0                  |20.0                 |14.0
|71.0                   |8.0              |180.0                |29.53
|30.32                  |NULL               |30.28           |840
|8.0              |19.0                  |17.0                 |12.0
|74.0                   |6.0              |180.0                |29.54
|30.33                  |NULL               |30.29           |840
|8.0              |17.0                  |15.0                 |10.0
|74.0                   |6.0              |200.0                |29.54
|30.33                  |NULL               |30.29           |840
|10.0             |16.0                  |14.0                 |8.0
```

```
|71.0                  |7.0              |230.0              |29.55
|30.34                 |NULL               |30.3             |840
|10.0              |16.0                  |14.0                 |8.0
|71.0                  |7.0              |200.0              |29.52
|30.32                 |NULL               |30.27            |840
|9.0               |14.0                  |12.0                 |7.0
|74.0                  |7.0              |200.0              |29.51
|30.31                 |NULL               |30.26            |840
|9.0               |12.0                  |11.0                 |6.0
|77.0                  |5.0              |210.0              |29.52
|30.31                 |NULL               |30.27            |840
|9.0                |12.0                 |10.0                 |5.0
|73.0                   |3.0             |260.0               |29.51
|30.3                   |NULL              |30.26            |840
|8.0                |10.0                 |9.0                  |4.0
|76.0                   |3.0             |220.0               |29.5
|30.29                  |NULL              |30.25            |840
|8.0                |12.0                 |10.0                 |5.0
|73.0                   |3.0             |270.0               |29.48
|30.27                  |NULL              |30.23            |360
|10.0              |31.0                  |28.0                 |21.0
|66.0                  |10.0             |160.0              |29.51
|30.27                 |NULL               |30.24            |360
|10.0              |28.0                  |25.0                 |20.0
|72.0                  |16.0             |170.0              |29.53
|30.3                  |NULL               |30.26            |360
|10.0              |26.0                  |23.0                 |18.0
|72.0                  |13.0             |170.0              |29.58
|30.34                 |NULL               |30.31            |360
|10.0              |25.0                  |22.0                 |15.0
|66.0                  |14.0             |190.0              |29.6
|30.36                 |NULL               |30.33            |360
|10.0              |23.0                  |20.0                 |14.0
|68.0                  |8.0              |180.0              |29.6
|30.36                 |NULL               |30.33            |360
|10.0              |19.0                  |17.0                 |11.0
|71.0                  |8.0              |150.0              |29.6
|30.36                 |NULL               |30.33            |360
|10.0              |19.0                  |17.0                 |12.0
|74.0                  |7.0              |180.0              |29.61
|30.38                 |NULL               |30.34            |360
|10.0              |14.0                  |13.0                 |8.0
|77.0                  |6.0              |180.0              |29.59
|30.35                 |NULL               |30.32            |360
|9.0               |18.0                  |16.0                 |10.0
|71.0                  |5.0              |180.0              |29.57
|30.34                 |NULL               |30.3             |360
|8.0               |16.0                  |14.0                 |10.0
|77.0                  |5.0              |170.0              |29.57
|30.34                 |NULL               |30.3             |360
|8.0                |12.0                 |11.0                 |7.0
|80.0                   |3.0             |190.0               |29.57
|30.34                  |NULL              |30.3             |360
|9.0                |8.0                  |7.0                  |3.0
|80.0                   |0.0             |0.0                 |29.56
|30.33                  |NULL              |30.29            |360
|9.0                |12.0                 |11.0                 |6.0
|77.0                   |3.0             |230.0               |29.55
|30.32                  |NULL              |30.28            |0
|2012   |
```

```
|20436                |290              |11292            |15016
|201201       |1       |2012    |1         |2      |2            |15      |
0         |0         |420              |10.0             |51.0
|38.0                  |19.0                   |28.0                   |
7.0             |160.0              |24.88                |30.38
|NULL               |30.37           |420               |10.0             |
51.0                  |37.0                   |18.0                   |2
7.0                  |6.0            |140.0               |24.89
|30.4                   |NULL               |30.39           |420
|10.0             |50.0                 |37.0                  |20.0
|31.0                  |9.0              |140.0               |24.91
|30.42                 |NULL               |30.41           |420
|10.0             |48.0                 |37.0                  |21.0
|34.0                  |13.0             |150.0               |24.94
|30.47                 |NULL               |30.45           |420
|10.0             |44.0                 |35.0                  |23.0
|44.0                  |9.0              |150.0               |24.98
|30.54                 |NULL               |30.49           |420
|10.0             |41.0                 |34.0                  |23.0
|49.0                  |9.0              |160.0               |25.0
|30.58                 |NULL               |30.52           |420
|10.0             |31.0                 |27.0                  |20.0
|64.0                  |8.0              |170.0               |25.02
|30.63                 |NULL               |30.54           |420
|10.0             |27.0                 |23.0                  |14.0
|58.0                  |10.0             |210.0               |25.02
|30.63                 |NULL               |30.54           |420
|10.0             |22.0                 |19.0                  |12.0
|65.0                  |8.0              |200.0               |25.01
|30.65                 |NULL               |30.53           |420
|10.0             |23.0                 |20.0                  |13.0
|65.0                  |8.0              |210.0               |25.03
|30.65                 |NULL               |30.55           |420
|10.0              |22.0                  |19.0                  |14.0
|71.0                   |14.0            |220.0               |25.04
|30.66                  |NULL               |30.56           |420
|10.0              |21.0                  |18.0                  |13.0
|71.0                   |14.0            |220.0               |25.06
|30.66                  |NULL               |30.58           |420
|10.0              |22.0                  |19.0                  |14.0
|71.0                   |11.0            |220.0               |25.07
|30.66                  |NULL               |30.59           |300
|10.0             |27.0                 |21.0                  |7.0
|43.0                  |16.0             |320.0               |29.68
|30.49                 |NULL               |30.45           |300
|10.0             |28.0                 |22.0                  |8.0
|43.0                  |18.0             |340.0               |29.66
|30.46                 |NULL               |30.43           |300
|10.0             |31.0                 |24.0                  |8.0
|38.0                  |20.0             |340.0               |29.62
|30.42                 |NULL               |30.39           |300
|10.0             |33.0                 |25.0                  |6.0
|32.0                  |24.0             |320.0               |29.59
|30.39                 |NULL               |30.36           |300
|10.0             |34.0                 |26.0                  |8.0
|34.0                  |25.0             |310.0               |29.58
|30.38                 |NULL               |30.35           |300
|10.0             |33.0                 |26.0                  |9.0
|36.0                  |20.0             |300.0               |29.59
|30.39                 |NULL               |30.36           |300
```

```
|10.0             |32.0                  |25.0                  |9.0
|38.0                   |22.0            |320.0                |29.6
|30.4                   |NULL               |30.37             |300
|10.0             |31.0                  |24.0                  |8.0
|38.0                   |21.0            |300.0                |29.62
|30.42                  |NULL               |30.39             |300
|10.0             |29.0                  |23.0                  |10.0
|45.0                   |22.0            |310.0                |29.63
|30.43                  |NULL               |30.4              |300
|10.0             |29.0                  |24.0                  |12.0
|49.0                   |21.0            |300.0                |29.62
|30.42                  |NULL               |30.39             |300
|10.0              |28.0                   |23.0                     |12.0
|51.0                    |23.0             |310.0                 |29.59
|30.38                   |NULL                |30.36            |300
|10.0              |29.0                   |25.0                  |15.0
|56.0                   |18.0             |300.0                |29.57
|30.37                  |NULL                |30.34            |300
|10.0              |28.0                   |24.0                  |15.0
|58.0                   |20.0             |290.0                |29.56
|30.35                  |NULL                |30.33            |0
|2012   |
+-------------------+----------------+----------------+--------------
+------------+--------+--------+---------+-------+------------+--------+
----------+----------+----------------+----------------+---------------
-------+--------------------+--------------------+-----------------
------+---------------+-------------------+--------------------+------
----------------+-----------------+----------------+-----------------+-
---------------+--------------------+--------------------+---------
--------------+--------------------+----------------+-----------------
---+--------------------+--------------------+-----------------+---
------------+----------------+----------------+--------------------+
--------------------+--------------------+--------------------+-
--------------+-------------------+--------------------+-------------
--------+-----------------+---------------+-----------------+---------
--------+--------------------+--------------------+-----------------
-------+--------------------+---------------+--------------------+----
-----------------+--------------------+-----------------+------------
-----+----------------+----------------+--------------------+--------
--------------+--------------------+--------------------+---------
-------+-----------------+-------------------+--------------------+
-+-----------------+---------------+----------------+-----------------
+--------------------+--------------------+--------------------+
----------------------+---------------+--------------------+-----------
----------+--------------------+-----------------+----------------+--
--------------+----------------+--------------------+---------------
-------+--------------------+--------------------+----------------+
------------------+--------------------+---------------------+------
------------+----------------+----------------+----------------+-------
---------------+--------------------+--------------------+--------
--------------+----------------+-------------------+-----------------
--+--------------------+-----------------+----------------+----------
-------+----------------+--------------------+--------------------+
----------------------+--------------------+----------------+--------
------------+--------------------+--------------------+-------------
-----+----------------+----------------+----------------+--------------
--------+--------------------+--------------------+---------------
-------+----------------+-------------------+--------------------+-----
-----------------+-----------------+---------------+-----------------
+-----------------+--------------------+--------------------+----
```



only showing top 3 rows

|is_delayed_15| count|

```
|            0|3142239|
|            1| 719967|
+-------------+-------+

After cast/fill:
root
 |-- OP_CARRIER_AIRLINE_ID: double (nullable = false)
 |-- OP_CARRIER_FL_NUM: double (nullable = false)
 |-- ORIGIN_AIRPORT_ID: double (nullable = false)
 |-- DEST_AIRPORT_ID: double (nullable = false)
 |-- flight_yyyymm: double (nullable = false)
 |-- jt_month: double (nullable = false)
 |-- dep_year: double (nullable = false)
 |-- dep_month: double (nullable = false)
 |-- dep_day: double (nullable = false)
 |-- dep_dayofweek: double (nullable = false)
 |-- dep_hour: double (nullable = false)
 |-- dep_minute: double (nullable = false)
 |-- is_weekend: double (nullable = false)
 |-- wo0_time_diff_sec: double (nullable = false)
 |-- wo0_wo_Visibility: double (nullable = false)
 |-- wo0_wo_DryBulbFarenheit: double (nullable = false)
 |-- wo0_wo_WetBulbFarenheit: double (nullable = false)
 |-- wo0_wo_DewPointFarenheit: double (nullable = false)
 |-- wo0_wo_RelativeHumidity: double (nullable = false)
 |-- wo0_wo_WindSpeed: double (nullable = false)
 |-- wo0_wo_WindDirection: double (nullable = false)
 |-- wo0_wo_StationPressure: double (nullable = false)
 |-- wo0_wo_SeaLevelPressure: double (nullable = false)
 |-- wo0_wo_HourlyPrecip: double (nullable = false)
 |-- wo0_wo_Altimeter: double (nullable = false)
 |-- wo1_time_diff_sec: double (nullable = false)
 |-- wo1_wo_Visibility: double (nullable = false)
 |-- wo1_wo_DryBulbFarenheit: double (nullable = false)
 |-- wo1_wo_WetBulbFarenheit: double (nullable = false)
 |-- wo1_wo_DewPointFarenheit: double (nullable = false)
 |-- wo1_wo_RelativeHumidity: double (nullable = false)
 |-- wo1_wo_WindSpeed: double (nullable = false)
 |-- wo1_wo_WindDirection: double (nullable = false)
 |-- wo1_wo_StationPressure: double (nullable = false)
 |-- wo1_wo_SeaLevelPressure: double (nullable = false)
 |-- wo1_wo_HourlyPrecip: double (nullable = false)
 |-- wo1_wo_Altimeter: double (nullable = false)
 |-- wo2_time_diff_sec: double (nullable = false)
 |-- wo2_wo_Visibility: double (nullable = false)
 |-- wo2_wo_DryBulbFarenheit: double (nullable = false)
 |-- wo2_wo_WetBulbFarenheit: double (nullable = false)
 |-- wo2_wo_DewPointFarenheit: double (nullable = false)
 |-- wo2_wo_RelativeHumidity: double (nullable = false)
 |-- wo2_wo_WindSpeed: double (nullable = false)
 |-- wo2_wo_WindDirection: double (nullable = false)
 |-- wo2_wo_StationPressure: double (nullable = false)
 |-- wo2_wo_SeaLevelPressure: double (nullable = false)
 |-- wo2_wo_HourlyPrecip: double (nullable = false)
 |-- wo2_wo_Altimeter: double (nullable = false)
 |-- wo3_time_diff_sec: double (nullable = false)
 |-- wo3_wo_Visibility: double (nullable = false)
 |-- wo3_wo_DryBulbFarenheit: double (nullable = false)
 |-- wo3_wo_WetBulbFarenheit: double (nullable = false)
 |-- wo3_wo_DewPointFarenheit: double (nullable = false)
```

```
|-- wo3_wo_RelativeHumidity: double (nullable = false)
|-- wo3_wo_WindSpeed: double (nullable = false)
|-- wo3_wo_WindDirection: double (nullable = false)
|-- wo3_wo_StationPressure: double (nullable = false)
|-- wo3_wo_SeaLevelPressure: double (nullable = false)
|-- wo3_wo_HourlyPrecip: double (nullable = false)
|-- wo3_wo_Altimeter: double (nullable = false)
|-- wo4_time_diff_sec: double (nullable = false)
|-- wo4_wo_Visibility: double (nullable = false)
|-- wo4_wo_DryBulbFarenheit: double (nullable = false)
|-- wo4_wo_WetBulbFarenheit: double (nullable = false)
|-- wo4_wo_DewPointFarenheit: double (nullable = false)
|-- wo4_wo_RelativeHumidity: double (nullable = false)
|-- wo4_wo_WindSpeed: double (nullable = false)
|-- wo4_wo_WindDirection: double (nullable = false)
|-- wo4_wo_StationPressure: double (nullable = false)
|-- wo4_wo_SeaLevelPressure: double (nullable = false)
|-- wo4_wo_HourlyPrecip: double (nullable = false)
|-- wo4_wo_Altimeter: double (nullable = false)
|-- wo5_time_diff_sec: double (nullable = false)
|-- wo5_wo_Visibility: double (nullable = false)
|-- wo5_wo_DryBulbFarenheit: double (nullable = false)
|-- wo5_wo_WetBulbFarenheit: double (nullable = false)
|-- wo5_wo_DewPointFarenheit: double (nullable = false)
|-- wo5_wo_RelativeHumidity: double (nullable = false)
|-- wo5_wo_WindSpeed: double (nullable = false)
|-- wo5_wo_WindDirection: double (nullable = false)
|-- wo5_wo_StationPressure: double (nullable = false)
|-- wo5_wo_SeaLevelPressure: double (nullable = false)
|-- wo5_wo_HourlyPrecip: double (nullable = false)
|-- wo5_wo_Altimeter: double (nullable = false)
|-- wo6_time_diff_sec: double (nullable = false)
|-- wo6_wo_Visibility: double (nullable = false)
|-- wo6_wo_DryBulbFarenheit: double (nullable = false)
|-- wo6_wo_WetBulbFarenheit: double (nullable = false)
|-- wo6_wo_DewPointFarenheit: double (nullable = false)
|-- wo6_wo_RelativeHumidity: double (nullable = false)
|-- wo6_wo_WindSpeed: double (nullable = false)
|-- wo6_wo_WindDirection: double (nullable = false)
|-- wo6_wo_StationPressure: double (nullable = false)
|-- wo6_wo_SeaLevelPressure: double (nullable = false)
|-- wo6_wo_HourlyPrecip: double (nullable = false)
|-- wo6_wo_Altimeter: double (nullable = false)
|-- wo7_time_diff_sec: double (nullable = false)
|-- wo7_wo_Visibility: double (nullable = false)
|-- wo7_wo_DryBulbFarenheit: double (nullable = false)
|-- wo7_wo_WetBulbFarenheit: double (nullable = false)
|-- wo7_wo_DewPointFarenheit: double (nullable = false)
|-- wo7_wo_RelativeHumidity: double (nullable = false)
|-- wo7_wo_WindSpeed: double (nullable = false)
|-- wo7_wo_WindDirection: double (nullable = false)
|-- wo7_wo_StationPressure: double (nullable = false)
|-- wo7_wo_SeaLevelPressure: double (nullable = false)
|-- wo7_wo_HourlyPrecip: double (nullable = false)
|-- wo7_wo_Altimeter: double (nullable = false)
|-- wo8_time_diff_sec: double (nullable = false)
|-- wo8_wo_Visibility: double (nullable = false)
|-- wo8_wo_DryBulbFarenheit: double (nullable = false)
|-- wo8_wo_WetBulbFarenheit: double (nullable = false)
|-- wo8_wo_DewPointFarenheit: double (nullable = false)
```

```
|-- wo8_wo_RelativeHumidity: double (nullable = false)
|-- wo8_wo_WindSpeed: double (nullable = false)
|-- wo8_wo_WindDirection: double (nullable = false)
|-- wo8_wo_StationPressure: double (nullable = false)
|-- wo8_wo_SeaLevelPressure: double (nullable = false)
|-- wo8_wo_HourlyPrecip: double (nullable = false)
|-- wo8_wo_Altimeter: double (nullable = false)
|-- wo9_time_diff_sec: double (nullable = false)
|-- wo9_wo_Visibility: double (nullable = false)
|-- wo9_wo_DryBulbFarenheit: double (nullable = false)
|-- wo9_wo_WetBulbFarenheit: double (nullable = false)
|-- wo9_wo_DewPointFarenheit: double (nullable = false)
|-- wo9_wo_RelativeHumidity: double (nullable = false)
|-- wo9_wo_WindSpeed: double (nullable = false)
|-- wo9_wo_WindDirection: double (nullable = false)
|-- wo9_wo_StationPressure: double (nullable = false)
|-- wo9_wo_SeaLevelPressure: double (nullable = false)
|-- wo9_wo_HourlyPrecip: double (nullable = false)
|-- wo9_wo_Altimeter: double (nullable = false)
|-- wo10_time_diff_sec: double (nullable = false)
|-- wo10_wo_Visibility: double (nullable = false)
|-- wo10_wo_DryBulbFarenheit: double (nullable = false)
|-- wo10_wo_WetBulbFarenheit: double (nullable = false)
|-- wo10_wo_DewPointFarenheit: double (nullable = false)
|-- wo10_wo_RelativeHumidity: double (nullable = false)
|-- wo10_wo_WindSpeed: double (nullable = false)
|-- wo10_wo_WindDirection: double (nullable = false)
|-- wo10_wo_StationPressure: double (nullable = false)
|-- wo10_wo_SeaLevelPressure: double (nullable = false)
|-- wo10_wo_HourlyPrecip: double (nullable = false)
|-- wo10_wo_Altimeter: double (nullable = false)
|-- wo11_time_diff_sec: double (nullable = false)
|-- wo11_wo_Visibility: double (nullable = false)
|-- wo11_wo_DryBulbFarenheit: double (nullable = false)
|-- wo11_wo_WetBulbFarenheit: double (nullable = false)
|-- wo11_wo_DewPointFarenheit: double (nullable = false)
|-- wo11_wo_RelativeHumidity: double (nullable = false)
|-- wo11_wo_WindSpeed: double (nullable = false)
|-- wo11_wo_WindDirection: double (nullable = false)
|-- wo11_wo_StationPressure: double (nullable = false)
|-- wo11_wo_SeaLevelPressure: double (nullable = false)
|-- wo11_wo_HourlyPrecip: double (nullable = false)
|-- wo11_wo_Altimeter: double (nullable = false)
|-- wo12_time_diff_sec: double (nullable = false)
|-- wo12_wo_Visibility: double (nullable = false)
|-- wo12_wo_DryBulbFarenheit: double (nullable = false)
|-- wo12_wo_WetBulbFarenheit: double (nullable = false)
|-- wo12_wo_DewPointFarenheit: double (nullable = false)
|-- wo12_wo_RelativeHumidity: double (nullable = false)
|-- wo12_wo_WindSpeed: double (nullable = false)
|-- wo12_wo_WindDirection: double (nullable = false)
|-- wo12_wo_StationPressure: double (nullable = false)
|-- wo12_wo_SeaLevelPressure: double (nullable = false)
|-- wo12_wo_HourlyPrecip: double (nullable = false)
|-- wo12_wo_Altimeter: double (nullable = false)
|-- wd0_time_diff_sec: double (nullable = false)
|-- wd0_wd_Visibility: double (nullable = false)
|-- wd0_wd_DryBulbFarenheit: double (nullable = false)
|-- wd0_wd_WetBulbFarenheit: double (nullable = false)
|-- wd0_wd_DewPointFarenheit: double (nullable = false)
```

```
|-- wd0_wd_RelativeHumidity: double (nullable = false)
|-- wd0_wd_WindSpeed: double (nullable = false)
|-- wd0_wd_WindDirection: double (nullable = false)
|-- wd0_wd_StationPressure: double (nullable = false)
|-- wd0_wd_SeaLevelPressure: double (nullable = false)
|-- wd0_wd_HourlyPrecip: double (nullable = false)
|-- wd0_wd_Altimeter: double (nullable = false)
|-- wd1_time_diff_sec: double (nullable = false)
|-- wd1_wd_Visibility: double (nullable = false)
|-- wd1_wd_DryBulbFarenheit: double (nullable = false)
|-- wd1_wd_WetBulbFarenheit: double (nullable = false)
|-- wd1_wd_DewPointFarenheit: double (nullable = false)
|-- wd1_wd_RelativeHumidity: double (nullable = false)
|-- wd1_wd_WindSpeed: double (nullable = false)
|-- wd1_wd_WindDirection: double (nullable = false)
|-- wd1_wd_StationPressure: double (nullable = false)
|-- wd1_wd_SeaLevelPressure: double (nullable = false)
|-- wd1_wd_HourlyPrecip: double (nullable = false)
|-- wd1_wd_Altimeter: double (nullable = false)
|-- wd2_time_diff_sec: double (nullable = false)
|-- wd2_wd_Visibility: double (nullable = false)
|-- wd2_wd_DryBulbFarenheit: double (nullable = false)
|-- wd2_wd_WetBulbFarenheit: double (nullable = false)
|-- wd2_wd_DewPointFarenheit: double (nullable = false)
|-- wd2_wd_RelativeHumidity: double (nullable = false)
|-- wd2_wd_WindSpeed: double (nullable = false)
|-- wd2_wd_WindDirection: double (nullable = false)
|-- wd2_wd_StationPressure: double (nullable = false)
|-- wd2_wd_SeaLevelPressure: double (nullable = false)
|-- wd2_wd_HourlyPrecip: double (nullable = false)
|-- wd2_wd_Altimeter: double (nullable = false)
|-- wd3_time_diff_sec: double (nullable = false)
|-- wd3_wd_Visibility: double (nullable = false)
|-- wd3_wd_DryBulbFarenheit: double (nullable = false)
|-- wd3_wd_WetBulbFarenheit: double (nullable = false)
|-- wd3_wd_DewPointFarenheit: double (nullable = false)
|-- wd3_wd_RelativeHumidity: double (nullable = false)
|-- wd3_wd_WindSpeed: double (nullable = false)
|-- wd3_wd_WindDirection: double (nullable = false)
|-- wd3_wd_StationPressure: double (nullable = false)
|-- wd3_wd_SeaLevelPressure: double (nullable = false)
|-- wd3_wd_HourlyPrecip: double (nullable = false)
|-- wd3_wd_Altimeter: double (nullable = false)
|-- wd4_time_diff_sec: double (nullable = false)
|-- wd4_wd_Visibility: double (nullable = false)
|-- wd4_wd_DryBulbFarenheit: double (nullable = false)
|-- wd4_wd_WetBulbFarenheit: double (nullable = false)
|-- wd4_wd_DewPointFarenheit: double (nullable = false)
|-- wd4_wd_RelativeHumidity: double (nullable = false)
|-- wd4_wd_WindSpeed: double (nullable = false)
|-- wd4_wd_WindDirection: double (nullable = false)
|-- wd4_wd_StationPressure: double (nullable = false)
|-- wd4_wd_SeaLevelPressure: double (nullable = false)
|-- wd4_wd_HourlyPrecip: double (nullable = false)
|-- wd4_wd_Altimeter: double (nullable = false)
|-- wd5_time_diff_sec: double (nullable = false)
|-- wd5_wd_Visibility: double (nullable = false)
|-- wd5_wd_DryBulbFarenheit: double (nullable = false)
|-- wd5_wd_WetBulbFarenheit: double (nullable = false)
|-- wd5_wd_DewPointFarenheit: double (nullable = false)
```

```
|-- wd5_wd_RelativeHumidity: double (nullable = false)
|-- wd5_wd_WindSpeed: double (nullable = false)
|-- wd5_wd_WindDirection: double (nullable = false)
|-- wd5_wd_StationPressure: double (nullable = false)
|-- wd5_wd_SeaLevelPressure: double (nullable = false)
|-- wd5_wd_HourlyPrecip: double (nullable = false)
|-- wd5_wd_Altimeter: double (nullable = false)
|-- wd6_time_diff_sec: double (nullable = false)
|-- wd6_wd_Visibility: double (nullable = false)
|-- wd6_wd_DryBulbFarenheit: double (nullable = false)
|-- wd6_wd_WetBulbFarenheit: double (nullable = false)
|-- wd6_wd_DewPointFarenheit: double (nullable = false)
|-- wd6_wd_RelativeHumidity: double (nullable = false)
|-- wd6_wd_WindSpeed: double (nullable = false)
|-- wd6_wd_WindDirection: double (nullable = false)
|-- wd6_wd_StationPressure: double (nullable = false)
|-- wd6_wd_SeaLevelPressure: double (nullable = false)
|-- wd6_wd_HourlyPrecip: double (nullable = false)
|-- wd6_wd_Altimeter: double (nullable = false)
|-- wd7_time_diff_sec: double (nullable = false)
|-- wd7_wd_Visibility: double (nullable = false)
|-- wd7_wd_DryBulbFarenheit: double (nullable = false)
|-- wd7_wd_WetBulbFarenheit: double (nullable = false)
|-- wd7_wd_DewPointFarenheit: double (nullable = false)
|-- wd7_wd_RelativeHumidity: double (nullable = false)
|-- wd7_wd_WindSpeed: double (nullable = false)
|-- wd7_wd_WindDirection: double (nullable = false)
|-- wd7_wd_StationPressure: double (nullable = false)
|-- wd7_wd_SeaLevelPressure: double (nullable = false)
|-- wd7_wd_HourlyPrecip: double (nullable = false)
|-- wd7_wd_Altimeter: double (nullable = false)
|-- wd8_time_diff_sec: double (nullable = false)
|-- wd8_wd_Visibility: double (nullable = false)
|-- wd8_wd_DryBulbFarenheit: double (nullable = false)
|-- wd8_wd_WetBulbFarenheit: double (nullable = false)
|-- wd8_wd_DewPointFarenheit: double (nullable = false)
|-- wd8_wd_RelativeHumidity: double (nullable = false)
|-- wd8_wd_WindSpeed: double (nullable = false)
|-- wd8_wd_WindDirection: double (nullable = false)
|-- wd8_wd_StationPressure: double (nullable = false)
|-- wd8_wd_SeaLevelPressure: double (nullable = false)
|-- wd8_wd_HourlyPrecip: double (nullable = false)
|-- wd8_wd_Altimeter: double (nullable = false)
|-- wd9_time_diff_sec: double (nullable = false)
|-- wd9_wd_Visibility: double (nullable = false)
|-- wd9_wd_DryBulbFarenheit: double (nullable = false)
|-- wd9_wd_WetBulbFarenheit: double (nullable = false)
|-- wd9_wd_DewPointFarenheit: double (nullable = false)
|-- wd9_wd_RelativeHumidity: double (nullable = false)
|-- wd9_wd_WindSpeed: double (nullable = false)
|-- wd9_wd_WindDirection: double (nullable = false)
|-- wd9_wd_StationPressure: double (nullable = false)
|-- wd9_wd_SeaLevelPressure: double (nullable = false)
|-- wd9_wd_HourlyPrecip: double (nullable = false)
|-- wd9_wd_Altimeter: double (nullable = false)
|-- wd10_time_diff_sec: double (nullable = false)
|-- wd10_wd_Visibility: double (nullable = false)
|-- wd10_wd_DryBulbFarenheit: double (nullable = false)
|-- wd10_wd_WetBulbFarenheit: double (nullable = false)
|-- wd10_wd_DewPointFarenheit: double (nullable = false)
```

```
|-- wd10_wd_RelativeHumidity: double (nullable = false)
|-- wd10_wd_WindSpeed: double (nullable = false)
|-- wd10_wd_WindDirection: double (nullable = false)
|-- wd10_wd_StationPressure: double (nullable = false)
|-- wd10_wd_SeaLevelPressure: double (nullable = false)
|-- wd10_wd_HourlyPrecip: double (nullable = false)
|-- wd10_wd_Altimeter: double (nullable = false)
|-- wd11_time_diff_sec: double (nullable = false)
|-- wd11_wd_Visibility: double (nullable = false)
|-- wd11_wd_DryBulbFarenheit: double (nullable = false)
|-- wd11_wd_WetBulbFarenheit: double (nullable = false)
|-- wd11_wd_DewPointFarenheit: double (nullable = false)
|-- wd11_wd_RelativeHumidity: double (nullable = false)
|-- wd11_wd_WindSpeed: double (nullable = false)
|-- wd11_wd_WindDirection: double (nullable = false)
|-- wd11_wd_StationPressure: double (nullable = false)
|-- wd11_wd_SeaLevelPressure: double (nullable = false)
|-- wd11_wd_HourlyPrecip: double (nullable = false)
|-- wd11_wd_Altimeter: double (nullable = false)
|-- wd12_time_diff_sec: double (nullable = false)
|-- wd12_wd_Visibility: double (nullable = false)
|-- wd12_wd_DryBulbFarenheit: double (nullable = false)
|-- wd12_wd_WetBulbFarenheit: double (nullable = false)
|-- wd12_wd_DewPointFarenheit: double (nullable = false)
|-- wd12_wd_RelativeHumidity: double (nullable = false)
|-- wd12_wd_WindSpeed: double (nullable = false)
|-- wd12_wd_WindDirection: double (nullable = false)
|-- wd12_wd_StationPressure: double (nullable = false)
|-- wd12_wd_SeaLevelPressure: double (nullable = false)
|-- wd12_wd_HourlyPrecip: double (nullable = false)
|-- wd12_wd_Altimeter: double (nullable = false)
|-- is_delayed_15: double (nullable = false)
|-- jt_year: integer (nullable = false)
```

In [ ]:

```python
# ============================================================
# 5. Train / test split by year
# ============================================================
train_df = jt_model.filter(F.col("jt_year") == 2012)
test_df  = jt_model.filter(F.col("jt_year") == 2013)

print("Train rows before balance:", train_df.count())
print("Test rows before balance :", test_df.count())

train_df.groupBy(target_col).count().orderBy(target_col).show()
test_df.groupBy(target_col).count().orderBy(target_col).show()

# ============================================================
# 6. Simple undersampling on training set
# ============================================================
train_pos = train_df.filter(F.col(target_col) == 1.0)
train_neg = train_df.filter(F.col(target_col) == 0.0)

pos_count = train_pos.count()
neg_count = train_neg.count()

print("Positive count:", pos_count)
print("Negative count:", neg_count)

if pos_count == 0 or neg_count == 0:
```

```
    raise ValueError("Train set only has one class. Cannot train classifi

if pos_count < neg_count:
    frac = pos_count / neg_count
    train_neg_sample = train_neg.sample(withReplacement=False, fraction=f
    train_balanced = train_pos.unionByName(train_neg_sample)
else:
    frac = neg_count / pos_count
    train_pos_sample = train_pos.sample(withReplacement=False, fraction=f
    train_balanced = train_pos_sample.unionByName(train_neg)

train_balanced = train_balanced.orderBy(F.rand(42))

print("Train rows after balance:", train_balanced.count())
train_balanced.groupBy(target_col).count().orderBy(target_col).show()

# ============================================================
# 7. Final feature list
# ============================================================
# jt_year 不参与训练
final_feature_cols = [c for c in jt_model.columns if c not in [target_col

print("Number of final features:", len(final_feature_cols))
print("First 20 final features:", final_feature_cols[:20])
```

```
Train rows before balance: 1908672
Test rows before balance : 1953534
+-------------+-------+
|is_delayed_15|  count|
+-------------+-------+
|          0.0|1576042|
|          1.0| 332630|
+-------------+-------+

+-------------+-------+
|is_delayed_15|  count|
+-------------+-------+
|          0.0|1566197|
|          1.0| 387337|
+-------------+-------+

Positive count: 332630
Negative count: 1576042
Train rows after balance: 665727
+-------------+------+
|is_delayed_15| count|
+-------------+------+
|          0.0|333097|
|          1.0|332630|
+-------------+------+

Number of final features: 325
First 20 final features: ['OP_CARRIER_AIRLINE_ID', 'OP_CARRIER_FL_NUM', 'O
RIGIN_AIRPORT_ID', 'DEST_AIRPORT_ID', 'flight_yyyymm', 'jt_month', 'dep_ye
ar', 'dep_month', 'dep_day', 'dep_dayofweek', 'dep_hour', 'dep_minute', 'i
s_weekend', 'wo0_time_diff_sec', 'wo0_wo_Visibility', 'wo0_wo_DryBulbFaren
heit', 'wo0_wo_WetBulbFarenheit', 'wo0_wo_DewPointFarenheit', 'wo0_wo_Rela
tiveHumidity', 'wo0_wo_WindSpeed']
```

In [15]:
```
# ============================================================
# 8. Assemble + Random Forest
# ============================================================
assembler = VectorAssembler(
    inputCols=final_feature_cols,
    outputCol="features",
    handleInvalid="skip"
)

rf = RandomForestClassifier(
    labelCol=target_col,
    featuresCol="features",
    numTrees=50,
    maxDepth=8,
    seed=42
)

pipeline = Pipeline(stages=[assembler, rf])

# ============================================================
# 9. Train
# ============================================================
rf_model = pipeline.fit(train_balanced)
```

In [16]:
```
# ============================================================
# 10. Predict
# ============================================================
pred_df = rf_model.transform(test_df)

print("Prediction rows:", pred_df.count())

pred_df.select(target_col, "prediction", "probability").show(20, truncate
```

```
Prediction rows: 1953534
+-------------+----------+----------------------------------------+
|is_delayed_15|prediction|probability                             |
+-------------+----------+----------------------------------------+
|0.0          |1.0       |[0.40100396163675894,0.598996038363241] |
|1.0          |1.0       |[0.47673787330432327,0.5232621266956767]|
|0.0          |1.0       |[0.4268198234512485,0.5731801765487515] |
|0.0          |0.0       |[0.6089381426593262,0.3910618573406739] |
|0.0          |1.0       |[0.4690863264818089,0.530913673518191]  |
|1.0          |0.0       |[0.5900851925869197,0.40991480741308023]|
|0.0          |0.0       |[0.5133902148558858,0.48660978514411424]|
|0.0          |0.0       |[0.5226550530609884,0.4773449469390117] |
|0.0          |0.0       |[0.5659681029282148,0.43403189707178524]|
|0.0          |1.0       |[0.455689053822123,0.544310946177877]   |
|1.0          |0.0       |[0.6519239870513482,0.3480760129486518] |
|0.0          |0.0       |[0.635496608999704,0.3645033910002961]  |
|0.0          |1.0       |[0.45099370277485934,0.5490062972251406]|
|0.0          |1.0       |[0.3487292670424798,0.6512707329575201] |
|0.0          |0.0       |[0.540683316198399,0.4593166838016009]  |
|1.0          |0.0       |[0.6231788870377476,0.37682111296225235]|
|1.0          |0.0       |[0.5098269859753365,0.4901730140246635] |
|0.0          |0.0       |[0.6110403424336122,0.3889596575663879] |
|0.0          |0.0       |[0.5933911920357087,0.40660880796429133]|
|1.0          |0.0       |[0.5117252132652894,0.4882747867347106] |
+-------------+----------+----------------------------------------+
only showing top 20 rows
```

In [17]:

```
# ============================================================
# 11. Evaluate
# ============================================================
auc_eval = BinaryClassificationEvaluator(
    labelCol=target_col,
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

pr_eval = BinaryClassificationEvaluator(
    labelCol=target_col,
    rawPredictionCol="rawPrediction",
    metricName="areaUnderPR"
)

acc_eval = MulticlassClassificationEvaluator(
    labelCol=target_col,
    predictionCol="prediction",
    metricName="accuracy"
)

f1_eval = MulticlassClassificationEvaluator(
    labelCol=target_col,
    predictionCol="prediction",
    metricName="f1"
)

auc = auc_eval.evaluate(pred_df)
aupr = pr_eval.evaluate(pred_df)
acc = acc_eval.evaluate(pred_df)
f1 = f1_eval.evaluate(pred_df)

print("AUC  =", auc)
print("AUPR =", aupr)
print("ACC  =", acc)
print("F1   =", f1)

# ============================================================
# 12. Confusion matrix
# ============================================================
print("Confusion matrix:")
pred_df.groupBy(target_col, "prediction").count().orderBy(target_col, "pr

# ============================================================
# 13. Feature importance
# ============================================================
rf_stage = rf_model.stages[-1]
importances = rf_stage.featureImportances

feat_imp = list(zip(final_feature_cols, importances.toArray().tolist()))
feat_imp_sorted = sorted(feat_imp, key=lambda x: x[1], reverse=True)

print("Top 20 feature importances:")
for feat, score in feat_imp_sorted[:20]:
    print(f"{feat:40s} {score:.6f}")
```

```
AUC  = 0.6668681984753594
AUPR = 0.3448345969817578
ACC  = 0.6728063089764499
F1   = 0.7003336065481781
Confusion matrix:
+-------------+----------+-------+
|is_delayed_15|prediction|  count|
+-------------+----------+-------+
|          0.0|       0.0|1105082|
|          0.0|       1.0| 461115|
|          1.0|       0.0| 178069|
|          1.0|       1.0| 209268|
+-------------+----------+-------+

Top 20 feature importances:
dep_hour                                0.115704
wd0_wd_RelativeHumidity                 0.058892
wo5_wo_DryBulbFarenheit                 0.027702
wd1_wd_RelativeHumidity                 0.025478
wo0_wo_SeaLevelPressure                 0.023700
wo12_wo_RelativeHumidity                0.021999
wd7_wd_WetBulbFarenheit                 0.020044
wo4_wo_DryBulbFarenheit                 0.018687
wo1_wo_SeaLevelPressure                 0.018039
wo6_wo_DryBulbFarenheit                 0.017237
wd7_wd_DryBulbFarenheit                 0.017060
wo7_wo_DryBulbFarenheit                 0.015459
wo2_wo_SeaLevelPressure                 0.013339
wd0_wd_DryBulbFarenheit                 0.011623
wo5_wo_WetBulbFarenheit                 0.011174
wo7_wo_WetBulbFarenheit                 0.011015
wd1_wd_Visibility                       0.010843
OP_CARRIER_AIRLINE_ID                   0.010430
wo0_wo_RelativeHumidity                 0.010429
wd8_wd_DryBulbFarenheit                 0.009924
```

In [ ]:

In [ ]:

```
# ============================================================
# Regenerate all report figures (fixed arrow directions)
# ============================================================

!pip install -q pyspark

import os
import textwrap
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

from pyspark.sql import SparkSession, functions as F

# ------------------------------------------------------------
# Spark
# ------------------------------------------------------------
spark = (
    SparkSession.builder
    .appName("Generate Fixed Report Figures")
    .config("spark.sql.shuffle.partitions", "200")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")

from google.colab import drive
drive.mount('/content/drive')

# ------------------------------------------------------------
# Paths
# ------------------------------------------------------------
FLIGHT_RAW_PATH = "/content/drive/MyDrive/flight_delay_project/raw/Flight
WEATHER_RAW_PATH = "/content/drive/MyDrive/flight_delay_project/raw/Weath

FLIGHT_CLEAN_PATH = "/content/drive/MyDrive/flight_delay_project/clean/Fl
WEATHER_CLEAN_PATH = "/content/drive/MyDrive/flight_delay_project/clean/W
JT_OUTPUT_PATH = "/content/drive/MyDrive/flight_delay_project/jt"

FIG_DIR = "/content/drive/MyDrive/flight_delay_project/report_figures"
os.makedirs(FIG_DIR, exist_ok=True)

# ------------------------------------------------------------
# Helpers
# ------------------------------------------------------------
def save_fig(path):
    plt.tight_layout()
    plt.savefig(path, dpi=200, bbox_inches="tight")
    plt.close()

def make_canvas(figsize=(14, 4), title=None):
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    if title:
        ax.text(0.5, 0.95, title, ha="center", va="top", fontsize=18)
    return fig, ax
```

```
def add_box(ax, x, y, w, h, text, fontsize=11):
    # x, y are lower-left coordinates in [0,1]
    rect = Rectangle((x, y), w, h, fill=False, linewidth=1.2)
    ax.add_patch(rect)
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", fontsiz
    return {"x": x, "y": y, "w": w, "h": h}

def anchor(box, side):
    if side == "left":
        return (box["x"], box["y"] + box["h"] / 2)
    if side == "right":
        return (box["x"] + box["w"], box["y"] + box["h"] / 2)
    if side == "top":
        return (box["x"] + box["w"] / 2, box["y"] + box["h"])
    if side == "bottom":
        return (box["x"] + box["w"] / 2, box["y"])
    raise ValueError("side must be left/right/top/bottom")

def connect(ax, box1, side1, box2, side2, shrinkA=2, shrinkB=2):
    x1, y1 = anchor(box1, side1)
    x2, y2 = anchor(box2, side2)
    ax.annotate(
        "",
        xy=(x2, y2),
        xytext=(x1, y1),
        arrowprops=dict(
            arrowstyle="->",
            lw=1.5,
            shrinkA=shrinkA,
            shrinkB=shrinkB,
            connectionstyle="arc3,rad=0.0"
        )
    )

def parquet_exists(path):
    try:
        spark.read.parquet(path).limit(1).count()
        return True
    except:
        return False

# ------------------------------------------------------------
# Load data
# ------------------------------------------------------------
flights_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(f"{FLIGHT_RAW_PATH}/*")
)

weather_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", False)
    .csv(f"{WEATHER_RAW_PATH}/*")
)

flights_clean = spark.read.parquet(FLIGHT_CLEAN_PATH)
weather_clean = spark.read.parquet(WEATHER_CLEAN_PATH)
```

```
jt_dfs = []
for m in [4, 6, 9, 11]:
    p = f"{JT_OUTPUT_PATH}/year=2013/month={m}"
    if parquet_exists(p):
        jt_dfs.append(spark.read.parquet(p).withColumn("run_month", F.lit

jt_all = None
if jt_dfs:
    jt_all = jt_dfs[0]
    for d in jt_dfs[1:]:
        jt_all = jt_all.unionByName(d)
```

In [ ]:
```
# ============================================================
# FIG 01 overall pipeline
# ============================================================
fig, ax = make_canvas(figsize=(14, 4), title="Overall Pipeline")

raw_flights = add_box(ax, 0.02, 0.35, 0.16, 0.22, "Raw Flights CSV", 11)
raw_weather = add_box(ax, 0.23, 0.35, 0.16, 0.22, "Raw Weather CSV", 11)
mapping     = add_box(ax, 0.44, 0.35, 0.16, 0.22, "WBAN-Airport-TimeZone\

clean_f     = add_box(ax, 0.66, 0.54, 0.14, 0.18, "Clean Flights\n(Parque
clean_w     = add_box(ax, 0.66, 0.18, 0.14, 0.18, "Clean Weather\n(Parque

jt_box      = add_box(ax, 0.86, 0.35, 0.14, 0.22, "Monthly Joint Table\n(

connect(ax, raw_flights, "right", clean_f, "left")
connect(ax, raw_weather, "right", clean_w, "left")
connect(ax, mapping, "right", clean_f, "left")
connect(ax, mapping, "right", clean_w, "left")
connect(ax, clean_f, "right", jt_box, "left")
connect(ax, clean_w, "right", jt_box, "left")

save_fig(f"{FIG_DIR}/fig01_overall_pipeline.png")
```

In [ ]:
```
# ============================================================
# FIG 02 data source relationship
# ============================================================
fig, ax = make_canvas(figsize=(12, 5), title="Relationship Among Data Sou

flights_box = add_box(ax, 0.08, 0.58, 0.20, 0.18, "Flights\nAirport-based
weather_box = add_box(ax, 0.08, 0.16, 0.20, 0.18, "Weather\nWBAN-based",
map_box     = add_box(ax, 0.40, 0.37, 0.24, 0.22, "Mapping Table\nAirport
join_box    = add_box(ax, 0.76, 0.37, 0.16, 0.22, "Joinable Data", 11)

connect(ax, flights_box, "right", map_box, "left")
connect(ax, weather_box, "right", map_box, "left")
connect(ax, map_box, "right", join_box, "left")

save_fig(f"{FIG_DIR}/fig02_data_sources_relationship.png")
```

In [ ]:
```
# ============================================================
# FIG 03 flight cleaning counts
# ============================================================
raw_count = flights_raw.count()

flights_1 = flights_raw.drop("_c12")
flights_2 = (
```

```
    flights_1
    .withColumn("FL_DATE", F.to_date("FL_DATE", "yyyy-MM-dd"))
    .withColumn("CANCELLED", F.col("CANCELLED").cast("double"))
    .withColumn("DIVERTED", F.col("DIVERTED").cast("double"))
    .withColumn("CRS_ELAPSED_TIME", F.col("CRS_ELAPSED_TIME").cast("doubl
    .withColumn("CRS_DEP_TIME", F.col("CRS_DEP_TIME").cast("string"))
)
count_after_cast = flights_2.count()

flights_3 = (
    flights_2
    .filter(F.col("CANCELLED") == 0.0)
    .filter(F.col("DIVERTED") == 0.0)
    .filter(F.col("CRS_DEP_TIME").isNotNull())
    .filter(F.col("CRS_ELAPSED_TIME").isNotNull())
)
count_after_filter = flights_3.count()

flights_4 = (
    flights_3
    .withColumn("CRS_DEP_TIME_4", F.lpad(F.regexp_replace(F.col("CRS_DEP_
    .withColumn("dep_hour", F.substring("CRS_DEP_TIME_4", 1, 2).cast("int
    .withColumn("dep_minute", F.substring("CRS_DEP_TIME_4", 3, 2).cast("i
    .filter(F.col("dep_hour").between(0, 23))
    .filter(F.col("dep_minute").between(0, 59))
)
count_after_valid_time = flights_4.count()

count_final = flights_clean.count()

labels = [
    "Raw CSV",
    "After Cast",
    "After Filter\nCancelled/Diverted",
    "After Valid\nDep Time",
    "Final Clean"
]
values = [
    raw_count,
    count_after_cast,
    count_after_filter,
    count_after_valid_time,
    count_final
]

plt.figure(figsize=(12, 5))
plt.bar(labels, values)
plt.title("Flight Row Counts Across Cleaning Steps")
plt.ylabel("Row count")
for i, v in enumerate(values):
    plt.text(i, v, f"{v:,}", ha="center", va="bottom", fontsize=10)
save_fig(f"{FIG_DIR}/fig03_flight_cleaning_counts.png")
```

In [ ]:

```
# ============================================================
# FIG 04 scheduled departure timestamp construction
# ============================================================
fig, ax = make_canvas(figsize=(14, 4), title="Scheduled Departure Timesta

b1 = add_box(ax, 0.03, 0.32, 0.18, 0.22, "FL_DATE", 11)
b2 = add_box(ax, 0.27, 0.32, 0.18, 0.22, "CRS_DEP_TIME", 11)
```

```
b3 = add_box(ax, 0.51, 0.32, 0.18, 0.22, "Pad to 4 digits\nSplit hour/min
b4 = add_box(ax, 0.75, 0.32, 0.22, 0.22, "scheduled_dep_ts", 11)

connect(ax, b1, "right", b3, "left")
connect(ax, b2, "right", b3, "left")
connect(ax, b3, "right", b4, "left")

save_fig(f"{FIG_DIR}/fig04_scheduled_dep_ts_construction.png")
```

In [ ]:
```
# ============================================================
# FIG 05 arrival timestamp and time zone logic
# ============================================================
fig, ax = make_canvas(figsize=(14, 5), title="Arrival Timestamp and Time

d1 = add_box(ax, 0.03, 0.38, 0.18, 0.20, "scheduled_dep_ts", 11)
d2 = add_box(ax, 0.27, 0.38, 0.18, 0.20, "CRS_ELAPSED_TIME", 11)
d3 = add_box(ax, 0.51, 0.38, 0.20, 0.20, "scheduled_arr_ts\n_origin_clock

tz_o = add_box(ax, 0.80, 0.62, 0.14, 0.14, "origin_tz", 11)
tz_d = add_box(ax, 0.80, 0.22, 0.14, 0.14, "dest_tz", 11)
tz_x = add_box(ax, 0.80, 0.42, 0.16, 0.14, "tz_diff_hours", 11)
d4   = add_box(ax, 0.80, 0.02, 0.16, 0.14, "scheduled_arr_ts\n_dest_local

connect(ax, d1, "right", d3, "left")
connect(ax, d2, "right", d3, "left")
connect(ax, d3, "right", tz_x, "left")
connect(ax, tz_o, "bottom", tz_x, "top")
connect(ax, tz_d, "top", tz_x, "bottom")
connect(ax, tz_x, "bottom", d4, "top")

save_fig(f"{FIG_DIR}/fig05_arrival_ts_timezone_logic.png")
```

In [ ]:
```
# ============================================================
# FIG 06 clean flight schema
# ============================================================
fig, ax = make_canvas(figsize=(10, 8), title="Core Schema of Clean Flight

schema_text = """
flight_id
FL_DATE
flight_year, flight_month, flight_yyyymm
OP_CARRIER_AIRLINE_ID, OP_CARRIER_FL_NUM
ORIGIN_AIRPORT_ID, DEST_AIRPORT_ID
CRS_DEP_TIME, dep_hour, dep_minute
CRS_ELAPSED_TIME
scheduled_dep_ts
origin_tz, dest_tz, tz_diff_hours
scheduled_arr_ts_origin_clock
scheduled_arr_ts_dest_local
ARR_DELAY_NEW, arr_delay_min
WEATHER_DELAY, NAS_DELAY
has_weather_delay, has_nas_delay
is_delayed_15, is_delayed_60
""".strip()

add_box(ax, 0.15, 0.10, 0.70, 0.78, schema_text, 10)
save_fig(f"{FIG_DIR}/fig06_flight_output_schema.png")
```

In [ ]:
```
# ============================================================
# FIG 07 weather obs timestamp construction
# ============================================================
fig, ax = make_canvas(figsize=(14, 4), title="Weather Observation Timesta

w1 = add_box(ax, 0.03, 0.32, 0.18, 0.22, "Date", 11)
w2 = add_box(ax, 0.25, 0.32, 0.18, 0.22, "Time", 11)
w3 = add_box(ax, 0.47, 0.32, 0.20, 0.22, "Pad to 4 digits\nSplit hour/min
w4 = add_box(ax, 0.72, 0.32, 0.22, 0.22, "obs_ts", 11)

connect(ax, w1, "right", w3, "left")
connect(ax, w2, "right", w3, "left")
connect(ax, w3, "right", w4, "left")

save_fig(f"{FIG_DIR}/fig07_weather_obs_ts_construction.png")
```

In [ ]:
```
# ============================================================
# FIG 08 weather cleaning pipeline
# ============================================================
fig, ax = make_canvas(figsize=(16, 4), title="Weather Cleaning and Airpor

p1 = add_box(ax, 0.02, 0.32, 0.13, 0.20, "Raw Weather CSV", 10)
p2 = add_box(ax, 0.18, 0.32, 0.14, 0.20, "Trim strings\nReplace '' / M wi
p3 = add_box(ax, 0.36, 0.32, 0.12, 0.20, "Keep selected columns", 10)
p4 = add_box(ax, 0.52, 0.32, 0.16, 0.20, "Build obs_ts\nClean numeric fie
p5 = add_box(ax, 0.72, 0.32, 0.13, 0.20, "Join WBAN mapping", 10)
p6 = add_box(ax, 0.88, 0.32, 0.10, 0.20, "Clean Weather Parquet", 10)

connect(ax, p1, "right", p2, "left")
connect(ax, p2, "right", p3, "left")
connect(ax, p3, "right", p4, "left")
connect(ax, p4, "right", p5, "left")
connect(ax, p5, "right", p6, "left")

save_fig(f"{FIG_DIR}/fig08_weather_cleaning_pipeline.png")
```

In [ ]:
```
# ============================================================
# FIG 09 buffered weather read logic
# ============================================================
fig, ax = make_canvas(figsize=(12, 4), title="Buffered Weather Read Logic

m1 = add_box(ax, 0.10, 0.42, 0.18, 0.18, "Prev Month\nWeather", 11)
m2 = add_box(ax, 0.41, 0.42, 0.18, 0.18, "Current Month\nWeather", 11)
m3 = add_box(ax, 0.72, 0.42, 0.18, 0.18, "Next Month\nWeather", 11)
m4 = add_box(ax, 0.41, 0.08, 0.18, 0.16, "Buffered Weather\nfor Join", 11

connect(ax, m1, "bottom", m4, "top")
connect(ax, m2, "bottom", m4, "top")
connect(ax, m3, "bottom", m4, "top")

save_fig(f"{FIG_DIR}/fig09_weather_buffer_logic.png")
```

In [ ]:
```
# ============================================================
# FIG 10 weather matching logic
# ============================================================
fig, ax = make_canvas(figsize=(14, 5), title="Flight Target Time and Weat

q1 = add_box(ax, 0.04, 0.36, 0.18, 0.20, "flight_id\nairport_id\nbase_ts"
```

```
q2 = add_box(ax, 0.28, 0.36, 0.16, 0.20, "Cross join\nlag_hour = 0..12",
q3 = add_box(ax, 0.50, 0.36, 0.18, 0.20, "target_ts =\nbase_ts - lag*1h",
q4 = add_box(ax, 0.74, 0.36, 0.18, 0.20, "Join weather by\nairport_id", 1
q5 = add_box(ax, 0.74, 0.08, 0.18, 0.16, "Keep nearest obs\nwithin 1 hour

connect(ax, q1, "right", q2, "left")
connect(ax, q2, "right", q3, "left")
connect(ax, q3, "right", q4, "left")
connect(ax, q4, "bottom", q5, "top")

save_fig(f"{FIG_DIR}/fig10_weather_matching_logic.png")
```

In [ ]:
```
# ============================================================
# FIG 11 final JT structure
# ============================================================
fig, ax = make_canvas(figsize=(12, 5), title="Structure of Final Monthly

j1 = add_box(ax, 0.06, 0.35, 0.20, 0.18, "flights_m\n(main flight table)"
j2 = add_box(ax, 0.38, 0.60, 0.22, 0.14, "wo_wide\n(origin weather wide)"
j3 = add_box(ax, 0.38, 0.16, 0.22, 0.14, "wd_wide\n(destination weather w
j4 = add_box(ax, 0.74, 0.35, 0.20, 0.18, "jt_month\n(final joint table)",

connect(ax, j1, "right", j4, "left")
connect(ax, j2, "right", j4, "top")
connect(ax, j3, "right", j4, "bottom")

save_fig(f"{FIG_DIR}/fig11_final_jt_structure.png")
```

In [ ]:
```
# ============================================================
# FIG 12 code structure
# ============================================================
fig, ax = make_canvas(figsize=(14, 5), title="Code Structure and Output D

c1 = add_box(ax, 0.04, 0.55, 0.18, 0.16, "Part 1:\nFlight Cleaning", 11)
c2 = add_box(ax, 0.04, 0.28, 0.18, 0.16, "Part 2:\nWeather Cleaning", 11)
c3 = add_box(ax, 0.35, 0.42, 0.18, 0.16, "Part 3:\nJT Construction", 11)

o1 = add_box(ax, 0.68, 0.62, 0.20, 0.12, "clean/Flights", 10)
o2 = add_box(ax, 0.68, 0.40, 0.20, 0.12, "clean/Weather", 10)
o3 = add_box(ax, 0.68, 0.18, 0.20, 0.12, "jt/year=2013/month=...", 10)

connect(ax, c1, "right", o1, "left")
connect(ax, c2, "right", o2, "left")
connect(ax, c1, "right", c3, "left")
connect(ax, c2, "right", c3, "left")
connect(ax, c3, "right", o3, "left")

save_fig(f"{FIG_DIR}/fig12_code_structure.png")
```

In [ ]:
```
# ============================================================
# FIG 13 future work
# ============================================================
fig, ax = make_canvas(figsize=(13, 6), title="Current Achievements, Limit

# Top center: achieved result
f0 = add_box(
    ax, 0.33, 0.72, 0.34, 0.16,
    "Current Achievements\n\n"
    "- Flight data cleaned and structured\n"
```

```
    "- Weather data cleaned and mapped to airports\n"
    "- Monthly joint tables successfully constructed",
    10
)

# Bottom left: limitations
f1 = add_box(
    ax, 0.07, 0.30, 0.34, 0.24,
    "Current Limitations\n\n"
    "- Only selected months were run in the JT stage\n"
    "- Weather matching keeps only the nearest record within 1 hour\n"
    "- Runtime cost is high in weather matching, reshaping, and writing\n
    "- Downstream prediction is only validated with a simple baseline",
    10
)

# Bottom right: future work
f2 = add_box(
    ax, 0.59, 0.30, 0.34, 0.24,
    "Future Extensions\n\n"
    "- Run more months or larger time ranges\n"
    "- Analyze missing rates after weather matching\n"
    "- Compare alternative weather aggregation strategies\n"
    "- Train and evaluate stronger prediction models\n"
    "- Further optimize Spark execution and output stages",
    10
)

# Middle small connector / interpretation
f3 = add_box(
    ax, 0.38, 0.08, 0.24, 0.11,
    "Overall Message\n\n"
    "The data-preparation pipeline is complete,\n"
    "but scalability and modeling can be further improved.",
    10
)

connect(ax, f0, "bottom", f1, "top")
connect(ax, f0, "bottom", f2, "top")
connect(ax, f1, "bottom", f3, "top")
connect(ax, f2, "bottom", f3, "top")

save_fig(f"{FIG_DIR}/fig13_future_work.png")

print("Saved:", f"{FIG_DIR}/fig13_future_work.png")
```